

ภาควิชาเทคโนโลยีสารสนเทศ

โครงการปลายภาค

Full Stack Web Development

ออกแบบและพัฒนาเว็บแอปพลิเคชันที่ขับเคลื่อนด้วยฐานข้อมูล

Express.js | EJS | Sequelize | PostgreSQL | CSS

ระบบบริหารจัดการยิม (GymBro Management System)

นายเมธัส ทองจันทร์	6606022610030
นายณพคุณ เหล่าอิมจันทร์	6606022610013
นายกาญจน์ชญา สู้สุข	6606022610048
นางสาวทิพย์สุดา สังข์เงิน	6606022620060

ปีการศึกษา 2568 ภาคเรียนที่ 2
กำหนดส่ง: 9 มีนาคม 2569

Contents

1	บทนำ	4
1.1	ที่มาและความสำคัญของโครงงาน (Background and Significance)	4
1.2	วัตถุประสงค์ของโครงงาน (Objectives)	4
1.3	ขอบเขตของโครงงาน (Scope)	4
1.3.1	ส่วนของผู้ดูแลระบบ (Administrator)	4
1.3.2	ส่วนของผู้สมาชิก (Member)	5
1.4	เป้าหมายของโครงงาน (Goals)	5
1.5	ประโยชน์ที่คาดว่าจะได้รับ (Expected Benefits)	5
2	ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Literature Review and Related Technologies)	6
2.1	สถาปัตยกรรมแบบไคลเอนต์-เซิร์ฟเวอร์ (Client-Server Architecture)	6
2.1.1	หลักการทำงาน (Working Principle)	6
2.1.2	การสื่อสารผ่านโปรโตคอล HTTP (Communication via HTTP Protocol)	6
2.1.3	วงจรชีวิตของเว็บรีเควส (Lifecycle of a Web Request)	6
2.2	การออกแบบและพัฒนา RESTful API (RESTful API Design & Development)	7
2.2.1	หลักการของ REST (Principles of REST)	7
2.2.2	HTTP Methods มาตรฐาน (Standard HTTP Methods)	7
2.2.3	รหัสสถานะ HTTP (HTTP Status Codes)	7
2.3	Node.js	8
2.3.1	ประวัติและวิวัฒนาการ (History and Evolution)	8
2.3.2	สถาปัตยกรรมแบบขับเคลื่อนด้วยเหตุการณ์ (Event-Driven Architecture)	8
2.3.3	Single-Threaded, Non-blocking I/O	8
2.4	Express.js	8
2.4.1	บทบาทในฐานะ Web Framework	8
2.4.2	การจัดการเส้นทาง (Routing Management)	8
2.4.3	Middleware Pipeline	9
2.5	ระบบจัดการฐานข้อมูล PostgreSQL (PostgreSQL Database System)	9
2.5.1	คุณลักษณะของ RDBMS (RDBMS Characteristics)	9
2.5.2	โครงสร้างข้อมูลและประสิทธิภาพ (Data Structuring and Performance)	9
2.6	การเชื่อมโยงข้อมูลเชิงวัตถุสัมพันธ์ (ORM) และ Sequelize	9
2.6.1	Impedance Mismatch	9
2.6.2	การทำงานของ Sequelize (Sequelize Operations)	10
2.7	EJS (Embedded JavaScript templating)	10
2.7.1	การเรนเดอร์ฝั่งเซิร์ฟเวอร์ (Server-Side Rendering - SSR)	10
2.7.2	ข้อดีของ SSR ด้วย EJS (Advantages of SSR with EJS)	10
3	การออกแบบพฤติกรรมระบบ (Behavioral System Design - UML Diagrams)	11
3.1	แผนภาพยูสเคสและคำอธิบายอย่างละเอียด (Use Case Diagram & Detailed Descriptions)	11
3.1.1	ผู้กระทำ (Actors)	11
3.1.2	รายการยูสเคส (Use Case List)	11
3.1.3	คำอธิบายยูสเคสอย่างละเอียด (Detailed Use Case Descriptions)	12
3.2	แผนภาพกิจกรรม (Activity Diagrams)	15
3.2.1	กระบวนการจองและการตรวจสอบเวลาซ้อนทับ (Booking & Time Conflict Detection Flow)	15
3.2.2	กระบวนการจัดการข้อมูลของผู้ดูแล (Data Management Flow)	15
3.3	แผนภาพลำดับ (Sequence Diagrams)	17
3.3.1	กระบวนการเข้าสู่ระบบของผู้ใช้ (The User Login Process)	17
3.3.2	กระบวนการจอง (The Booking Process)	17

4	การวิเคราะห์และออกแบบระบบ	19
4.1	สถาปัตยกรรมระบบ (System Architecture)	19
4.1.1	ภาพรวมสถาปัตยกรรม (Architecture Overview)	19
4.2	ความต้องการของระบบ (System Requirements)	19
4.2.1	ความต้องการฟังก์ชันการทำงาน (Functional Requirements)	19
4.2.2	ความต้องการที่ไม่ใช่ฟังก์ชันการทำงาน (Non-Functional Requirements)	21
5	ER Diagram	22
5.1	คำอธิบายความสัมพันธ์ (Relationships Explanation)	23
5.2	รายละเอียดโครงสร้างตาราง (Table Schema Detail)	23
5.2.1	1. ตาราง Customers (ข้อมูลสมาชิกและโปรไฟล์ผู้ใช้)	23
5.2.2	2. ตาราง Trainers (ข้อมูลเทรนเนอร์)	23
5.2.3	3. ตาราง GymEquipments (ข้อมูลอุปกรณ์)	24
5.2.4	4. ตาราง TrainingSessions (ตารางการจองฝึกซ้อม)	24
6	ตาราง Database	25
6.1	Database (ฐานข้อมูล)	25
6.1.1	ภาพรวมฐานข้อมูล (Database Overview)	25
6.1.2	การเชื่อมต่อฐานข้อมูล (Database Connection)	25
6.1.3	db.js	25
6.1.4	schema.sql	27
6.2	พจนานุกรมข้อมูล (Data Dictionary)	28
6.2.1	ตาราง: Customers	28
6.2.2	ตาราง: Trainers	28
6.2.3	ตาราง: GymEquipments	29
6.2.4	ตาราง: TrainingSessions	29
7	การใช้งานเว็บ	30
7.1	ส่วนผู้ใช้งานทั่วไป (Public Section)	30
7.1.1	การใช้งานหน้าเข้าสู่ระบบ (Login Page)	30
7.1.2	การใช้งานหน้าลงทะเบียน (Register Page)	30
7.2	ส่วนสมาชิก (Member Section)	33
7.2.1	การใช้งานหน้าแดชบอร์ดสมาชิก (User Dashboard)	33
7.2.2	การจองเซสชัน (Booking Session)	33
7.2.3	การใช้งานหน้าข้อมูลส่วนตัว (My Profile)	34
7.3	ส่วนผู้ดูแลระบบ (Admin Section)	36
7.3.1	การเข้าสู่ระบบสำหรับผู้ดูแลระบบ (Admin Login)	36
7.3.2	การใช้งานหน้าแดชบอร์ดผู้ดูแลระบบ (Admin Dashboard)	36
7.3.3	การจัดการข้อมูลลูกค้า (Customer Management)	37
7.3.4	การจัดการข้อมูลเทรนเนอร์ (Trainer Management)	37
7.3.5	การจัดการอุปกรณ์ (Equipment Management)	37
7.3.6	การจัดการตารางฝึกซ้อม (Session Management)	38
7.3.7	การดูบันทึกระบบ (System Logs)	38
7.3.8	การออกรายงาน (Reports)	38
7.4	การแสดงผลในโหมดมืด (Dark Mode Interface)	41
7.4.1	หน้าจอผู้ดูแลระบบในโหมดมืด	41
7.4.2	หน้ารายงานในโหมดมืด	41
7.4.3	หน้าจอสมาชิกในโหมดมืด	41
8	อธิบายโค้ด	46
8.1	Backend (ระบบหลังบ้าน)	46
8.1.1	server.js	46
8.2	ฟีเจอร์เพิ่มเติม (Additional Features)	51
8.2.1	การค้นหาและแบ่งหน้า (Search & Pagination) - ส่วน Backend	51
8.2.2	Validation Middleware	52
8.2.3	การเชื่อมต่อฐานข้อมูล (Database Connection - db.js)	52

8.2.4	API Endpoints (server.js)	53
8.3	Frontend (ระบบหน้าบ้าน)	55
8.3.1	frontend/server.js	55
8.3.2	frontend/index.js	57
8.3.3	frontend/js/config.js	59
8.3.4	frontend/js/auth.js	59
8.3.5	frontend/js/user/dashboard.js	62
8.3.6	ไฟล์ frontend/views/index.ejs	67
8.3.7	ไฟล์ frontend/views/login.ejs	70
8.3.8	ไฟล์ frontend/views/register.ejs	72
8.3.9	ไฟล์ frontend/views/customers.ejs	73
8.3.10	ไฟล์ frontend/views/equipments.ejs	76
8.3.11	ไฟล์ frontend/views/trainers.ejs	77
8.3.12	ไฟล์ frontend/views/sessions.ejs	79
8.3.13	ไฟล์ frontend/views/logs.ejs	82
8.3.14	ไฟล์ frontend/views/user/dashboard.ejs	85
8.3.15	ไฟล์ frontend/views/user/profile.ejs	88
8.4	ฟีเจอร์เพิ่มเติม (Additional Features)	91
8.4.1	Dark Mode (โหมดมืด)	91
8.4.2	รายงานพร้อมพิมพ์ (Printable Report)	92
8.4.3	การค้นหาและแบ่งหน้า (Search & Pagination) - ส่วน Frontend	92
8.4.4	frontend/js/equipments.js	93
8.4.5	frontend/js/trainers.js	95
8.4.6	frontend/js/sessions.js	96
8.4.7	frontend/js/index.js (Admin Dashboard)	99
8.4.8	frontend/js/logs.js	99
8.4.9	frontend/js/theme.js	102
8.4.10	frontend/js/user/profile.js	103
9	การทดสอบและประเมินผลระบบ (System Testing and Evaluation)	105
9.1	ระเบียบวิธีและสภาพแวดล้อมการทดสอบ (Testing Methodology and Environment)	105
9.1.1	วิธีการทดสอบ (Testing Methodology)	105
9.1.2	สภาพแวดล้อมการทดสอบ (Testing Environment)	105
9.2	การทดสอบฟังก์ชันการทำงาน (Functional Testing)	105
9.2.1	โมดูลการยืนยันตัวตน (Authentication Module)	105
9.2.2	โมดูลการจองและการจัดการตารางเวลา (Booking Module)	106
9.2.3	โมดูลการจัดการข้อมูลโดยผู้ดูแลระบบ (Data Management)	106
9.3	การทดสอบประสิทธิภาพและการรองรับภาระงาน (Performance and Load Testing)	107
9.3.1	แนวคิดการทดสอบประสิทธิภาพบน Node.js	107
9.3.2	สถานการณ์จำลองการทดสอบ (Simulated Load Test Scenario)	107
9.4	การทดสอบการยอมรับของผู้ใช้ (User Acceptance Testing - UAT)	107
9.4.1	เกณฑ์การยอมรับ (Acceptance Criteria)	107
9.4.2	ผลการประเมินและความคิดเห็น (Evaluation Summary)	107
10	สรุป	108
10.1	สรุปผลการดำเนินงาน	108
10.1.1	สรุปภาพรวมของระบบ (System Overview Summary)	108
10.1.2	ผลการพัฒนาระบบ (Development Achievements)	108
10.1.3	ปัญหาและข้อจำกัด (Limitations and Challenges)	109
10.1.4	แนวทางในการพัฒนาต่อยอด (Future Work)	109
A	Comprehensive System Installation and Deployment Guide	111
A.1	Prerequisites	111
A.1.1	Required Software	111
A.2	Environment Configuration	111
A.2.1	Database Setup	112
A.2.2	Setting up the .env File	112
A.3	Installation Process	112

A.3.1	Cloning the Repository	112
A.3.2	Installing Dependencies	113
A.4	Database Initialization	113
A.4.1	Running Migrations	113
A.4.2	Seeding Data	113
A.5	Execution and Deployment	113
A.5.1	Local Development	113
A.5.2	Production Deployment	113
B	Core System Source Code Reference	114
B.1	Main Application Entry Point (server.js)	114
B.2	Sequelize Model Definition (models/TrainingSession.js)	115
B.3	Core Logic Controller (Booking Conflict Detection)	116
B.4	Frontend View Template (views/index.ejs)	118

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของโครงการ (Background and Significance)

ในปัจจุบัน กระแสการดูแลสุขภาพและการออกกำลังกายได้รับความนิยมเพิ่มขึ้นอย่างต่อเนื่อง ส่งผลให้ธุรกิจฟิตเนสหรือยิม (Gym) มีการเติบโตและขยายตัวอย่างรวดเร็ว อย่างไรก็ตาม การบริหารจัดการยิมแบบดั้งเดิมที่อาศัยการจดบันทึกด้วยกระดาษหรือการใช้โปรแกรมตารางคำนวณ (Spreadsheet) ทั่วไป มักประสบปัญหาความยุ่งยาก ซับซ้อน และขาดประสิทธิภาพ โดยเฉพาะอย่างยิ่งในการบริหารจัดการตารางการใช้งานอุปกรณ์ การนัดหมายเทรนเนอร์ และการจัดเก็บข้อมูลสมาชิก ซึ่งอาจนำไปสู่ข้อผิดพลาดต่างๆ เช่น การจองเวลาซ้อนทับกัน (Double Booking), ข้อมูลสมาชิกสูญหาย หรือความล่าช้าในการตรวจสอบสถานะของอุปกรณ์และเทรนเนอร์

เพื่อแก้ไขปัญหาดังกล่าวและยกระดับการให้บริการ คณะผู้จัดทำจึงได้แนวคิดในการพัฒนา “ระบบบริหารจัดการยิม (GymBro Management System)” ซึ่งเป็นเว็บแอปพลิเคชันที่ถูกออกแบบมาเพื่อช่วยอำนวยความสะดวกในการดำเนินงานของยิมอย่างครบวงจร ระบบนี้จะทำหน้าที่เป็นศูนย์กลางในการเชื่อมโยงข้อมูลระหว่างสมาชิก (Members), ผู้ดูแลระบบ (Admins), เทรนเนอร์ (Trainers), และทรัพยากรภายในยิม (Equipments) เข้าด้วยกัน ช่วยให้การบริหารจัดการเป็นไปอย่างเป็นระบบ รวดเร็ว และตรวจสอบได้ ซึ่งไม่เพียงแต่จะช่วยลดภาระงานของผู้ดูแลระบบ แต่ยังช่วยสร้างประสบการณ์ที่ดีให้กับสมาชิกผู้ใช้บริการอีกด้วย

1.2 วัตถุประสงค์ของโครงการ (Objectives)

1. เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับการบริหารจัดการข้อมูลและการจองคลาสเรียนในฟิตเนส
2. เพื่ออำนวยความสะดวกให้กับสมาชิกในการตรวจสอบตารางเวลาและจองคลาสเรียนด้วยตนเอง
3. เพื่อช่วยให้ผู้ดูแลระบบสามารถบริหารจัดการข้อมูลสมาชิก เทรนเนอร์ และอุปกรณ์ได้อย่างมีประสิทธิภาพ
4. เพื่อป้องกันปัญหาการจองเวลาซ้อนทับกัน (Double Booking) ของเทรนเนอร์และอุปกรณ์
5. เพื่อศึกษาและประยุกต์ใช้ความรู้ด้าน Full Stack Development ในการพัฒนาระบบใช้งานจริง

1.3 ขอบเขตของโครงการ (Scope)

โครงการนี้ครอบคลุมการพัฒนาเว็บแอปพลิเคชัน โดยแบ่งขอบเขตการทำงานออกเป็น 2 ส่วนหลัก ดังนี้:

1.3.1 ส่วนของผู้ดูแลระบบ (Administrator)

- สามารถดูภาพรวมสถิติของระบบ (Dashboard) เช่น จำนวนสมาชิก, เทรนเนอร์, อุปกรณ์ และรายการจองในปัจจุบัน
- สามารถจัดการข้อมูลสมาชิก (เพิ่ม, ลบ, แก้ไข, ตรวจสอบข้อมูล)
- สามารถจัดการข้อมูลเทรนเนอร์ (เพิ่ม, ลบ, แก้ไข, ระดับความสามารถ, ความเชี่ยวชาญ)
- สามารถจัดการข้อมูลอุปกรณ์ออกกำลังกาย (เพิ่ม, ลบ, แก้ไขสถานะการใช้งาน/ซ่อมบำรุง)

- สามารถบริหารจัดการตารางการฝึกซ้อม (Session Management) และตรวจสอบประวัติการจอง
- สามารถดูบันทึกการใช้งานระบบ (System Logs) เพื่อตรวจสอบการกระทำย้อนหลังได้

1.3.2 ส่วนของสมาชิก (Member)

- สามารถลงทะเบียนสมัครสมาชิกและเข้าสู่ระบบได้
- สามารถดูตารางเวลาการฝึกซ้อม (Schedule) ของยิมได้
- สามารถจองเวลาการฝึกซ้อม (Booking) โดยเลือกเทรนเนอร์และอุปกรณ์ที่ต้องการได้
- ระบบจะตรวจสอบความว่างของเทรนเนอร์และอุปกรณ์ให้โดยอัตโนมัติก่อนการจอง
- สามารถยกเลิกการจองของตนเองได้
- สามารถดูและตรวจสอบข้อมูลโปรไฟล์ของตนเองได้

1.4 เป้าหมายของโครงการ (Goals)

1. ได้ระบบบริหารจัดการยิมที่สามารถใช้งานได้จริง มีความเสถียร และถูกต้องตามความต้องการ
2. ระบบมีความปลอดภัยในระดับพื้นฐาน มีการตรวจสอบสิทธิ์การเข้าใช้งาน (Authentication & Authorization)
3. ส่วนติดต่อผู้ใช้งาน (User Interface) มีความทันสมัย ใช้งานง่าย (User-friendly) และรองรับ การแสดงผลบนอุปกรณ์ต่างๆ (Responsive)
4. โค้ดของโปรแกรมมีความเป็นระเบียบง่ายต่อการดูแลรักษาและพัฒนาต่อยอด

1.5 ประโยชน์ที่คาดว่าจะได้รับ (Expected Benefits)

1. ช่วยลดความผิดพลาดในการบริหารจัดการตารางเวลาและการจองคลาสเรียน
2. เพิ่มความสะดวกสบายให้กับสมาชิกในการเข้าถึงบริการของยิม
3. ผู้ดูแลระบบสามารถทำงานได้รวดเร็วและมีประสิทธิภาพมากขึ้น ลดการใช้กระดาษและขั้นตอนที่ซ้ำซ้อน
4. มีฐานข้อมูลที่เป็นระบบ สามารถนำข้อมูลไปวิเคราะห์เพื่อวางแผนการบริหารงานในอนาคตได้
5. ผู้จัดทำได้รับความรู้และประสบการณ์ในการพัฒนา Full Stack Web Application ตั้งแต่การออกแบบจนถึงการ Deploy

บทที่ 2

ทฤษฎีและเทคโนโลยีที่เกี่ยวข้อง (Literature Review and Related Technologies)

บทนี้จะกล่าวถึงทฤษฎีพื้นฐาน รูปแบบสถาปัตยกรรม และเทคโนโลยีที่ใช้ในการพัฒนาระบบบริหารจัดการยิม (GymBro Management System) โดยระบบถูกพัฒนาขึ้นบนพื้นฐานของการพัฒนาเว็บแอปพลิเคชันแบบ Full-Stack สมัยใหม่ โดยใช้สถาปัตยกรรมแบบ Client-Server, การสื่อสารผ่าน RESTful API และระบบ Backend ที่ขับเคลื่อนด้วย Node.js, Express.js และ PostgreSQL

2.1 สถาปัตยกรรมแบบไคลเอนต์-เซิร์ฟเวอร์ (Client-Server Architecture)

สถาปัตยกรรมแบบไคลเอนต์-เซิร์ฟเวอร์ เป็นโครงสร้างแอปพลิเคชันแบบกระจายศูนย์ที่แบ่งภาระงานระหว่างผู้ให้บริการทรัพยากรหรือบริการ เรียกว่า "เซิร์ฟเวอร์" (Server) และผู้ร้องขอใช้บริการ เรียกว่า "ไคลเอนต์" (Client) โมเดลนี้เป็นรากฐานสำคัญของ World Wide Web และเว็บแอปพลิเคชันในปัจจุบัน

2.1.1 หลักการทำงาน (Working Principle)

ใน สถาปัตยกรรม นี้ ไคล เอน ต์ (โดยทั่วไป คือ เว็บ เบรราวเซอร์ หรือ แอปพลิเคชัน บน มือ ถือ) และ เซิร์ฟเวอร์ (เครื่องคอมพิวเตอร์ที่มีประสิทธิภาพสูง) จะสื่อสารกันผ่านเครือข่ายคอมพิวเตอร์ ไคลเอนต์จะเริ่มต้นการสื่อสารโดยส่งคำขอ (Request) ไปยังเซิร์ฟเวอร์ และเซิร์ฟเวอร์จะประมวลผลคำขอนั้นแล้วส่งคำตอบ (Response) กลับมา การแยกส่วนการทำงานนี้ช่วยให้การพัฒนาส่วนของไคลเอนต์และเซิร์ฟเวอร์เป็นอิสระต่อกัน เพิ่มความยืดหยุ่นในการขยายระบบ และง่ายต่อการจัดการ

2.1.2 การสื่อสารผ่านโปรโตคอล HTTP (Communication via HTTP Protocol)

โปรโตคอลหลักที่ใช้ในการสื่อสารในสถาปัตยกรรมเว็บคือ Hypertext Transfer Protocol (HTTP) ซึ่งเป็นโปรโตคอลในระดับ Application Layer ที่กำหนดรูปแบบการรับส่งข้อความ

- **ไร้สถานะ (Statelessness):** แต่ละคำขอจากไคลเอนต์ไปยังเซิร์ฟเวอร์จะถูกปฏิบัติเสมือนเป็นธุรกรรมอิสระที่ไม่เกี่ยวข้องกับคำขอก่อนหน้า สิ่งนี้ช่วยลดความซับซ้อนในการออกแบบเซิร์ฟเวอร์ แต่จำเป็นต้องมีกลไกเพิ่มเติม (เช่น คุกกี้ หรือ โทเค็น) เพื่อรักษาสถานะเซสชัน
- **วงจรคำขอ-คำตอบ (Request-Response Cycle):** การสื่อสารเป็นไปตามรูปแบบที่เคร่งครัด ไคลเอนต์ส่งข้อความ HTTP Request ซึ่งประกอบด้วยบรรทัดคำขอ (Method, URI, Protocol Version), ส่วนหัว (Headers), และส่วนเนื้อหา (Body) เซิร์ฟเวอร์จะตอบกลับด้วยข้อความ HTTP Response ซึ่งประกอบด้วยสถานะ (Status Code), ส่วนหัว, และเนื้อหาที่ร้องขอ

2.1.3 วงจรชีวิตของเว็บรีเควส (Lifecycle of a Web Request)

วงจรการทำงานที่สมบูรณ์ของเว็บรีเควสประกอบด้วยขั้นตอนดังนี้:

1. **DNS Resolution:** ไคลเอนต์แปลงชื่อโดเมนเป็น IP Address
2. **TCP Handshake:** สร้างการเชื่อมต่อ TCP ระหว่างไคลเอนต์และเซิร์ฟเวอร์
3. **TLS Negotiation:** หากใช้ HTTPS จะมีการเจรจาช่องทางที่ปลอดภัย

4. **Request Transmission:** ไคลเอนต์ส่งข้อมูล HTTP Request
5. **Server Processing:** เว็บเซิร์ฟเวอร์ (เช่น Nginx, Apache) รับคำขอ และส่งต่อไปยัง Application Server (เช่น Node.js) เพื่อประมวลผล Business Logic, ดึงข้อมูล และสร้างคำตอบ
6. **Response Transmission:** เซิร์ฟเวอร์ส่ง HTTP Response กลับไปยังไคลเอนต์
7. **Rendering:** ไคลเอนต์ (เบราว์เซอร์) แปลงผลลัพธ์ (HTML, CSS, JSON) และแสดงผลให้ผู้ใช้งานเห็น

2.2 การออกแบบและพัฒนา RESTful API (RESTful API Design & Development)

Representational State Transfer (REST) เป็นรูปแบบสถาปัตยกรรมซอฟต์แวร์ที่กำหนดข้อจำกัดสำหรับการสร้าง Web Services โดย Web Services ที่ปฏิบัติตามรูปแบบ REST จะเรียกว่า RESTful Web Services ซึ่งช่วยให้ระบบคอมพิวเตอร์ต่างๆ บนอินเทอร์เน็ตสามารถทำงานร่วมกันได้

2.2.1 หลักการของ REST (Principles of REST)

REST อาศัยโปรโตคอลการสื่อสารแบบ Stateless, Client-Server และ Cacheable ซึ่งโดยส่วนใหญ่จะใช้โปรโตคอล HTTP หลักการสำคัญได้แก่:

- **Uniform Interface:** การมีอินเทอร์เฟซที่เป็นมาตรฐานเดียวกัน ช่วยลดความซับซ้อนและแยกส่วนสถาปัตยกรรมออกจากกัน ประกอบด้วยการระบุทรัพยากร (URIs), การจัดการทรัพยากรผ่าน Representation, และข้อความที่อธิบายตนเองได้
- **Stateless:** เซิร์ฟเวอร์จะไม่เก็บสถานะของไคลเอนต์ แต่ละคำขอต้องมีข้อมูลบริบทเพียงพอสำหรับการประมวลผล
- **Client-Server:** การแยกส่วนความรับผิดชอบที่ชัดเจน
- **Cacheable:** การตอบกลับต้องระบุได้ว่าสามารถแคชได้หรือไม่ เพื่อลดการดึงข้อมูลซ้ำซ้อน
- **Layered System:** ไคลเอนต์ไม่จำเป็นต้องรู้ว่ากำลังเชื่อมต่อกับเซิร์ฟเวอร์ปลายทางโดยตรงหรือผ่านตัวกลาง

2.2.2 HTTP Methods มาตรฐาน (Standard HTTP Methods)

RESTful API ใช้วิธีการ HTTP มาตรฐานในการดำเนินการ CRUD (Create, Read, Update, Delete) กับทรัพยากร:

- **GET:** ใช้ดึงข้อมูลทรัพยากร (Safe และ Idempotent)
- **POST:** ใช้ส่งข้อมูลเพื่อสร้างทรัพยากรใหม่ (มักทำให้เกิดการเปลี่ยนแปลงสถานะบนเซิร์ฟเวอร์)
- **PUT:** แทนที่ข้อมูลทรัพยากรทั้งหมดด้วยข้อมูลใหม่ (Idempotent)
- **DELETE:** ลบทรัพยากรที่ระบุ
- **PATCH:** แก้ไขข้อมูลบางส่วน of ทรัพยากร

2.2.3 รหัสสถานะ HTTP (HTTP Status Codes)

การจัดการรหัสสถานะที่ถูกต้องมีความสำคัญต่อการสื่อสารของ API:

- **2xx Success:** การร้องขอสำเร็จ (เช่น 200 OK, 201 Created)
- **3xx Redirection:** ต้องการการดำเนินการเพิ่มเติมเพื่อทำคำขอให้สมบูรณ์
- **4xx Client Error:** ข้อผิดพลาดเกิดจากไคลเอนต์ (เช่น 400 Bad Request, 401 Unauthorized, 404 Not Found)
- **5xx Server Error:** เซิร์ฟเวอร์ไม่สามารถทำตามคำขอที่ถูกต้องได้ (เช่น 500 Internal Server Error)

2.3 Node.js

Node.js เป็น JavaScript Runtime Environment แบบ Open-source และ Cross-platform ที่ทำงานบนฝั่ง Backend โดยใช้ V8 Engine ในการประมวลผล JavaScript ภายนอกเว็บเบราว์เซอร์

2.3.1 ประวัติและวิวัฒนาการ (History and Evolution)

สร้างขึ้นโดย Ryan Dahl ในปี 2009 เพื่อสร้างแอปพลิเคชันเครือข่ายที่รองรับการขยายตัวได้ดี (Scalable) ก่อนหน้านี้ JavaScript ถูกใช้เป็นหลักในการเขียนสคริปต์ฝั่งไคลเอนต์ Node.js ได้รวบรวมการพัฒนาเว็บแอปพลิเคชันให้อยู่ภายใต้ภาษาโปรแกรมเดียว ทำให้สามารถใช้ JavaScript ได้ทั้งฝั่งเซิร์ฟเวอร์และไคลเอนต์

2.3.2 สถาปัตยกรรมแบบขับเคลื่อนด้วยเหตุการณ์ (Event-Driven Architecture)

Node.js ใช้สถาปัตยกรรมแบบ Event-driven ที่มีความสามารถในการทำ Asynchronous I/O การออกแบบนี้มุ่งเน้นเพิ่มประสิทธิภาพ Throughput และ Scalability ในเว็บแอปพลิเคชันที่มีการ Input/Output จำนวนมาก รวมถึง Real-time Web Applications กลไกสำคัญคือ **Event Loop** ซึ่งช่วยให้ Node.js ทำงานแบบ Non-blocking I/O ได้แม้จะเป็น Single-threaded โดยจะผลักภาระงานไปยัง System Kernel เมื่อเป็นไปได้

2.3.3 Single-Threaded, Non-blocking I/O

ต่างจากเทคนิคการให้บริการเว็บแบบดั้งเดิมที่แต่ละการเชื่อมต่อ (Request) จะสร้าง Thread ใหม่ ซึ่งสิ้นเปลือง RAM และมีขีดจำกัด Node.js ทำงานบน Thread เดียว

- **Non-blocking:** เมื่อ Node.js ต้องทำ I/O Operation (เช่น อ่านไฟล์, เชื่อมต่อฐานข้อมูล) แทนที่จะบล็อก Thread เพื่อรอ Node.js จะดำเนินการต่อทันทีและกลับมาจัดการเมื่อได้รับผลลัพธ์กลับมา (Callback)
- **Performance:** โมเดลนี้ทำให้ Node.js มีความเบาและมีประสิทธิภาพ เหมาะสำหรับแอปพลิเคชันที่เน้นข้อมูล (Data-intensive) และทำงานแบบ Real-time

2.4 Express.js

Express.js หรือเรียกสั้นๆ ว่า Express เป็น Web Application Framework สำหรับ Node.js ที่ออกแบบมาเพื่อสร้างเว็บแอปพลิเคชันและ API ถือเป็นมาตรฐานหลักของ Server Framework สำหรับ Node.js

2.4.1 บทบาทในฐานะ Web Framework

Express ให้บริการฟิเจอร์พื้นฐานสำหรับเว็บแอปพลิเคชันโดยไม่ปิดกั้นความสามารถของ Node.js ช่วยลดความซับซ้อนในการสร้างเซิร์ฟเวอร์ โดยมีเครื่องมือจัดการ HTTP Servers ที่แข็งแกร่ง ทำให้ง่ายต่อการจัดการฟังก์ชันการทำงานในรูปแบบของ Handler สำหรับ Route และ HTTP Verb ต่างๆ

2.4.2 การจัดการเส้นทาง (Routing Management)

Routing หมายถึงวิธีการที่แอปพลิเคชันตอบสนองต่อคำขอของไคลเอนต์ในแต่ละ Endpoints (URIs) Express มีกลไก Routing ที่ซับซ้อน ช่วยให้กำหนด Route โดยใช้เมธอดของ 'app' object ที่ตรงกับ HTTP methods

- Route สามารถเป็น string path, string pattern หรือ regular expression
- รองรับ Route parameters (เช่น '/users/:userId') เพื่อดึงค่าจาก URL
- สามารถจัดกลุ่ม Route โดยใช้ 'express.Router' เพื่อแยกโมดูล Handler

2.4.3 Middleware Pipeline

ฟังก์ชัน Middleware คือฟังก์ชันที่มีสิทธิ์เข้าถึง request object (req), response object (res), และฟังก์ชัน middleware ถัดไปในวงจร request-response

- **การทำงาน:** Middleware สามารถรันโค้ดใดๆ, แก้ไข req/res object, จบวงจร request-response, หรือเรียก middleware ถัดไป
- **Pipeline:** แอปพลิเคชัน Express เปรียบเสมือนชุดของการเรียกฟังก์ชัน Middleware ต่อเนื่องกัน ช่วยให้แยกส่วนความกังวล (Separation of Concerns) ได้ดี เช่น การแปลง Request Body, การจัดการ Cookie, การ Logging, การยืนยันตัวตน และการจัดการ Error ก่อนที่คำขอจะไปถึง Route Handler หลัก

2.5 ระบบจัดการฐานข้อมูล PostgreSQL (PostgreSQL Database System)

PostgreSQL หรือเรียกสั้นๆ ว่า Postgres เป็นระบบฐานข้อมูลเชิงสัมพันธ์ (RDBMS) แบบ Open-source ระดับองค์กรที่มีความก้าวหน้าสูง รองรับทั้งการสอบถามข้อมูลแบบ SQL (Relational) และ JSON (Non-relational)

2.5.1 คุณลักษณะของ RDBMS (RDBMS Characteristics)

ในฐานะ RDBMS PostgreSQL เก็บข้อมูลในรูปแบบตาราง (Tables) ที่มีแถวและคอลัมน์ มีการบังคับใช้ Schema ที่มีโครงสร้างชัดเจน เพื่อรับประกันความถูกต้องของข้อมูล (Data Integrity) ผ่าน:

- **ACID Compliance:** Atomicity, Consistency, Isolation, Durability รับประกันความน่าเชื่อถือของธุรกรรมฐานข้อมูล
- **Foreign Keys:** บังคับความสัมพันธ์ระหว่างตาราง
- **Constraints:** กฎเกณฑ์ต่างๆ เช่น PRIMARY KEY, UNIQUE, NOT NULL, และ CHECK เพื่อรักษาคุณภาพข้อมูล

2.5.2 โครงสร้างข้อมูลและประสิทธิภาพ (Data Structuring and Performance)

PostgreSQL มีชื่อเสียงด้านความเสถียรและความสามารถ

- **Extensibility:** ผู้ใช้สามารถนิยาม Data Types, Index Types และภาษาฟังก์ชันของตนเองได้
- **Concurrency:** ใช้ Multi-Version Concurrency Control (MVCC) ช่วยให้การอ่านและเขียนข้อมูลเกิดขึ้นพร้อมกันได้โดยไม่ต้องล็อกทั้งตาราง เพิ่มประสิทธิภาพในสภาพแวดล้อมที่มีการใช้งานพร้อมกันสูง
- **Advanced Indexing:** รองรับดัชนีแบบ B-tree, Hash, GiST, SP-GiST, GIN, และ BRIN เพื่อเพิ่มประสิทธิภาพการค้นหาข้อมูลหลากหลายรูปแบบ

2.6 การเชื่อมโยงข้อมูลเชิงวัตถุสัมพันธ์ (ORM) และ Sequelize

Object-Relational Mapping (ORM) เป็นเทคนิคการเขียนโปรแกรมเพื่อแปลงข้อมูลระหว่างระบบชนิดข้อมูลที่ไม่เข้ากัน โดยใช้ภาษาโปรแกรมเชิงวัตถุ

2.6.1 Impedance Mismatch

”Object-Relational Impedance Mismatch” หมายถึงความยากลำบากทางแนวคิดและเทคนิคเมื่อต้องใช้งาน RDBMS ร่วมกับภาษาโปรแกรมเชิงวัตถุ (OOP)

- วัตถุ (Objects) มีโครงสร้างแบบกราฟและมีการสืบทอดคุณสมบัติ (Inheritance)
- ฐานข้อมูลเชิงสัมพันธ์ใช้โครงสร้างแบบตารางและทฤษฎีเซต
- ORM ช่วยเชื่อมต่อช่องว่างนี้โดยการแมปตารางฐานข้อมูลเป็นคลาส และแถวข้อมูลเป็นวัตถุ ช่วยให้นักพัฒนาจัดการฐานข้อมูลโดยใช้ syntax ของภาษาโปรแกรมแทนการเขียน SQL ดิบ

2.6.2 การทำงานของ Sequelize (Sequelize Operations)

Sequelize เป็น Node.js ORM แบบ Promise-based สำหรับ Postgres, MySQL, MariaDB, SQLite และ Microsoft SQL Server

- **Model Definition:** นักพัฒนานิยาม Model เป็น JavaScript Class/Object ระบุชนิดข้อมูลและการตรวจสอบความถูกต้อง Sequelize จะสร้างคำสั่ง SQL เพื่อสร้างตารางให้อัตโนมัติ
- **Associations:** รองรับความสัมพันธ์มาตรฐาน เช่น One-To-One, One-To-Many, และ Many-To-Many โดยจัดการความซับซ้อนของ Foreign Keys และ Join Tables ให้
- **Synchronization:** มีเมธอดเช่น 'sync()' เพื่อสร้างหรืออัปเดตตารางในฐานข้อมูลให้ตรงกับ Model
- **Querying:** มี API ที่หลากหลายสำหรับการค้นหาข้อมูล (เช่น 'Model.findAll()', 'Model.create()') โดยซ่อนความซับซ้อนของการเขียน SQL JOINS

2.7 EJS (Embedded JavaScript templating)

EJS เป็นภาษาเทมเพลตที่เรียบง่ายที่ช่วยให้สามารถสร้าง HTML Markup ด้วย JavaScript แบบธรรมดา

2.7.1 การเรนเดอร์ฝั่งเซิร์ฟเวอร์ (Server-Side Rendering - SSR)

Server-Side Rendering คือกระบวนการเรนเดอร์หน้าเว็บที่ฝั่งเซิร์ฟเวอร์และส่ง HTML ที่สมบูรณ์แล้วไปยังเบราว์เซอร์ของไคลเอนต์

- ในบริบทของ EJS และ Express เมื่อมีคำขอเข้ามา เซิร์ฟเวอร์จะคอมไพล์เทมเพลต EJS, ประมวลผลตรรกะ JavaScript ที่ฝังอยู่ (Loop, Conditional) โดยใช้ข้อมูลที่ส่งให้ (เช่น จากฐานข้อมูล) และสร้างเป็น HTML string
- HTML string นี้จะถูกส่งกลับเป็น HTTP Response

2.7.2 ข้อดีของ SSR ด้วย EJS (Advantages of SSR with EJS)

- **SEO Friendly:** เนื่องจากเนื้อหาถูกเรนเดอร์สมบูรณ์ก่อนถึงไคลเอนต์ Search Engine Crawler สามารถเก็บข้อมูล (Index) ได้ง่าย
- **Performance (First Contentful Paint):** ผู้ใช้เห็นเนื้อหาได้เร็วกว่า เพราะเบราว์เซอร์ได้รับเอกสาร HTML ที่มีข้อมูลครบถ้วน ไม่ต้องรอโหลดและรัน JavaScript ฝั่งไคลเอนต์เพื่อแสดงผลหน้าจอก็เริ่มต้น
- **Simplicity:** สำหรับแอปพลิเคชันจำนวนมาก SSR ช่วยลดความซับซ้อนของ Tech Stack โดยลดความจำเป็นในการใช้ Library จัดการ State ฝั่งไคลเอนต์ที่ซับซ้อน EJS ยังช่วยให้สามารถนำส่วนประกอบ UI (เช่น Header, Footer) กลับมาใช้ซ้ำได้ง่ายผ่านพีเจอร์ 'include'

บทที่ 3

การออกแบบพฤติกรรมระบบ (Behavioral System Design - UML Diagrams)

บทนี้จะอธิบายรายละเอียดการออกแบบพฤติกรรมของระบบบริหารจัดการยิม (GymBro Management System) โดยใช้แผนภาพ Unified Modeling Language (UML) แบบจำลองพฤติกรรมแสดงให้เห็นว่าระบบมีปฏิสัมพันธ์กับหน่วยงานภายนอก (ผู้ใช้) อย่างไร และส่วนประกอบภายในทำงานร่วมกันอย่างไรเพื่อให้เป็นไปตามข้อกำหนดการทำงาน ส่วนนี้ครอบคลุมแผนภาพยูสเคส (Use Case Diagrams), แผนภาพกิจกรรม (Activity Diagrams) และแผนภาพลำดับ (Sequence Diagrams)

3.1 แผนภาพยูสเคสและคำอธิบายอย่างละเอียด (Use Case Diagram & Detailed Descriptions)

แผนภาพยูสเคสให้มุมมองระดับสูงของการทำงานของระบบจากมุมมองของผู้กระทำภายนอก (External Actors) โดยกำหนดขอบเขตของระบบและการปฏิสัมพันธ์ระหว่างผู้กระทำและยูสเคส

3.1.1 ผู้กระทำ (Actors)

ระบบเกี่ยวข้องกับผู้กระทำหลักสองกลุ่ม:

- **ผู้ดูแลระบบ (Administrator):** ผู้ใช้ที่มีสิทธิ์พิเศษ รับผิดชอบในการจัดการข้อมูลระบบ รวมถึงระเบียบสมาชิก โปรไฟล์เทรนเนอร์ สถานะอุปกรณ์ และตรวจสอบบันทึกระบบ
- **สมาชิก (Member):** ผู้ใช้ที่ลงทะเบียนแล้ว ซึ่งโต้ตอบกับระบบเพื่อดูตารางเวลา จองเซชันการฝึกซ้อม และจัดการโปรไฟล์ส่วนตัว

3.1.2 รายการยูสเคส (Use Case List)

ยูสเคสของผู้ดูแลระบบ:

- เข้าสู่ระบบ (Admin Authentication)
- จัดการสมาชิก (ดู, แก้ไข, ลบ)
- จัดการเทรนเนอร์ (เพิ่ม, แก้ไข, ลบ)
- จัดการอุปกรณ์ (เพิ่ม, แก้ไขสถานะ, ลบ)
- ดูแดชบอร์ดและสถิติระบบ
- ดูบันทึกระบบ (System Logs)
- จัดการเซชันการจองทั้งหมด (ดู, ยกเลิก)

ยูสเคสของสมาชิก:

- ลงทะเบียน (สร้างบัญชี)
- เข้าสู่ระบบ (Member Authentication)
- ดูแดชบอร์ดส่วนตัวและตารางเวลา

- จองเซสชันการฝึกซ้อม
- ยกเลิกการจองของตนเอง
- ดูโปรไฟล์ผู้ใช้

3.1.3 คำอธิบายยูสเคสอย่างละเอียด (Detailed Use Case Descriptions)

ตารางต่อไปนี้อธิบายกระบวนการสำคัญ 4 ประการอย่างละเอียด

รหัสยูสเคส	UC-01
ชื่อยูสเคส	การยืนยันตัวตนผู้ใช้ (Register/Login)
ผู้กระทำหลัก	สมาชิก, ผู้ดูแลระบบ
เงื่อนไขก่อนเริ่ม	ผู้ใช้เข้าถึงเว็บแอปพลิเคชัน GymBro
สถานการณ์ หลัก (Success Scenario)	1. ผู้ใช้ไปยังหน้าเข้าสู่ระบบ (Login) 2. ผู้ใช้กรอกอีเมลและเบอร์โทรศัพท์ที่ลงทะเบียนไว้ 3. ผู้ใช้กดปุ่ม "Sign In" 4. ระบบตรวจสอบข้อมูลกับฐานข้อมูล 5. ระบบดึงบทบาทของผู้ใช้ (Admin หรือ Member) 6. ระบบสร้าง Session Token (เก็บใน localStorage) 7. ระบบนำทางผู้ใช้ไปยัง Dashboard ที่เหมาะสม (Admin Dashboard หรือ Member Schedule)
ทางเลือกอื่น (Alternative Flows)	A1: ข้อมูลไม่ถูกต้อง 1. ระบบไม่พบข้อมูลที่ตรงกัน 2. ระบบแสดงข้อความแจ้งเตือน "Invalid credentials" 3. ผู้ใช้ยังคงอยู่ที่หน้า Login เพื่อลองใหม่ A2: การลงทะเบียนผู้ใช้ใหม่ 1. ผู้ใช้คลิก "Register here" 2. ผู้ใช้กรอกชื่อ-นามสกุล, อีเมล, และเบอร์โทรศัพท์ 3. ระบบตรวจสอบอีเมลซ้ำ 4. ระบบสร้างระเบียบสมาชิกใหม่ด้วยสถานะ "Pending" (หรือ Active ตามค่าเริ่มต้น) 5. ระบบนำทางผู้ใช้ไปหน้า Login
เงื่อนไขหลังจบ	ผู้ใช้ได้รับการยืนยันตัวตนและเข้าถึงฟีเจอร์ตามสิทธิ์

รหัสยูสเคส	UC-04
ชื่อยูสเคส	การจองอุปกรณ์/คลาสเรียน (Equipment/Class Booking)
ผู้กระทำหลัก	สมาชิก
เงื่อนไขก่อนเริ่ม	สมาชิกเข้าสู่ระบบแล้วและอยู่ที่หน้า User Dashboard

สถานการณ์ หลัก (Success Scenario)	1. สมาชิกคลิก "Book Session" 2. ระบบแสดงหน้าต่างการจอง (Booking Modal) 3. สมาชิกเลือกวันที่, เวลาเริ่ม, และระยะเวลา 4. ระบบเรียก อัลกอริทึมตรวจสอบเวลาซ้อนทับ (UC-05) 5. ระบบอัปเดตรายการเทรนเนอร์และอุปกรณ์: • รายการที่ว่างจะเลือกได้ • รายการที่ไม่ว่างหรือซ่อมบำรุงจะถูกปิดการใช้งาน (Disabled) 6. สมาชิกเลือกเทรนเนอร์และอุปกรณ์ที่ว่าง 7. สมาชิกคลิก "Confirm Booking" 8. ระบบบันทึกการจองลงฐานข้อมูล 9. ระบบแสดงแจ้งเตือนสำเร็จและรีเฟรชตารางเวลา
ทางเลือกอื่น (Alternative Flows)	A1: ไม่มีความพร้อมให้บริการ 1. สมาชิกเลือกเวลาที่เทรนเนอร์ทุกคนไม่ว่าง 2. ระบบปิดการใช้งานตัวเลือกเทรนเนอร์ทั้งหมด 3. สมาชิกต้องเลือกช่วงเวลาอื่น
เงื่อนไขหลังจบ	ระเบียบ 'TrainingSession' ใหม่ถูกสร้างขึ้น เชื่อมโยงสมาชิก เทรนเนอร์ และอุปกรณ์

รหัสยูสเคส	UC-05
ชื่อยูสเคส	อัลกอริทึมตรวจสอบเวลาซ้อนทับ (Time Conflict Detection)
ผู้กระทำหลัก	ระบบ (กระบวนการภายใน)
เงื่อนไขก่อนเริ่ม	ผู้ใช้เลือกช่วงเวลาที่ต้องการจอง
สถานการณ์ หลัก (Success Scenario)	1. ระบบคำนวณ 'startTime' และ 'endTime' ที่ร้องขอ 2. ระบบค้นหาฐานข้อมูลสำหรับเซสชันที่มีอยู่ทั้งหมดที่ทับซ้อนกับช่วงเวลานี้ 3. ตรรกะ: $(ExistingStart < NewEnd) \text{ AND } (ExistingEnd > NewStart)$ 4. ระบบระบุว่าเทรนเนอร์และอุปกรณ์ใดเกี่ยวข้องกับเซสชันที่ทับซ้อน 5. ระบบส่งคืนรายการ ID ที่ "ไม่ว่าง" ไปยัง Frontend 6. Frontend ปิดการใช้งาน ID เหล่านี้ใน UI การเลือก
ทางเลือกอื่น (Alternative Flows)	N/A (เป็นกระบวนการตรวจสอบเบื้องหลัง)
เงื่อนไขหลังจบ	ผู้ใช้ถูกป้องกันจากการจองทรัพยากรซ้อนทับ

รหัสยูสเคส	UC-08
ชื่อยูสเคส	การจัดการข้อมูลและการบันทึกการทำงานของผูดูแล (System Logging)
ผู้กระทำหลัก	ผูดูแลระบบ

เงื่อนไขก่อนเริ่ม	Admin เข้าสู่ระบบแล้ว
สถานการณ์ หลัก (Success Scenario)	1. Admin ทำการดำเนินการสำคัญ (เช่น ลบสมาชิก, แก้ไขเทรนเนอร์) 2. ระบบดำเนินการกับฐานข้อมูล 3. เมื่อสำเร็จ ระบบสร้างรายการ Log ประกอบด้วย: Timestamp, Action Type, Admin ID, และ Details 4. ระบบเพิ่มรายการนี้ลงในไฟล์ 'logs.json' หรือตาราง Log ในฐานข้อมูล 5. Admin ไปยังหน้า "System Logs" 6. ระบบอ่านและแสดงประวัติการใช้งาน
ทางเลือกอื่น (Alternative Flows)	A1: ข้อผิดพลาดไฟล์ Log • หากระบบไม่สามารถเขียนไฟล์ Log ได้ จะแสดงข้อผิดพลาดที่ Server Console แต่ยังคงดำเนินการตามคำสั่งผู้ใช้เพื่อให้งานไม่สะดุด
เงื่อนไขหลังจบ	บันทึกการกระทำของผู้ดูแลระบบถูกเก็บรักษาไว้เพื่อการตรวจสอบ

3.2 แผนภาพกิจกรรม (Activity Diagrams)

แผนภาพกิจกรรมอธิบายลักษณะพลวัตของระบบ แสดงการไหลของการควบคุมจากกิจกรรมหนึ่งไปยังอีกกิจกรรมหนึ่ง

3.2.1 กระบวนการจองและการตรวจสอบเวลาซ้อนทับ (Booking & Time Conflict Detection Flow)

กระบวนการนี้แสดงตรรกะที่ใช้เพื่อให้มั่นใจในความถูกต้องของข้อมูล ระหว่างการจอง ระบบต้องตรวจสอบว่าทั้งเทรนเนอร์และอุปกรณ์ที่เลือกไม่ถูกใช้งานในช่วงเวลาที่ร้องขอ

```
1 graph TD
2   A[Start: Member requests booking] --> B[Select Date & Time]
3   B --> C[Calculate Time Interval]
4   C --> D[Fetch Existing Sessions]
5   D --> E[Is Trainer Available?]
6   E -- No --> F[Disable Trainer Option]
7   E -- Yes --> G[Enable Trainer Option]
8   D --> H[Is Equipment Available?]
9   H -- No --> I[Disable Equipment Option]
10  H -- Yes --> J[Enable Equipment Option]
11  F --> K[User Selects Resources]
12  G --> K
13  I --> K
14  J --> K
15  K --> L[Validation Passed?]
16  L -- No --> M[Show Error]
17  L -- Yes --> N[Save to Database]
18  N --> O[End: Booking Confirmed]
```

Listing 3.1: Mermaid Code for Booking Flow

Figure 3.1: โค้ด Mermaid แสดงกระบวนการจองและการตรวจสอบความขัดแย้ง

คำอธิบายอย่างละเอียด:

1. **การเริ่มต้น:** กระบวนการเริ่มเมื่อสมาชิกเปิดหน้าต่างการจอง
2. **การเลือกเวลา:** ผู้ใช้ระบุวันที่ เวลาเริ่ม และระยะเวลา ระบบคำนวณ 'endTime' ทันที
3. **การสอบถามความขัดแย้ง:** Backend (หรือ Logic ฝั่ง Frontend ผ่าน API) ดึงข้อมูลเซสชันทั้งหมดที่ทับซ้อนกับ '[startTime, endTime]'
4. **การกรองทรัพยากร:**
 - ระบบตรวจสอบรายการเทรนเนอร์ หาก ID เทรนเนอร์ปรากฏในเซสชันที่ทับซ้อน จะถูกระบุว่า "ไม่ว่าง"
 - ในทำนองเดียวกัน ID อุปกรณ์ที่พบในเซสชันที่ทับซ้อนจะถูกระบุว่า "ไม่ว่าง"
5. **การอัปเดต UI:** Frontend แสดงผล Dropdown ตัวเลือกที่ไม่ว่างจะถูกทำเป็นสีเทา (Disabled)
6. **การส่งข้อมูล:** ผู้ใช้เลือกจากตัวเลือกที่ถูกต้องและกดส่ง
7. **การตรวจสอบขั้นสุดท้าย:** Server ทำการตรวจสอบครั้งสุดท้ายก่อนบันทึกเพื่อป้องกัน Race Condition หากผ่านจะบันทึกข้อมูล

3.2.2 กระบวนการจัดการข้อมูลของผู้ดูแล (Data Management Flow)

กระบวนการนี้แสดงวิธีการที่ผู้ดูแลระบบจัดการข้อมูลในระบบ (เช่น เพิ่มเทรนเนอร์ใหม่)

คำอธิบายอย่างละเอียด:

1. **การเลือก:** Admin เลือกหมวดหมู่ (Trainers, Customers, หรือ Equipment)
2. **การเริ่มการกระทำ:** Admin เลือกที่จะ เพิ่ม, แก้ไข, หรือ ลบ

```

1 graph TD
2   A[Start: Admin Dashboard] --> B[Select Entity Type e.g. Trainer]
3   B --> C[View List]
4   C --> D[Choose Action]
5   D -- Add New --> E[Fill Input Form]
6   D -- Edit --> F[Modify Existing Data]
7   D -- Delete --> G[Confirm Deletion]
8   E --> H[Validate Input?]
9   F --> H
10  H -- Invalid --> I[Show Validation Error]
11  H -- Valid --> J[Send API Request]
12  G --> J
13  J --> K[Database Operation]
14  K --> L[Log System Action]
15  L --> M[Update UI List]
16  M --> N[End]

```

Listing 3.2: Mermaid Code for Data Management

Figure 3.2: โค้ด Mermaid แสดงกระบวนการจัดการข้อมูลของผู้ดูแลระบบ

3. การตรวจสอบ: สำหรับการเพิ่ม/แก้ไข ระบบตรวจสอบข้อกำหนด (เช่น ชื่อห้ามว่าง, อีเมลต้องไม่ซ้ำ)
4. การดำเนินการ: ส่งคำขอไปยัง API Server ดำเนินการ CRUD กับฐานข้อมูล PostgreSQL
5. การบันทึก: เมื่อสำเร็จ ฟังก์ชัน 'logAction()' จะบันทึกเหตุการณ์
6. การตอบสนอง: รายการใน UI จะถูกรีเฟรชเพื่อแสดงการเปลี่ยนแปลงทันที

3.3 แผนภาพลำดับ (Sequence Diagrams)

แผนภาพ ลำดับ แสดง รายละเอียด การโต้ตอบ ทางเทคนิค ระหว่าง Frontend (View), Backend (Controller) และ Database (Model) ตามลำดับเวลา

3.3.1 กระบวนการเข้าสู่ระบบของผู้ใช้ (The User Login Process)

แผนภาพนี้แสดงขั้นตอนการยืนยันตัวตนตั้งแต่ HTTP Request แรกจนถึงการสร้าง Session ฝั่งไคลเอนต์

```
1 sequenceDiagram
2     participant User
3     participant Browser (Frontend)
4     participant Server (Express.js)
5     participant Database (PostgreSQL)
6
7     User->>Browser: Enter Email & Phone
8     User->>Browser: Click "Sign In"
9     Browser->>Server: POST /api/login {email, phone}
10    activate Server
11    Server->>Database: SELECT * FROM Customers WHERE email = ?
12    activate Database
13    Database-->>Server: Return User Record
14    deactivate Database
15
16    alt User Found
17        Server->>Server: Validate Phone Number
18        Server-->>Browser: HTTP 200 OK {user, role}
19        Browser->>Browser: localStorage.setItem('user', data)
20        Browser->>User: Redirect to Dashboard
21    else User Not Found
22        Server-->>Browser: HTTP 401 Unauthorized
23        Browser->>User: Show Error Message
24    end
25    deactivate Server
```

Listing 3.3: Mermaid Code for Login Process

Figure 3.3: โค้ด Mermaid แสดงลำดับการเข้าสู่ระบบ

คำอธิบายลำดับเหตุการณ์:

1. ผู้ใช้ กรอกข้อมูลในฝั่งไคลเอนต์
2. เบราร์เซอร์ ส่งคำขอ 'fetch' แบบ Asynchronous (POST) ไปยัง '/api/login'
3. เซิร์ฟเวอร์ รับคำขอและใช้ Sequelize ค้นหาใน ฐานข้อมูล
4. ฐานข้อมูล ส่งคืนระเบียบผู้ใช้ที่ตรงกับอีเมล
5. เซิร์ฟเวอร์ เปรียบเทียบเบอร์โทรศัพท์ที่ระบุกับที่เก็บไว้
6. หากถูกต้อง เซิร์ฟเวอร์ตอบกลับด้วย JSON Object ที่มีโปรไฟล์และบทบาทของผู้ใช้
7. เบราร์เซอร์ เก็บข้อมูล Session นี้ใน 'localStorage' เพื่อคงสถานะและนำทางผู้ใช้ต่อไปต่อ

3.3.2 กระบวนการจอง (The Booking Process)

แผนภาพนี้แสดงขั้นตอนตั้งแต่ต้นจนจบของการสร้างเซสชันการฝึกซ้อมใหม่

คำอธิบายลำดับเหตุการณ์:

1. การตรวจสอบความพร้อม: เมื่อสมาชิกเลือกเวลา UI ร้องขอข้อมูลเซสชันที่มีอยู่จาก API เพื่อตรวจสอบความขัดแย้ง
2. การกรอก: ตรรกะของ UI ใช้ผลลัพธ์จาก API เพื่อปิดการใช้งานตัวเลือกที่ไม่ว่าง

```

1 sequenceDiagram
2     participant Member
3     participant UI (Booking Modal)
4     participant API (Controller)
5     participant DB (TrainingSessions)
6
7     Member->>UI: Select Date, Time, Duration
8     UI->>API: GET /api/sessions?date=...
9     activate API
10    API->>DB: SELECT * FROM TrainingSessions WHERE date = ...
11    activate DB
12    DB-->>API: Return List of Sessions
13    deactivate DB
14    API-->>UI: Return Occupied Slots
15    deactivate API
16
17    UI->>UI: Filter Available Trainers/Equipment
18    Member->>UI: Select Trainer & Equipment
19    Member->>UI: Confirm Booking
20
21    UI->>API: POST /api/sessions {userId, trainerId, ...}
22    activate API
23    API->>DB: INSERT INTO TrainingSessions VALUES (...)
24    activate DB
25    DB-->>API: Success (New ID)
26    deactivate DB
27    API-->>UI: HTTP 201 Created
28    deactivate API
29
30    UI->>Member: Show Success Alert
31    UI->>UI: Refresh Schedule Grid

```

Listing 3.4: Mermaid Code for Booking Process

Figure 3.4: โค้ด Mermaid แสดงลำดับการจอง

3. การส่งข้อมูล: สมาชิกกดส่งแบบฟอร์มการจอง

4. การบันทึก: API รับคำขอ POST และใช้เมธอด 'create()' ของ Sequelize เพื่อเพิ่มแถวใหม่ใน DB

5. การยืนยัน: เมื่อบันทึกสำเร็จ ฐานข้อมูลยืนยันธุรกรรม API ส่งสถานะ 201 กลับไปยังไคลเอนต์ เพื่อแสดงแจ้งเตือนความสำเร็จ

บทที่ 4

การวิเคราะห์และออกแบบระบบ

4.1 สถาปัตยกรรมระบบ (System Architecture)

4.1.1 ภาพรวมสถาปัตยกรรม (Architecture Overview)

ระบบ GymBro ถูกออกแบบด้วยสถาปัตยกรรมแบบ **Client-Server Architecture** โดยมีการแยกส่วนการทำงานระหว่างส่วนติดต่อผู้ใช้ (Frontend) และส่วนประมวลผลหลัก (Backend) อย่างชัดเจน เพื่อความยืดหยุ่นในการพัฒนาและดูแลรักษา โดยมีการเชื่อมต่อข้อมูลผ่าน RESTful API

ส่วนประกอบหลักของระบบ (System Components)

1. Frontend (Client Side):

- พัฒนาด้วย **HTML5** และ **EJS (Embedded JavaScript templates)** สำหรับการแสดงผล
- ใช้ **Tailwind CSS** สำหรับการจัดรูปแบบหน้าจอ (Styling) ให้มีความทันสมัยและรองรับการแสดงผลบนอุปกรณ์ต่างๆ (Responsive Design)
- ใช้ **Vanilla JavaScript (ES6+)** ในการจัดการ Logic ฝั่งไคลเอนต์ เช่น การตรวจสอบข้อมูลในฟอร์ม (Form Validation), การเรียกใช้งาน API (Fetch API), และการจัดการ Session ผ่าน localStorage
- ใช้ **Lucide Icons** สำหรับแสดงไอคอนประกอบในส่วนติดต่อผู้ใช้

2. Backend (Server Side):

- พัฒนากับ **Node.js Runtime Environment**
- ใช้ **Express.js Framework** ในการสร้าง Web Server และจัดการเส้นทาง (Routing) ของ API
- ใช้ **Sequelize ORM** เป็นตัวกลางในการติดต่อกับฐานข้อมูล เพื่อให้สามารถจัดการข้อมูลในรูปแบบ Object-Oriented แทนการเขียน SQL Query โดยตรง

3. Database (Data Layer):

- ใช้ฐานข้อมูล **PostgreSQL** ซึ่งเป็น Relational Database Management System (RDBMS) ที่มีความเสถียรสูง
- ติดตั้งและใช้งานบนระบบคลาวด์ (Cloud Database) ผ่านผู้ให้บริการ Ruk-Com Cloud

4.2 ความต้องการของระบบ (System Requirements)

4.2.1 ความต้องการฟังก์ชันการทำงาน (Functional Requirements)

1. ระบบการยืนยันตัวตนและความปลอดภัย (Authentication & Security)

- การเข้าสู่ระบบ (Login):
 - รองรับการเข้าสู่ระบบโดยใช้อีเมล (Email) และเบอร์โทรศัพท์ (Phone) แทนรหัสผ่าน
 - มีระบบตรวจสอบสิทธิ์ (Role-based Authentication) เพื่อแยกการเข้าถึงระหว่างผู้ดูแลระบบ (Admin) และสมาชิกทั่วไป (Member)
 - บัญชีผู้ดูแลระบบถูกกำหนดสิทธิ์ไว้ในระบบ (Hardcoded Admin) คือ `admin@gymbro.com`

- **การจัดการเซสชัน (Session Management):**

- บันทึกสถานะการเข้าสู่ระบบของผู้ใช้ใน localStorage ของเบราว์เซอร์
- มีระบบ **Auth Guard** ตรวจสอบสิทธิ์ก่อนการเข้าถึงหน้าต่างๆ หากไม่มีสิทธิ์จะถูกส่งกลับไปหน้า Login โดยอัตโนมัติ
- รองรับการออกจากระบบ (Logout) ซึ่งจะทำการลบข้อมูลเซสชันและนำผู้ใช้กลับสู่หน้าแรก

- **การสมัครสมาชิก (Registration):**

- ผู้ใช้งานทั่วไปสามารถลงทะเบียนผ่านหน้าเว็บไซต์ได้
- ระบบตรวจสอบความซ้ำซ้อนของอีเมล (Email Uniqueness Check)
- กำหนดค่าเริ่มต้นให้สมาชิกใหม่ทันที: ประเภท "Member", ระดับ "Beginner", และมีอายุสมาชิก 1 ปี

2. ระบบสำหรับสมาชิก (Member Portal Features)

- **แดชบอร์ดสมาชิก (User Dashboard):**

- แสดงตารางการจองเซสชันการฝึกซ้อมของวันนี้ (Today's Schedule)
- แสดงสถานะการจองด้วยสีที่แตกต่าง: รายการของฉัน (My Booking - สีเขียว), ถูกจองแล้ว (Occupied - สีเทา)
- แสดงเวลาในรูปแบบท้องถิ่น (Thai Locale Format)

- **ระบบการจองเซสชัน (Booking System):**

- **การตรวจสอบความพร้อม (Real-time Availability Check):** ระบบจะตรวจสอบตารางเวลาของเทรนเนอร์และอุปกรณ์ทันทีที่ผู้ใช้เลือกวันและเวลา เพื่อป้องกันการจองซ้อนทับ (Double Booking)
- **ตัวเลือกอัจฉริยะ (Smart Dropdowns):** รายชื่อเทรนเนอร์และอุปกรณ์จะแสดงสถานะ "(Busy)" หรือถูกปิดการเลือก (Disabled) หากไม่ว่างในช่วงเวลานั้น
- **สถานะซ่อมบำรุง (Maintenance Check):** อุปกรณ์ที่อยู่ในสถานะ "Maintenance" จะไม่สามารถเลือกจองได้
- **การยกเลิกการจอง (Cancellation):** สมาชิกสามารถยกเลิกการจองของตนเองได้ผ่านหน้า Dashboard

- **โปรไฟล์ผู้ใช้ (User Profile):**

- แสดงข้อมูลส่วนตัว, ประเภทสมาชิก, และวันหมดอายุสมาชิก

3. ระบบสำหรับผู้ดูแลระบบ (Admin Back Office Features)

- **แดชบอร์ดผู้ดูแลระบบ (Admin Dashboard):**

- แสดงสถิติภาพรวม: จำนวนลูกค้าทั้งหมด, จำนวนเทรนเนอร์, จำนวนอุปกรณ์, และจำนวนเซสชันที่มีการจองในวันนี้

- **การจัดการข้อมูลพื้นฐาน (Master Data Management):**

- **จัดการลูกค้า (Customer Management):** ดูรายการ, เพิ่ม, ลบ, และแก้ไขข้อมูลลูกค้า (ใช้ Popup Prompt ในการแก้ไขข้อมูล)
- **จัดการเทรนเนอร์ (Trainer Management):** ดูรายการ, เพิ่ม, ลบ, และแก้ไขข้อมูลเทรนเนอร์ (ชื่อ, ระดับ, ความเชี่ยวชาญ, เบอร์โทร)
- **จัดการอุปกรณ์ (Equipment Management):** ดูรายการ, เพิ่ม, ลบ, และแก้ไขสถานะอุปกรณ์ (Active/Maintenance)

- **การจัดการตารางฝึกซ้อม (Session Management):**

- ดูรายการการจองทั้งหมดในระบบ
- ค้นหา/กรองรายการจองตามวันที่ (Filter by Date)
- ผู้ดูแลระบบมีสิทธิ์ยกเลิกการจองของสมาชิกได้ทุกรายการ

- **ดูบันทึกการทำงาน (System Logs):**

- ตรวจสอบประวัติการใช้งานระบบ (Action Logs) ย้อนหลังได้

4. ระบบงานเบื้องหลัง (Background Services & Logic)

- **การตรวจสอบความขัดแย้งของเวลา (Time Conflict Detection Algorithm):**

- ใช้ตรรกะทางคณิตศาสตร์ตรวจสอบช่วงเวลาทับซ้อน: $(Start_A < End_B) \wedge (End_A > Start_B)$
- ตรวจสอบทั้งฝั่งเทรนเนอร์และอุปกรณ์พร้อมกัน

- **ระบบลบสมาชิกอัตโนมัติ (Auto-Cleanup Service):**

- มี Scheduled Task ทำงานทุก 24 ชั่วโมง เพื่อตรวจสอบสมาชิกที่หมดอายุ (Expired Members)
- หากพบสมาชิกหมดอายุ ระบบจะลบข้อมูลสมาชิกคนนั้น พร้อมทั้งลบประวัติการจองทั้งหมดที่เกี่ยวข้อง (Cascade Delete) เพื่อคืนพื้นที่ฐานข้อมูล

- **ระบบจัดเรียง ID (Smart ID Reordering):**

- เมื่อมีการลบข้อมูล (User/Trainer/Equipment) ระบบจะทำการจัดลำดับ ID ของข้อมูลที่เหลือใหม่อัตโนมัติ เพื่อให้ ID เรียงกันสวยงามและไม่กระโดดข้าม

4.2.2 ความต้องการที่ไม่ใช่ฟังก์ชันการทำงาน (Non-Functional Requirements)

- **ประสิทธิภาพ (Performance):**

- รองรับการทำงานแบบ Asynchronous (Async/Await) เพื่อไม่ให้ Server หยุดชะงักระหว่างรอฐานข้อมูลตอบกลับ

- **ความน่าเชื่อถือของข้อมูล (Data Integrity):**

- ใช้ Foreign Key Constraints ในระดับ Application Code เพื่อรักษาความถูกต้องของความสัมพันธ์ข้อมูล

- **ความปลอดภัย (Security):**

- แยกไฟล์ Configuration (.env) ไม่ให้รวมอยู่ใน Source Code
- จำกัดการเข้าถึง API ด้วย CORS Policy (อนุญาตเฉพาะ Frontend ที่กำหนด)

บทที่ 5

ER Diagram

แผนภาพความสัมพันธ์ระหว่างข้อมูล (Entity Relationship Diagram) ของระบบบริหารจัดการยิม (GymBro Management System) แสดงถึงโครงสร้างการจัดเก็บข้อมูลเชิงสัมพันธ์ (Relational Database) โดยเน้นที่การเชื่อมโยงข้อมูลหลัก 4 ส่วน ได้แก่ ข้อมูลลูกค้า (Customers), ข้อมูลเทรนเนอร์ (Trainers), ข้อมูลอุปกรณ์ (GymEquipments), และข้อมูลการจอบ (TrainingSessions) ซึ่งมีความสัมพันธ์กันดังปรากฏในรูปภาพต่อไปนี้

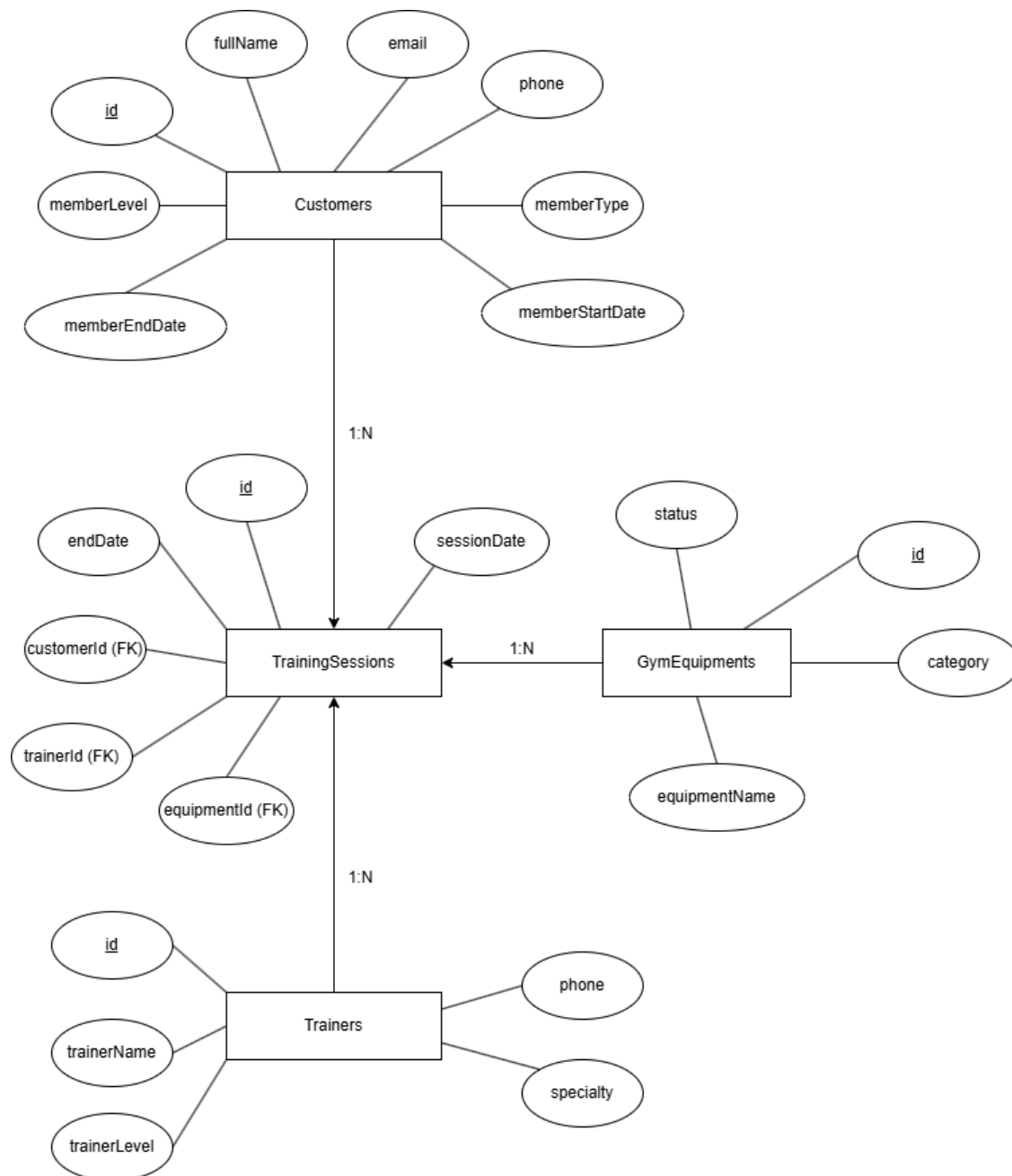


Figure 5.1: ER Diagram แสดงความสัมพันธ์ระหว่างตารางในฐานข้อมูล

5.1 คำอธิบายความสัมพันธ์ (Relationships Explanation)

ระบบฐานข้อมูลถูกออกแบบตามหลักการ Normalization เพื่อลดความซ้ำซ้อนของข้อมูล โดยมีความสัมพันธ์แบบ One-to-Many (1:N) ระหว่างตารางหลัก (Master Tables) และตารางรายการเคลื่อนไหว (Transaction Table) ดังนี้:

1. **Customers (1) → TrainingSessions (N):** ความสัมพันธ์ระหว่างลูกค้าและการจอง เป็นแบบ 1-ต่อ-กลุ่ม โดยลูกค้าหนึ่งคน (Customer) สามารถทำรายการจองเซสชันการฝึกซ้อม (Training Sessions) ได้หลายครั้งในช่วงเวลาที่ต่างกัน แต่แต่ละรายการจองจะเป็นของลูกค้าเพียงคนเดียว
2. **Trainers (1) → TrainingSessions (N):** ความสัมพันธ์ระหว่างเทรนเนอร์และการจอง เป็นแบบ 1-ต่อ-กลุ่ม โดยเทรนเนอร์หนึ่งคน (Trainer) สามารถรับผิดชอบดูแลการฝึกซ้อมได้หลายเซสชัน แต่ในหนึ่งเซสชันจะมีเทรนเนอร์ผู้ดูแลหลักเพียงคนเดียว
3. **GymEquipments (1) → TrainingSessions (N):** ความสัมพันธ์ระหว่างอุปกรณ์และการจอง เป็นแบบ 1-ต่อ-กลุ่ม โดยอุปกรณ์หนึ่งชิ้น (Equipment) สามารถถูกจองใช้งานได้หลายเซสชัน (คนละช่วงเวลา) แต่ในแต่ละเซสชันจะมีการระบุการจองใช้อุปกรณ์หลักเพียงหนึ่งชิ้น

5.2 รายละเอียดโครงสร้างตาราง (Table Schema Detail)

รายละเอียดของคอลัมน์ (Attributes) ประเภทข้อมูล (Data Types) และข้อจำกัด (Constraints) ของแต่ละตาราง มีดังนี้:

5.2.1 1. ตาราง Customers (ข้อมูลสมาชิกและโปรไฟล์ผู้ใช้)

ตารางหลักสำหรับเก็บข้อมูลส่วนตัวของผู้ใช้งาน (User Profile) และสถานะสมาชิก

- **id (PK):** รหัสสมาชิก (Integer, Auto Increment, SERIAL) เป็น Primary Key สำหรับอ้างอิง
- **fullName:** ชื่อ-นามสกุลจริงของผู้ใช้งาน (VARCHAR(150), NOT NULL)
- **email:** อีเมลสำหรับเข้าสู่ระบบและติดต่อ (VARCHAR(150), NOT NULL, UNIQUE) ห้ามซ้ำกันในระบบ
- **phone:** เบอร์โทรศัพท์ (VARCHAR(20)) ใช้เป็นรหัสผ่านทางเลือกในการเข้าสู่ระบบ
- **memberType:** ประเภทระดับสมาชิก (VARCHAR(50), NOT NULL) เช่น 'Standard', 'Premium', 'VIP'
- **memberLevel:** ระดับความสามารถสมาชิก (VARCHAR(50), NOT NULL) เช่น 'Basic', 'Silver', 'Gold'
- **memberStartDate:** วันที่และเวลาเริ่มต้นเป็นสมาชิก (TIMESTAMP, NOT NULL)
- **memberEndDate:** วันที่และเวลาสิ้นสุดสมาชิก (TIMESTAMP) หากเป็น NULL หมายถึงสมาชิกตลอดชีพ

5.2.2 2. ตาราง Trainers (ข้อมูลเทรนเนอร์)

ตารางเก็บข้อมูลรายชื่อเทรนเนอร์และคุณสมบัติ

- **id (PK):** รหัสเทรนเนอร์ (Integer, Auto Increment, SERIAL)
- **trainerName:** ชื่อเทรนเนอร์ (VARCHAR(150), NOT NULL)
- **trainerLevel:** ระดับประสบการณ์ของเทรนเนอร์ (VARCHAR(50), NOT NULL) เช่น 'Junior', 'Senior', 'Master'
- **specialty:** ความเชี่ยวชาญพิเศษ (VARCHAR(100)) เช่น 'Yoga', 'Body Building', 'Cardio'
- **phone:** เบอร์โทรศัพท์ติดต่อ (VARCHAR(20))

5.2.3 3. ตาราง GymEquipments (ข้อมูลอุปกรณ์)

ตารางเก็บข้อมูลอุปกรณ์ออกกำลังกายที่ให้บริการในยิม

- **id (PK):** รหัสอุปกรณ์ (Integer, Auto Increment, SERIAL)
- **equipmentName:** ชื่อเรียกอุปกรณ์ (VARCHAR(150), NOT NULL)
- **category:** หมวดหมู่อุปกรณ์ (VARCHAR(100)) เช่น 'Strength', 'Cardio', 'Flexibility'
- **status:** สถานะความพร้อมใช้งาน (VARCHAR(50), NOT NULL) เช่น 'Available', 'In Use', 'Maintenance'

5.2.4 4. ตาราง TrainingSessions (ตารางการจองฝึกซ้อม)

ตารางเก็บข้อมูล Transaction การจองคลาสหรือเซสชันการฝึกซ้อม เชื่อมโยงความสัมพันธ์ (Foreign Keys) ไปยังตารางอื่นๆ

- **id (PK):** รหัสรายการจอง (Integer, Auto Increment, SERIAL)
- **sessionDate:** วันและเวลาเริ่มต้นการฝึกซ้อม (TIMESTAMP, NOT NULL)
- **endDate:** วันและเวลาสิ้นสุดการฝึกซ้อม (TIMESTAMP) คำนวณอัตโนมัติจากระยะเวลาเซสชัน
- **customerId (FK):** รหัสสมาชิกผู้จอง (Integer, NOT NULL)
อ้างอิง (REFERENCES) ไปยังตาราง **Customers(id)**
กำหนด **ON DELETE CASCADE** (หากลบสมาชิก ประวัติการจองจะถูกลบด้วย)
- **trainerId (FK):** รหัสเทรนเนอร์ผู้ดูแล (Integer, NOT NULL)
อ้างอิง (REFERENCES) ไปยังตาราง **Trainers(id)**
- **equipmentId (FK):** รหัสอุปกรณ์ที่ใช้ (Integer, NOT NULL)
อ้างอิง (REFERENCES) ไปยังตาราง **GymEquipments(id)**

บทที่ 6

ตาราง Database

6.1 Database (ฐานข้อมูล)

6.1.1 ภาพรวมฐานข้อมูล (Database Overview)

ระบบ GymBro เลือกใช้ **PostgreSQL** ซึ่งเป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์ (Relational Database Management System - RDBMS) ที่มีความเสถียร ประสิทธิภาพสูง และรองรับมาตรฐาน SQL อย่างครบถ้วน โดยฐานข้อมูลถูกโฮสต์อยู่บน **Ruk-Com Cloud** ซึ่งเป็นผู้ให้บริการ Cloud Hosting ที่มีความน่าเชื่อถือและประสิทธิภาพสูงในประเทศไทย

การเลือกใช้ PostgreSQL บน Ruk-Com Cloud ช่วยให้ระบบมีความยืดหยุ่นในการขยายตัว (Scalability) รองรับการใช้งานจากผู้ใช้งานจำนวนมากพร้อมกัน และมีการสำรองข้อมูลอัตโนมัติ (Automated Backup) เพื่อป้องกันการสูญหายของข้อมูลสำคัญ

6.1.2 การเชื่อมต่อฐานข้อมูล (Database Connection)

ระบบ Backend เชื่อมต่อกับฐานข้อมูลผ่าน **Sequelize ORM** ซึ่งช่วยลดความซับซ้อนในการเขียนคำสั่ง SQL ดิบ (Raw SQL) และช่วยให้โค้ดมีความเป็นระเบียบ อ่านง่าย และดูแลรักษาได้สะดวก

6.1.3 db.js

ภาพรวม (Overview)

ไฟล์ db.js รับผิดชอบการจัดการการเชื่อมต่อฐานข้อมูล PostgreSQL โดยใช้ Sequelize ORM รวมถึงการนิยาม Model (Schema) ของตารางต่างๆ ภายในระบบ และการกำหนดความสัมพันธ์ (Associations) ระหว่างตารางเหล่านั้น

การเชื่อมต่อฐานข้อมูล (Database Connection)

```
1 const path = require("path");
2 require("dotenv").config({ path: path.resolve(__dirname, ".env") });
3 const { Sequelize, DataTypes } = require("sequelize");
4
5 const dbUrl = process.env.DATABASE_URL ||
6   ↳ \discretionary{}{}{} "postgres://webadmin:...@.../gymbro_db";
7
8 const sequelize = new Sequelize(dbUrl, {
9   dialect: "postgres",
10  logging: false,
11 });
```

Listing 6.1: ซอร์สโค้ดจากไฟล์ db.js

คำอธิบาย:

1. โหลดค่าตัวแปรสภาพแวดล้อม (Environment Variables) และเตรียม URL สำหรับการเชื่อมต่อฐานข้อมูล
2. สร้าง Instance ของ Sequelize โดยระบุ Dialect เป็น postgres และปิดการแสดงผล Logging เพื่อลดความซับซ้อนของข้อมูลใน Console

การนิยามแบบจำลองข้อมูล (Models Definition)

ส่วนนี้เป็นการกำหนดโครงสร้างของตารางต่างๆ ในฐานข้อมูล

แบบจำลองข้อมูลลูกค้า (Customers Model)

```
1 const Customers = sequelize.define(  
2   "Customers",  
3   {  
4     id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },  
5     fullName: { type: DataTypes.STRING(150), allowNull: false },  
6     email: { type: DataTypes.STRING(150), allowNull: false },  
7     phone: { type: DataTypes.STRING(20) },  
8     memberType: { type: DataTypes.STRING(50), allowNull: false },  
9     memberLevel: { type: DataTypes.STRING(50), allowNull: false },  
10    memberStartDate: { type: DataTypes.DATE, allowNull: false },  
11    memberEndDate: { type: DataTypes.DATE },  
12  },  
13  { tableName: "Customers", timestamps: false, freezeTableName: true }  
14 );
```

Listing 6.2: ซอร์สโค้ดจากไฟล์ db.js

คำอธิบาย:

1. นิยามตาราง Customers สำหรับจัดเก็บข้อมูลลูกค้า
2. กำหนดคอลัมน์ต่างๆ เช่น fullName, email, memberType พร้อมทั้งระบุชนิดข้อมูลและข้อจำกัด (Constraints)
3. ปิดการใช้งาน timestamps (created_at, updated_at) และใช้ freezeTableName เพื่อบังคับให้ใช้ชื่อตารางตามที่กำหนดไว้อย่างเคร่งครัด

แบบจำลองข้อมูลเซสชันการฝึกซ้อมและความสัมพันธ์ (TrainingSessions Model & Associations)

```
1 const TrainingSessions = sequelize.define(  
2   "TrainingSessions",  
3   {  
4     id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },  
5     sessionDate: { type: DataTypes.DATE, allowNull: false },  
6     endDate: { type: DataTypes.DATE },  
7     customerId: { type: DataTypes.INTEGER, allowNull: false },  
8     trainerId: { type: DataTypes.INTEGER, allowNull: true },  
9     equipmentId: { type: DataTypes.INTEGER, allowNull: false },  
10  },  
11  { tableName: "TrainingSessions", timestamps: false, freezeTableName: true }  
12 );  
13  
14 TrainingSessions.belongsTo(Customers, { as: "customer", foreignKey: "customerId" });  
15 TrainingSessions.belongsTo(Trainers, { as: "trainer", foreignKey: "trainerId" });  
16 TrainingSessions.belongsTo(GymEquipments, { as: "equipment", foreignKey: "equipmentId" });  
17 Customers.hasMany(TrainingSessions, { as: "sessions", foreignKey: "customerId" });  
18 // ... (Similar relations for Trainers and GymEquipments)
```

Listing 6.3: ซอร์สโค้ดจากไฟล์ db.js

คำอธิบาย:

1. นิยาม ตาราง TrainingSessions ซึ่ง ประกอบด้วย Foreign Keys ได้แก่ customerId, trainerId, และ equipmentId
2. การกำหนดความสัมพันธ์ (Associations):
 - (a) belongsTo: กำหนดความสัมพันธ์แบบ Many-to-One ระบุว่าเซสชันหนึ่งๆ เป็นของลูกค้า, เทรนเนอร์, และอุปกรณ์ใด
 - (b) hasMany: กำหนดความสัมพันธ์แบบ One-to-Many ระบุว่าลูกค้าหนึ่งคน (รวมถึงเทรนเนอร์และอุปกรณ์) สามารถมีได้หลายเซสชัน
3. การตั้งค่านีช่วยให้ Sequelize สามารถทำการ Join ตารางโดยอัตโนมัติเมื่อมีการใช้คำสั่ง include ในขั้นตอนการ Query ข้อมูล

6.1.4 schema.sql

ภาพรวม (Overview)

ไฟล์ `schema.sql` คือชุดคำสั่ง SQL (Structured Query Language) สำหรับสร้างโครงสร้างตารางในฐานข้อมูล PostgreSQL ไฟล์นี้เปรียบเสมือนพิมพ์เขียว (Blueprint) หลักของฐานข้อมูล ใช้สำหรับการอ้างอิงโครงสร้างที่แท้จริง หรือใช้สำหรับการเริ่มระบบฐานข้อมูลใหม่ (Initialize) ในสภาพแวดล้อมที่ไม่ได้ใช้งาน ORM

การนิยามตาราง (Table Definitions)

ตารางลูกค้า (Customers Table)

```
1 CREATE TABLE IF NOT EXISTS "Customers" (  
2   id SERIAL PRIMARY KEY,  
3   "fullName" VARCHAR(150) NOT NULL,  
4   email VARCHAR(150) NOT NULL,  
5   phone VARCHAR(20),  
6   "memberType" VARCHAR(50) NOT NULL,  
7   "memberLevel" VARCHAR(50) NOT NULL,  
8   "memberStartDate" TIMESTAMP NOT NULL,  
9   "memberEndDate" TIMESTAMP  
10 );
```

Listing 6.4: ซอร์สโค้ดจากไฟล์ `schema.sql`

คำอธิบาย:

1. สร้างตาราง Customers หากยังไม่มีอยู่ในระบบ (IF NOT EXISTS)
2. id SERIAL PRIMARY KEY: กำหนดให้ id เป็น Primary Key และมีการเพิ่มค่าโดยอัตโนมัติ (Auto Increment)
3. คอลัมน์อื่นๆ มีการกำหนดชนิดข้อมูล (Data Type) ให้สอดคล้องกับที่ได้นิยามไว้ใน Sequelize Model

ตารางเทรนเนอร์ (Trainers Table)

```
1 CREATE TABLE IF NOT EXISTS "Trainers" (  
2   id SERIAL PRIMARY KEY,  
3   "trainerName" VARCHAR(150) NOT NULL,  
4   "trainerLevel" VARCHAR(50) NOT NULL,  
5   specialty VARCHAR(100),  
6   phone VARCHAR(20)  
7 );
```

Listing 6.5: ซอร์สโค้ดจากไฟล์ `schema.sql`

คำอธิบาย:

1. สร้างตาราง Trainers สำหรับจัดเก็บข้อมูลเทรนเนอร์
2. trainerName และ trainerLevel ถูกกำหนดเป็น NOT NULL เพื่อบังคับให้ต้องมีข้อมูลเสมอ

ตารางอุปกรณ์ยิม (GymEquipments Table)

```
1 CREATE TABLE IF NOT EXISTS "GymEquipments" (  
2   id SERIAL PRIMARY KEY,  
3   "equipmentName" VARCHAR(150) NOT NULL,  
4   category VARCHAR(100),  
5   status VARCHAR(50) NOT NULL  
6 );
```

Listing 6.6: ซอร์สโค้ดจากไฟล์ `schema.sql`

คำอธิบาย:

1. สร้างตาราง GymEquipments สำหรับจัดเก็บข้อมูลอุปกรณ์ภายในยิม
2. status ใช้สำหรับระบุสถานะความพร้อมใช้งานของอุปกรณ์ เช่น 'Available' หรือ 'Maintenance'

ตารางเซชันการฝึกซ้อมและ Foreign Keys (TrainingSessions Table)

```
1 CREATE TABLE IF NOT EXISTS "TrainingSessions" (  
2   id SERIAL PRIMARY KEY,  
3   "sessionId" TIMESTAMP NOT NULL,  
4   "customerId" INTEGER NOT NULL REFERENCES "Customers"(id) ON DELETE CASCADE,  
5   "trainerId" INTEGER NOT NULL REFERENCES "Trainers"(id),  
6   "equipmentId" INTEGER NOT NULL REFERENCES "GymEquipments"(id)  
7 );
```

Listing 6.7: ซอร์สโค้ดจากไฟล์ schema.sql

คำอธิบาย:

1. REFERENCES "Customers"(id): สร้าง Foreign Key เชื่อมโยงไปยังตาราง Customers เพื่อรักษาความสัมพันธ์ของข้อมูล
2. ON DELETE CASCADE: กำหนดกฎว่าหากข้อมูล Customer ถูกลบ ให้ทำการลบ Session ที่เกี่ยวข้องทั้งหมดทิ้งโดยอัตโนมัติ (ดำเนินการในระดับฐานข้อมูล)
3. มีการเชื่อมโยงกับตาราง Trainers และ GymEquipments ในลักษณะเดียวกัน เพื่อรักษาความถูกต้องของข้อมูล (Referential Integrity)

6.2 พจนานุกรมข้อมูล (Data Dictionary)

ส่วนนี้แสดงรายละเอียดโครงสร้างของตารางทั้งหมดในฐานข้อมูล (Database Schema) รวมถึงคำอธิบายของแต่ละฟิลด์

6.2.1 ตาราง: Customers

ตารางสำหรับจัดเก็บข้อมูลลูกค้าและสมาชิกของยิม

Table 6.1: รายละเอียดโครงสร้างตาราง Customers

ชื่อฟิลด์ (Field)	ชนิดข้อมูล (Type)	Key	คำอธิบาย (Description)
id	INTEGER	PK	รหัส ระบุ ตัว ตน ลูกค้า (Customer ID) สร้างอัตโนมัติ
fullName	VARCHAR(150)		ชื่อและนามสกุลของผู้ใช้งาน
email	VARCHAR(150)		ที่อยู่ อีเมล สำหรับ การ ติดต่อ และ เข้า สู่ ระบบ
phone	VARCHAR(20)		หมายเลขโทรศัพท์สำหรับการติดต่อ
memberType	VARCHAR(50)		ประเภทของสมาชิก (เช่น รายวัน, รายเดือน, รายปี)
memberLevel	VARCHAR(50)		ระดับชั้นของสมาชิก (เช่น Bronze, Silver, Gold)
memberStartDate	TIMESTAMP		วันและเวลาที่เริ่มต้นสถานะสมาชิก
memberEndDate	TIMESTAMP		วันและเวลาที่สิ้นสุดสถานะสมาชิก

6.2.2 ตาราง: Trainers

ตารางสำหรับจัดเก็บข้อมูลเทรนเนอร์หรือผู้ฝึกสอน

Table 6.2: รายละเอียดโครงสร้างตาราง Trainers

ชื่อฟิลด์ (Field)	ชนิดข้อมูล (Type)	Key	คำอธิบาย (Description)
id	INTEGER	PK	รหัสระบุตัวตนเทรนเนอร์ (Trainer ID) สร้างอัตโนมัติ
trainerName	VARCHAR(150)		ชื่อและนามสกุลของเทรนเนอร์
trainerLevel	VARCHAR(50)		ระดับความเชี่ยวชาญของเทรนเนอร์
specialty	VARCHAR(100)		ความถนัดหรือประเภทกีฬาที่สอน (เช่น Yoga, Body Building)
phone	VARCHAR(20)		หมายเลขโทรศัพท์สำหรับการติดต่อ

6.2.3 ตาราง: GymEquipments

ตารางสำหรับจัดเก็บข้อมูลอุปกรณ์ออกกำลังกายในยิม

Table 6.3: รายละเอียดโครงสร้างตาราง GymEquipments

ชื่อฟิลด์ (Field)	ชนิดข้อมูล (Type)	Key	คำอธิบาย (Description)
id	INTEGER	PK	รหัสระบุตัวตนอุปกรณ์ (Equipment ID) สร้างอัตโนมัติ
equipmentName	VARCHAR(150)		ชื่อเรียกของอุปกรณ์ออกกำลังกาย
category	VARCHAR(100)		หมวดหมู่ของอุปกรณ์ (เช่น Cardio, Strength)
status	VARCHAR(50)		สถานะความพร้อมใช้งาน (เช่น Available, Maintenance)

6.2.4 ตาราง: TrainingSessions

ตารางสำหรับจัดเก็บข้อมูลการจองเซสชันการฝึกซ้อม เชื่อมโยงระหว่างลูกค้า, เทรนเนอร์ และอุปกรณ์

Table 6.4: รายละเอียดโครงสร้างตาราง TrainingSessions

ชื่อฟิลด์ (Field)	ชนิดข้อมูล (Type)	Key	คำอธิบาย (Description)
id	INTEGER	PK	รหัสระบุตัวตนเซสชัน (Session ID) สร้างอัตโนมัติ
sessionDate	TIMESTAMP		วันและเวลาที่เริ่มต้นเซสชันการฝึกซ้อม
endDate	TIMESTAMP		วันและเวลาที่สิ้นสุดเซสชันการฝึกซ้อม
customerId	INTEGER	FK	รหัสอ้างอิงลูกค้า (เชื่อมโยงกับ ตาราง Customers)
trainerId	INTEGER	FK	รหัสอ้างอิงเทรนเนอร์ (เชื่อมโยงกับ ตาราง Trainers)
equipmentId	INTEGER	FK	รหัสอ้างอิงอุปกรณ์ (เชื่อมโยงกับตาราง GymEquipments)

บทที่ 7

การใช้งานเว็บ

บทนี้อธิบายวิธีการใช้งานระบบ GymBro Management System อย่างละเอียด แบ่งตามสิทธิ์การใช้งานของผู้ใช้ 3 กลุ่มหลัก ได้แก่ ผู้ใช้งานทั่วไป (Public), สมาชิก (Member), และผู้ดูแลระบบ (Admin) โดยมีการแสดงภาพประกอบหน้าจอทั้งในโหมดปกติ (Light Mode) และโหมดมืด (Dark Mode)

7.1 ส่วนผู้ใช้งานทั่วไป (Public Section)

ส่วนนี้สำหรับผู้ที่ยังไม่ได้เข้าสู่ระบบ สามารถใช้งานได้ทุกคน

7.1.1 การใช้งานหน้าเข้าสู่ระบบ (Login Page)

หน้าแรกของระบบที่ใช้สำหรับยืนยันตัวตนก่อนเข้าใช้งาน

ขั้นตอนการเข้าสู่ระบบ

1. เปิดเว็บเบราว์เซอร์และเข้าสู่หน้าเว็บไซต์ ระบบจะแสดงแบบฟอร์ม Login
2. กรอก **Email Address** ที่ได้ลงทะเบียนไว้
3. กรอก **Phone Number** (ใช้แทนรหัสผ่าน)
4. กดปุ่ม **SIGN IN**
5. หากข้อมูลถูกต้อง ระบบจะตรวจสอบสิทธิ์ (Role) และเปลี่ยนหน้าไปยัง Dashboard ที่เหมาะสม (User หรือ Admin)
6. หากข้อมูลไม่ถูกต้อง ระบบจะแสดงข้อความแจ้งเตือนสีแดง "Invalid credentials"

7.1.2 การใช้งานหน้าลงทะเบียน (Register Page)

สำหรับผู้ใช้งานใหม่ที่ต้องการสมัครสมาชิก

ขั้นตอนการสมัครสมาชิก

1. ในหน้า Login ให้กดลิงก์ "Don't have an account? Register here"
2. ระบบจะพาไปที่หน้า Register
3. กรอก **Full Name** (ชื่อ-นามสกุล)
4. กรอก **Email Address** (ห้ามซ้ำกับที่มีในระบบ)
5. กรอก **Phone Number** (ใช้สำหรับล็อกอินในภายหลัง)
6. กดปุ่ม **CREATE ACCOUNT**

http://localhost:5500/login

LOGIN

GYMBRO
QUEUE BOOKING SYSTEM

EMAIL ADDRESS
john@gmail.com

PHONE NUMBER (PASSWORD)

SIGN IN

DON'T HAVE AN ACCOUNT? REGISTER HERE.

Figure 7.1: หน้าจอเข้าสู่ระบบ (Login) ประกอบด้วยช่องกรอกอีเมลและเบอร์โทรศัพท์ พร้อมปุ่มเข้าสู่ระบบ

http://localhost:5500/register

REGISTER

GYMBRO
JOIN OUR COMMUNITY

FULL NAME
John Chaorai

EMAIL ADDRESS
john@gmail.com

PHONE NUMBER (PASSWORD)
0899999999

CREATE ACCOUNT

ALREADY HAVE AN ACCOUNT? LOGIN HERE.

Figure 7.2: หน้าจอลงทะเบียน (Register) สำหรับกรอกข้อมูลส่วนตัวเพื่อสมัครสมาชิกใหม่

การรอการอนุมัติ (Pending Approval)

เมื่อสมัครสมาชิกสำเร็จ ระบบจะแสดงข้อความแจ้งเตือนว่าบัญชีอยู่ในสถานะ **รอการอนุมัติ (Pending)** ผู้ใช้งานจะต้องติดต่อเจ้าหน้าที่เพื่อชำระค่าบริการ และรอให้เจ้าหน้าที่ยืนยันสถานะบัญชีจึงจะสามารถใช้งานได้

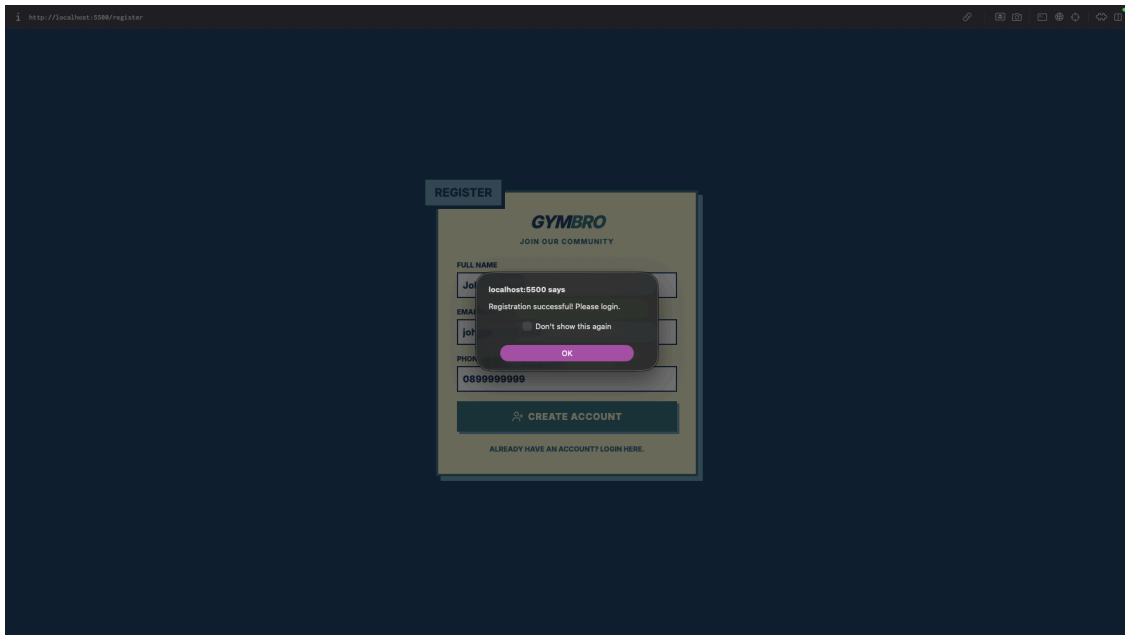


Figure 7.3: หน้าต่างแจ้งเตือนการสมัครสมาชิกสำเร็จและรอการอนุมัติจากผู้ดูแลระบบ

7.2 ส่วนสมาชิก (Member Section)

ส่วนนี้สำหรับผู้ใช้งานที่ล็อกอินด้วยสิทธิ์สมาชิก (User)

7.2.1 การใช้งานหน้าแดชบอร์ดสมาชิก (User Dashboard)

หน้าหลักของสมาชิก ใช้สำหรับดูตารางฝึกซ้อมและจองเซสชัน

การดูตารางฝึกซ้อม

1. เมื่อเข้าสู่ระบบสำเร็จ จะพบตารางแสดงรายการจองทั้งหมดในวันนี้ (All Bookings)
2. รายการที่เป็นของตนเอง (My Session) จะถูกไฮไลต์ด้วยสีพื้นหลังที่แตกต่างเพื่อให้สังเกตได้ง่าย
3. ตารางแสดงข้อมูลเวลา (Time), กิจกรรม/อุปกรณ์ (Activity), และชื่อเทรนเนอร์ (Trainer)

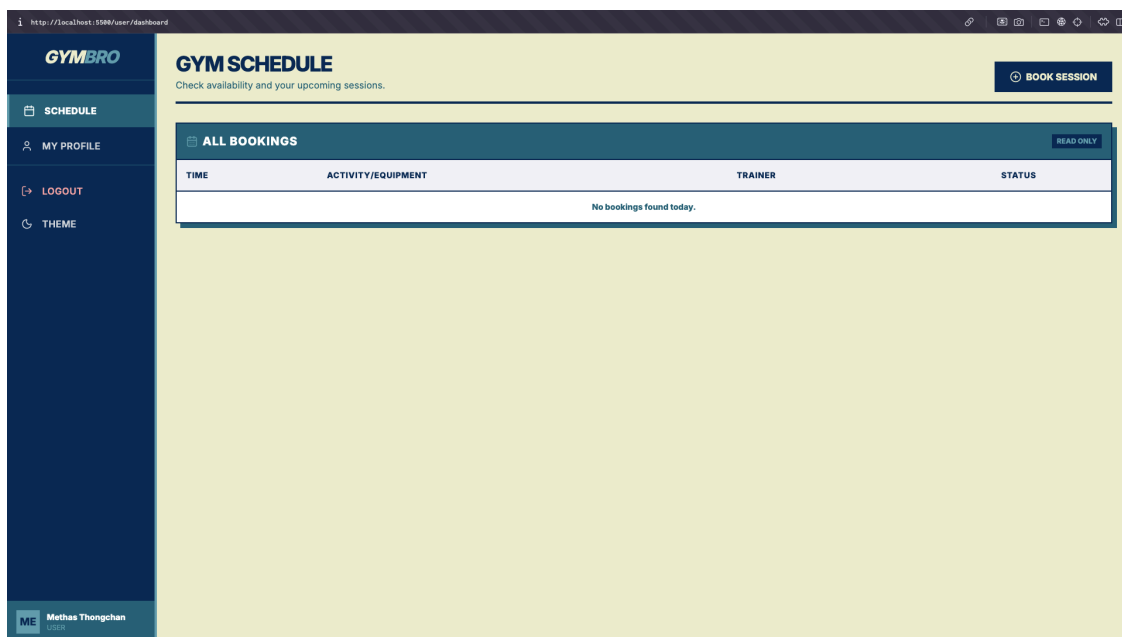


Figure 7.4: หน้าแดชบอร์ดสมาชิกแสดงตารางการฝึกซ้อม (User Schedule) พร้อมไฮไลต์รายการจองของตนเอง

7.2.2 การจองเซสชัน (Booking Session)

ขั้นตอนการจอง

1. กดปุ่ม **BOOK SESSION** ที่มุมขวาบน
2. ระบบจะแสดงหน้าต่าง (Modal) สำหรับกรอกข้อมูลการจอง
3. เลือก **Date** (วันที่), **Time** (เวลา), และ **Duration** (ระยะเวลา)
4. ระบบจะตรวจสอบความว่าง (Availability) ของเทรนเนอร์และอุปกรณ์โดยอัตโนมัติ หากไม่ว่างตัวเลือกนั้นจะถูกปิดการใช้งาน (Disabled)
5. เลือก **Trainer** และ **Equipment** ที่ต้องการ
6. กดปุ่ม **CONFIRM BOOKING**

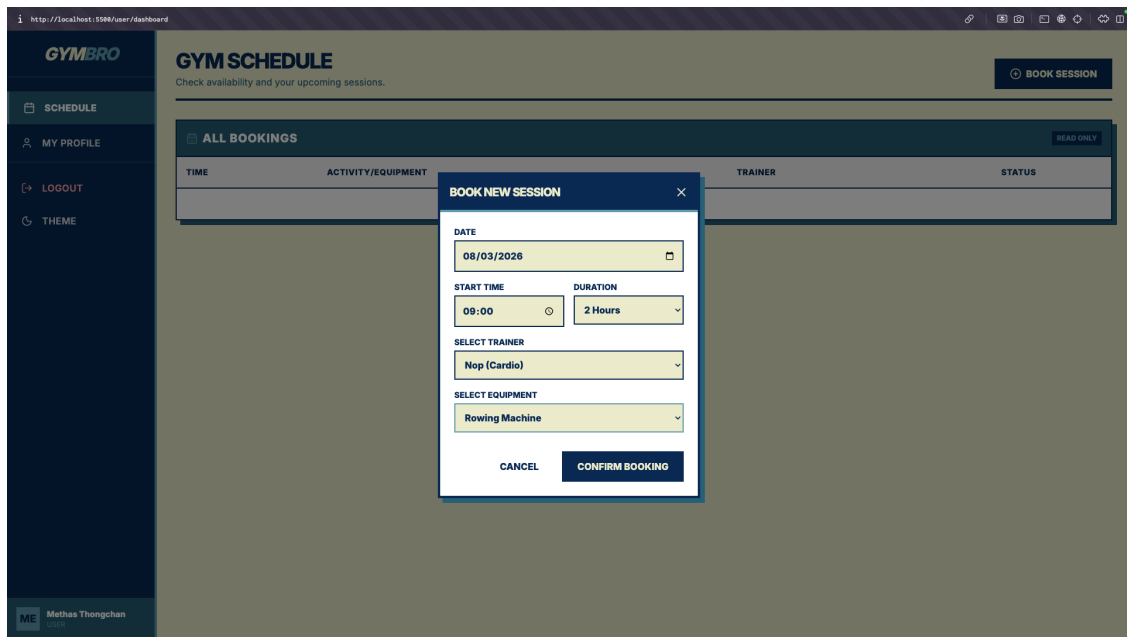


Figure 7.5: หน้าต่างสำหรับการจองเซสชัน (Booking Modal) ให้ผู้ใช้เลือกวัน เวลา เทรนเนอร์ และอุปกรณ์

การยืนยันการจอง

เมื่อทำการจองสำเร็จ ระบบจะแสดงข้อความแจ้งเตือน (Alert) เพื่อยืนยันว่าการจองได้รับการบันทึกเรียบร้อยแล้ว และตารางจะถูกรีเฟรชเพื่อแสดงรายการจองใหม่

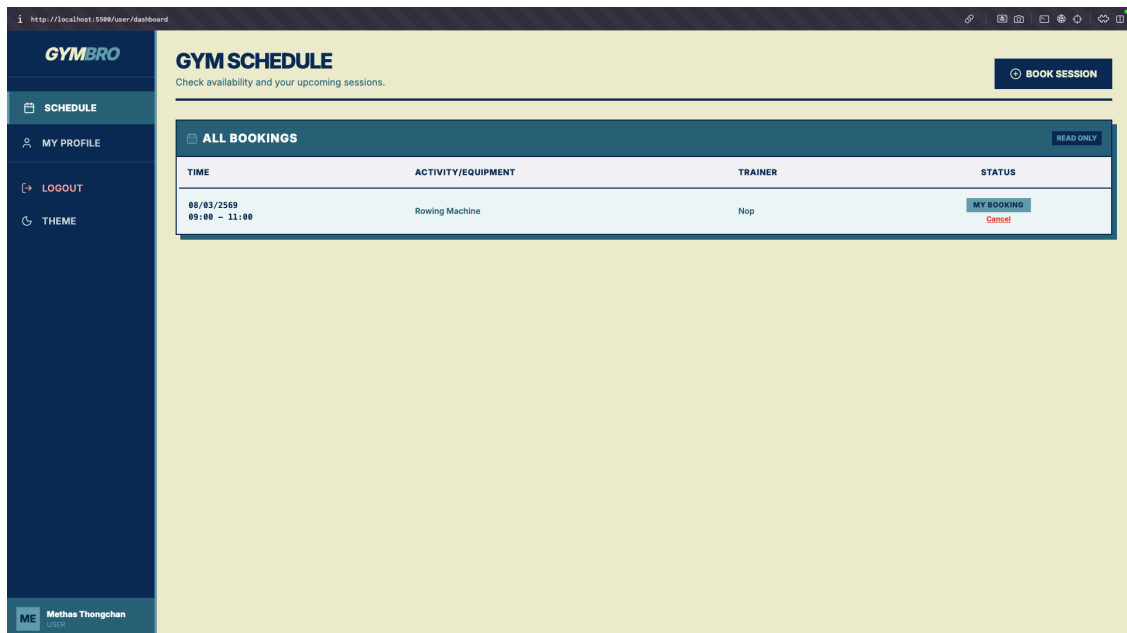


Figure 7.6: หน้าจอแสดงสถานะการจองสำเร็จ (Booking Success) พร้อมแสดงรายการใหม่ในตาราง

7.2.3 การใช้งานหน้าข้อมูลส่วนตัว (My Profile)

หน้าแสดงรายละเอียดข้อมูลสมาชิก และสถานะบัญชี (แสดงตัวอย่างในโหมดมืด)

1. ที่แถบเมนูด้านซ้าย เลือกเมนู **My Profile**
2. ระบบจะแสดงข้อมูลส่วนตัว ได้แก่ ชื่อ-นามสกุล, อีเมล, และเบอร์โทรศัพท์

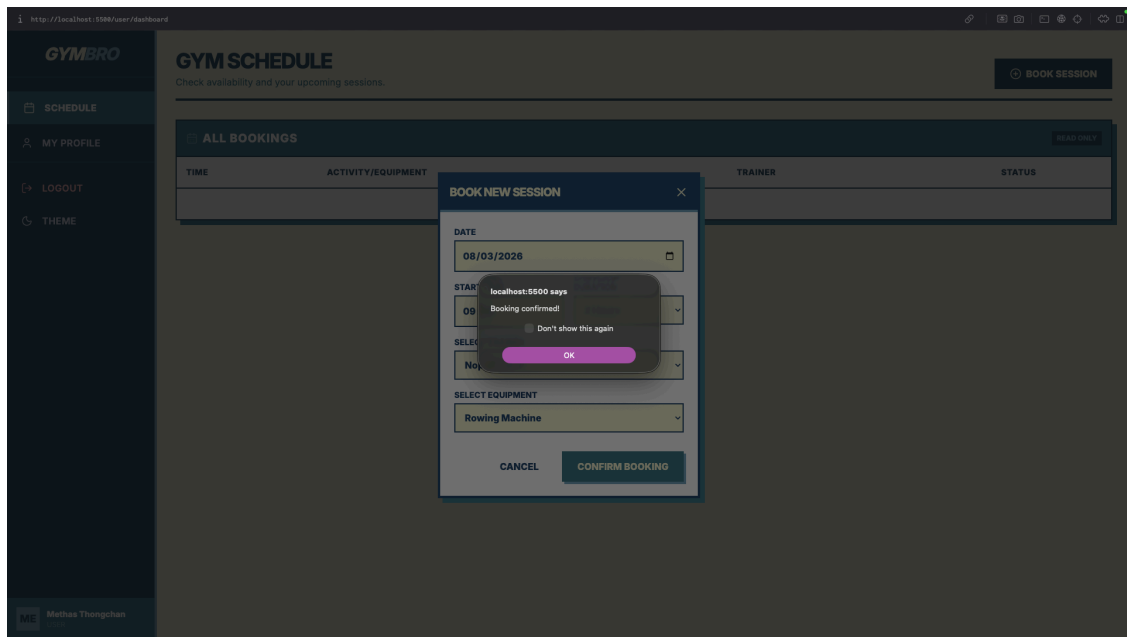


Figure 7.7: ข้อความแจ้งเตือนยืนยันการจองสำเร็จ (Booking Alert) ที่มุมขวาบนของหน้าจอ

3. แสดงประเภทสมาชิก (Member Type) และวันหมดอายุสมาชิก
4. แสดงสถานะบัญชี (Account Status)

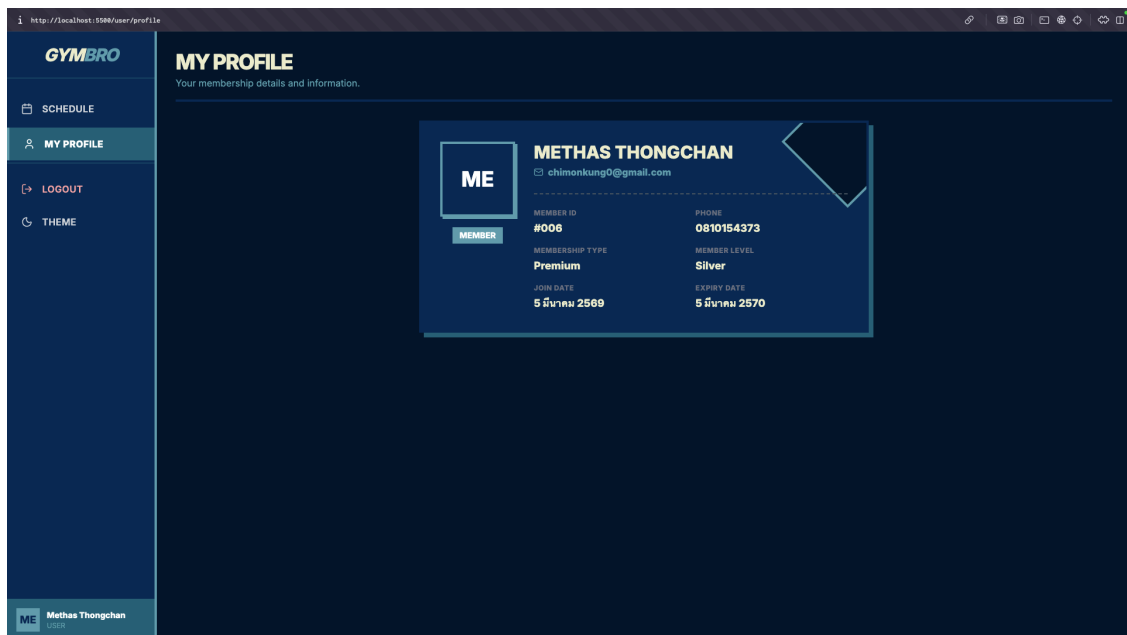


Figure 7.8: หน้าข้อมูลส่วนตัว (My Profile) ในโหมดมืด แสดงรายละเอียดสมาชิกและสถานะบัญชี

7.3 ส่วนผู้ดูแลระบบ (Admin Section)

ส่วนนี้สำหรับผู้ใช้งานที่ล็อกอินด้วยสิทธิ์ผู้ดูแลระบบ (Admin)

7.3.1 การเข้าสู่ระบบสำหรับผู้ดูแลระบบ (Admin Login)

ผู้ดูแลระบบสามารถเข้าสู่ระบบผ่านหน้า Login เดียวกับผู้ใช้งานทั่วไป หรือผ่าน URL เฉพาะ

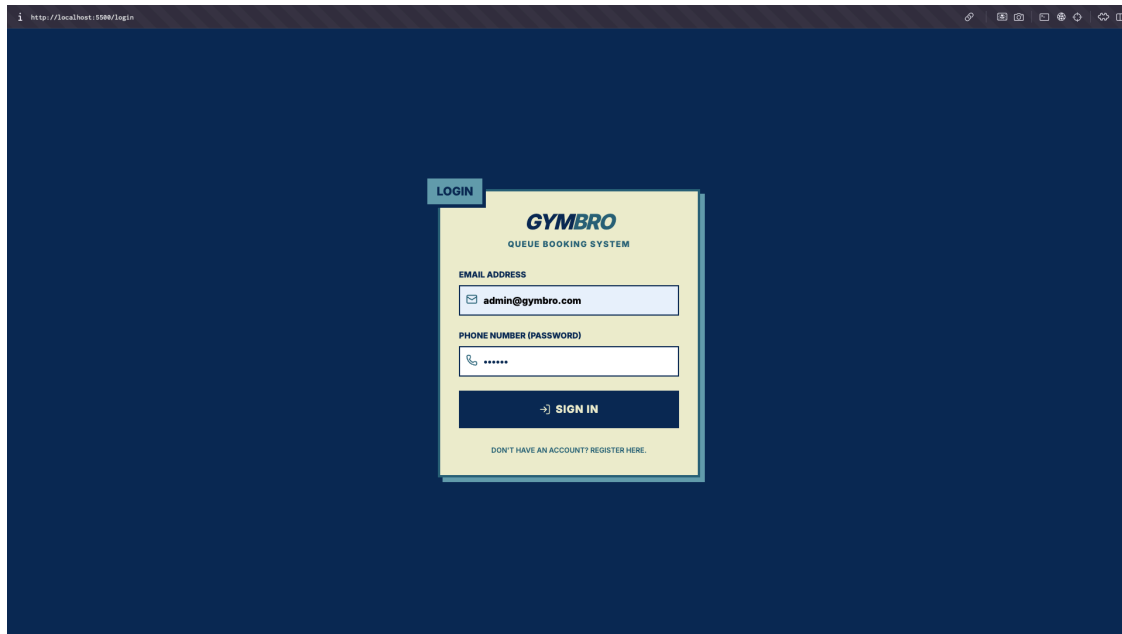


Figure 7.9: หน้าจอเข้าสู่ระบบสำหรับผู้ดูแลระบบ (Admin Login) ใช้แบบฟอร์มเดียวกับผู้ใช้งานทั่วไป

7.3.2 การใช้งานหน้าแดชบอร์ดผู้ดูแลระบบ (Admin Dashboard)

หน้าสรุปภาพรวมของระบบ แสดงสถิติและตารางงานวันนี้

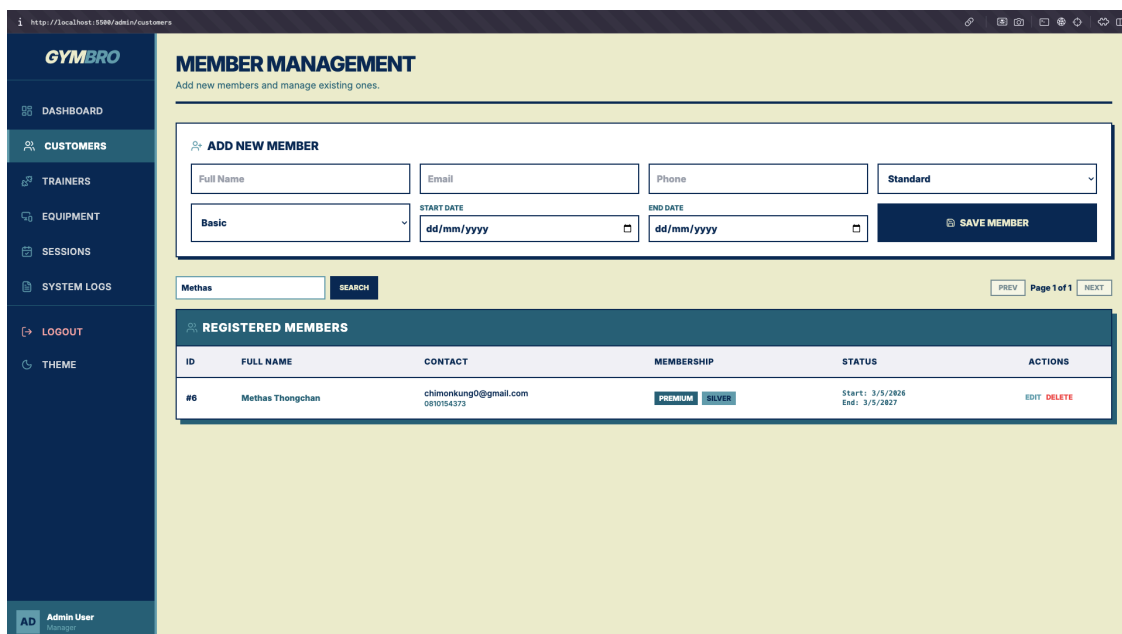


Figure 7.10: หน้าแดชบอร์ดผู้ดูแลระบบ (Admin Dashboard) แสดงจำนวนลูกค้า เทรนเนอร์ อุปกรณ์ และเซชันวันนี้

7.3.3 การจัดการข้อมูลลูกค้า (Customer Management)

เมนู **Customers** สำหรับบริหารจัดการข้อมูลสมาชิก ตรวจสอบสถานะ และอนุมัติสมาชิกใหม่

ID	FULL NAME	CONTACT	MEMBERSHIP	STATUS	ACTIONS
#1	mafia	nono@gmail.com 099	STANDARD BASIC	Start: 2/19/2026 End: -	EDIT DELETE
#6	Methas Thongchan	chimonkung@gmail.com 081054329	PREMIUM SILVER	Start: 3/5/2026 End: 3/5/2027	EDIT DELETE
#7	Tianhom Mayu	ohn@gmail.com	STANDARD BASIC	Start: 3/5/2026 End: 3/31/2026	EDIT DELETE
#8	Mint	mint@gmail.com 090	STANDARD BASIC	Start: 3/5/2026 End: 4/26/2026	EDIT DELETE
#9	doe hee	doe@gmail.com 123	MEMBER BEGINNER	Start: 3/5/2026 End: 3/5/2027	EDIT DELETE
#10	nn	nn@gmail.com nn123	MEMBER BEGINNER	Start: 3/6/2026 End: 3/6/2027	EDIT DELETE
...	...	korn@gmail.com	...	Start: 3/7/2026	...

Figure 7.11: หน้าจัดการข้อมูลลูกค้า (Customer Management) แสดงรายชื่อสมาชิกและปุ่มสำหรับการจัดการข้อมูล

7.3.4 การจัดการข้อมูลเทรนเนอร์ (Trainer Management)

เมนู **Trainers** สำหรับบริหารจัดการข้อมูลผู้ฝึกสอน ระบุความเชี่ยวชาญ

ID	NAME	LEVEL	SPECIALTY	PHONE	ACTIONS
#1	Shinnamon	Yoga	SENIOR	000	EDIT DELETE
#2	Nop	Cardio	JUNIOR	000	EDIT DELETE
#3	Korn	Weight training	MASTER	000	EDIT DELETE

Figure 7.12: หน้าจัดการข้อมูลเทรนเนอร์ (Trainer Management) แสดงรายชื่อและความเชี่ยวชาญของเทรนเนอร์

7.3.5 การจัดการอุปกรณ์ (Equipment Management)

เมนู **Equipment** สำหรับบริหารจัดการอุปกรณ์และกำหนดสถานะการใช้งาน

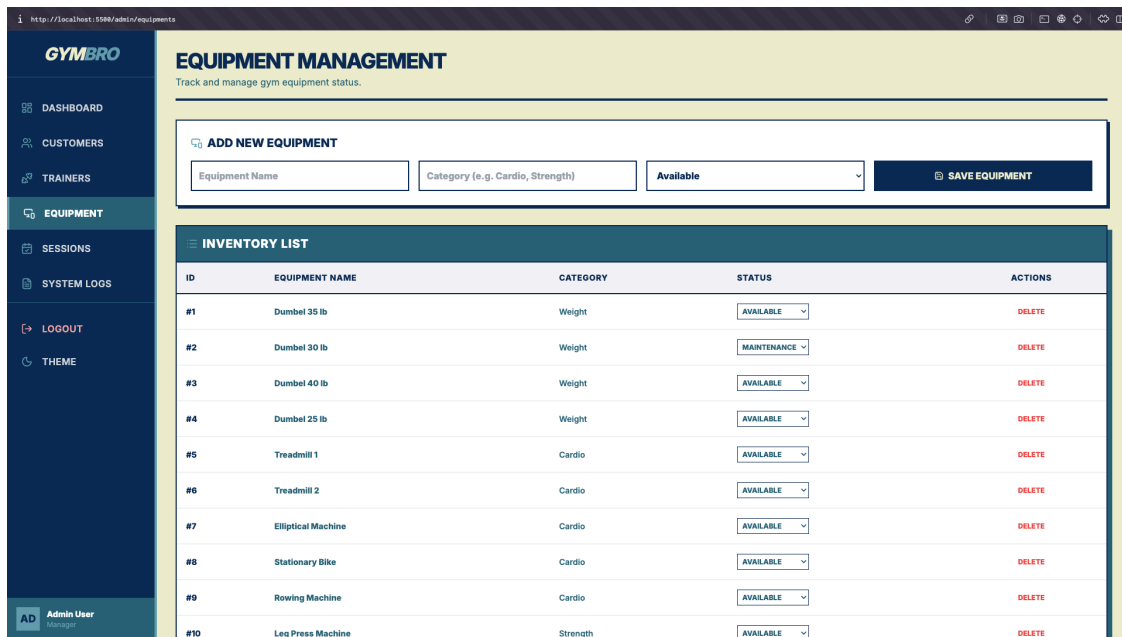


Figure 7.13: หน้าจัดการอุปกรณ์ (Equipment Management) แสดงรายการอุปกรณ์และสถานะความพร้อมใช้งาน

7.3.6 การจัดการตารางฝึกซ้อม (Session Management)

เมนู Sessions สำหรับดูและจัดการการจองทั้งหมด

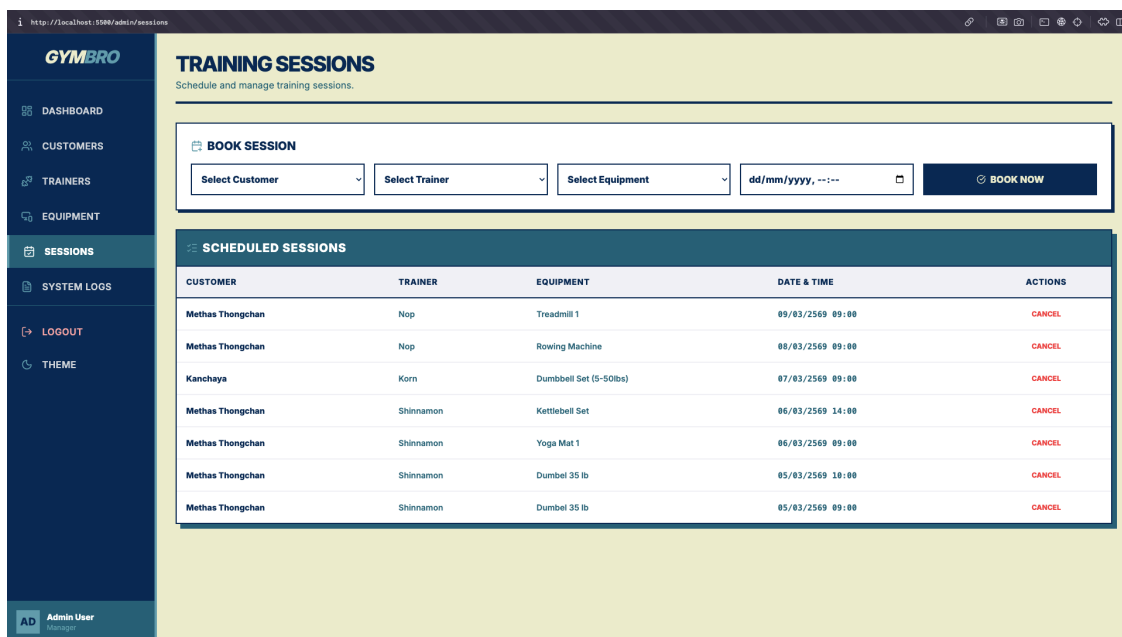


Figure 7.14: หน้าจัดการตารางฝึกซ้อม (Session Management) แสดงรายการจองทั้งหมดและฟังก์ชันการค้นหา

7.3.7 การดูบันทึกระบบ (System Logs)

เมนู System Logs สำหรับตรวจสอบประวัติการใช้งาน

7.3.8 การออกรายงาน (Reports)

ระบบมีฟังก์ชันสำหรับออกรายงานสรุปข้อมูลต่างๆ

The screenshot shows the 'REPORTS & SYSTEM LOGS' page with the 'SYSTEM LOGS' tab selected. The 'RECENT ACTIVITY' table lists various system actions with timestamps, action names, and details.

TIMESTAMP	ACTION	DETAILS
09/03/2569 09:52:06	CREATE SESSION	Created session ID: 7 (Customer: 6, Trainer: 2)
09/03/2569 09:51:19	DELETE SESSION	Deleted session ID: 7
09/03/2569 09:51:00	CREATE SESSION	Created session ID: 7 (Customer: 6, Trainer: 1)
09/03/2569 09:50:39	CREATE SESSION	Created session ID: 6 (Customer: 6, Trainer: 2)
07/03/2569 22:43:50	AUTO DELETE EXPIRED	Deleted user ID: 12 (Kanchaya)
07/03/2569 22:43:39	CREATE CUSTOMER	Created customer: Kanchaya (ID: 12)
07/03/2569 21:57:38	CREATE SESSION	Created session ID: 5 (Customer: 11, Trainer: 3)
07/03/2569 21:56:51	REGISTER USER	New user registered: Kanchaya (korn@gmail.com)
06/03/2569 11:51:36	DELETE SESSION	Deleted session ID: 4
06/03/2569 11:51:20	CREATE SESSION	Created session ID: 5 (Customer: 6, Trainer: 1)
06/03/2569	CREATE SESSION	Created session ID: 4 (Customer: 6, Trainer: 1)

Figure 7.15: หน้าบันทึกระบบ (System Logs) แสดงประวัติการกระทำต่างๆ ของผู้ใช้ในระบบ

รายงานข้อมูลลูกค้าและเทรนเนอร์

The screenshot shows the 'REPORTS & SYSTEM LOGS' page with the 'CUSTOMER REPORT' tab selected. The 'CUSTOMER USAGE HISTORY' table displays details for individual customers, including their name, equipment used, trainer, time range, and duration.

CUSTOMER NAME	EQUIPMENT USED	TRAINER	TIME RANGE	DURATION
Methas Thongchan	Treadmill 1	Nop	09/03/2569 09:00 - 10:00	1 hr 0 min
Methas Thongchan	Rowing Machine	Nop	08/03/2569 09:00 - 11:00	2 hr 0 min
Kanchaya	Dumbbell Set (5-50lbs)	Korn	07/03/2569 09:00 - 10:00	1 hr 0 min
Methas Thongchan	Kettlebell Set	Shinnamon	06/03/2569 14:00 - 15:30	1 hr 30 min
Methas Thongchan	Yoga Mat 1	Shinnamon	06/03/2569 09:00 - 10:00	1 hr 0 min
Methas Thongchan	Dumbel 35 lb	Shinnamon	05/03/2569 10:00 - 11:00	1 hr 0 min
Methas Thongchan	Dumbel 35 lb	Shinnamon	05/03/2569 09:00 - 10:00	1 hr 0 min

Figure 7.16: รายงานข้อมูลลูกค้า (Customer Report) แสดงในรูปแบบตารางพร้อมสำหรับพิมพ์

การพิมพ์รายงาน (Printing Report)

GYM BRO

REPORTS & SYSTEM LOGS
View usage reports and system activity.

SYSTEM LOGS CUSTOMER REPORT **TRAINER REPORT**

FILTER DATE: dd/mm/yyyy **APPLY** **CLEAR**

TRAINER SESSION HISTORY **REFRESH**

TRAINER NAME	CUSTOMER	EQUIPMENT USED	TIME RANGE	DURATION
Nop	Methas Thongchan	Treadmill 1	06/03/2569 09:00 - 10:00	1 hr 0 min
Nop	Methas Thongchan	Rowing Machine	06/03/2569 09:00 - 11:00	2 hr 0 min
Korn	Kanchaya	Dumbbell Set (5-50lbs)	07/03/2569 09:00 - 10:00	1 hr 0 min
Shinnamon	Methas Thongchan	Kettlebell Set	06/03/2569 14:00 - 15:30	1 hr 30 min
Shinnamon	Methas Thongchan	Yoga Mat 1	06/03/2569 09:00 - 10:00	1 hr 0 min
Shinnamon	Methas Thongchan	Dumbel 35 lb	05/03/2569 10:00 - 11:00	1 hr 0 min
Shinnamon	Methas Thongchan	Dumbel 35 lb	05/03/2569 09:00 - 10:00	1 hr 0 min

Figure 7.17: รายงานข้อมูลเทรนเนอร์ (Trainer Report) แสดงรายชื่อและความถี่ของข้อมูล

GYM BRO

REPORTS & SYSTEM LOGS
View usage reports and system activity.

FILTER DATE: dd/mm/yyyy **APPLY** **CLEAR**

RECENT ACTIVITY

TIMESTAMP	ACTION	DETAILS
06/03/2569 03:52:06	CREATE SESSION	Created session ID: 7 (Customer: 6, Trainer: 2)
06/03/2569 03:51:19	DELETE SESSION	Deleted session ID: 7
06/03/2569 03:51:00	CREATE SESSION	Created session ID: 7 (Customer: 6, Trainer: 1)
06/03/2569 03:50:53	CREATE SESSION	Created session ID: 6 (Customer: 6, Trainer: 2)
07/03/2569 22:43:50	AUTO DELETE EXPIRED	Deleted user ID: 12 (Kanchaya)
07/03/2569 22:43:33	CREATE CUSTOMER	Created customer: Kanchaya (ID: 12)
07/03/2569 21:17:38	CREATE SESSION	Created session ID: 5 (Customer: 11, Trainer: 3)
07/03/2569 21:16:51	REGISTER USER	New user registered: Kanchaya (korn@gmail.com)
06/03/2569 11:51:06	DELETE SESSION	Deleted session ID: 4

Print 4 sheets of paper

Destination: Brother DCP-TS20W

Pages: All

Copies: 1

Layout: Portrait

More settings

Cancel **Print**

Figure 7.18: หน้าตัวอย่างก่อนพิมพ์รายงาน (Print Preview) จัดรูปแบบหน้ากระดาษอัตโนมัติ

7.4 การแสดงผลในโหมดมืด (Dark Mode Interface)

ระบบรองรับการแสดงผลในโหมดมืด (Dark Mode) เพื่อถนอมสายตาและประหยัดพลังงาน โดยมีการปรับเปลี่ยนโทนสีของพื้นหลัง ตัวอักษร และองค์ประกอบต่างๆ ให้เหมาะสมกับการใช้งานในที่ที่มีแสงน้อย

7.4.1 หน้าจอผู้ดูแลระบบในโหมดมืด

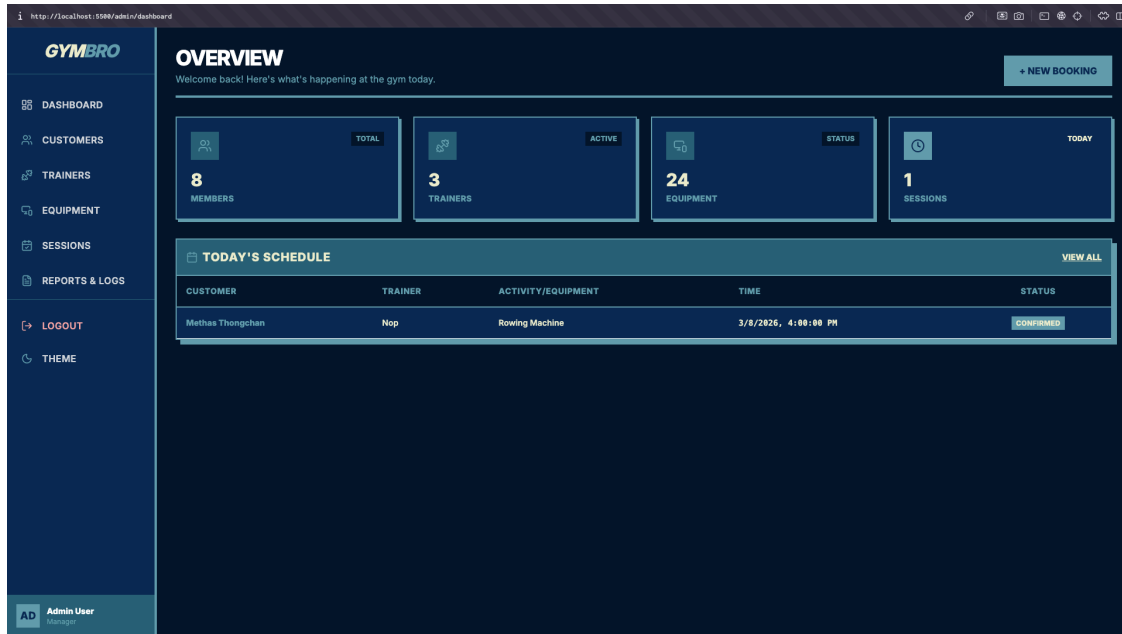


Figure 7.19: หน้าแดชบอร์ดผู้ดูแลระบบในโหมดมืด (Dark Admin Dashboard) ใช้โทนสีน้ำเงินเข้มและตัวอักษรสีอ่อน

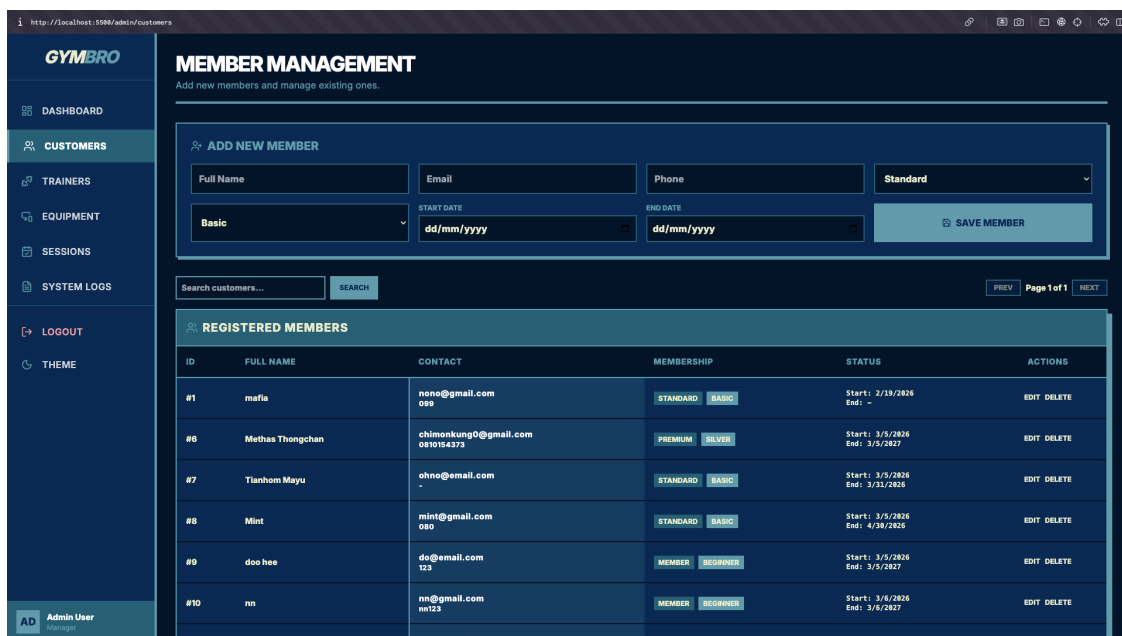


Figure 7.20: หน้าจัดการข้อมูลลูกค้าในโหมดมืด (Dark Customer Management)

7.4.2 หน้ารายงานในโหมดมืด

7.4.3 หน้าจอสมาชิกในโหมดมืด

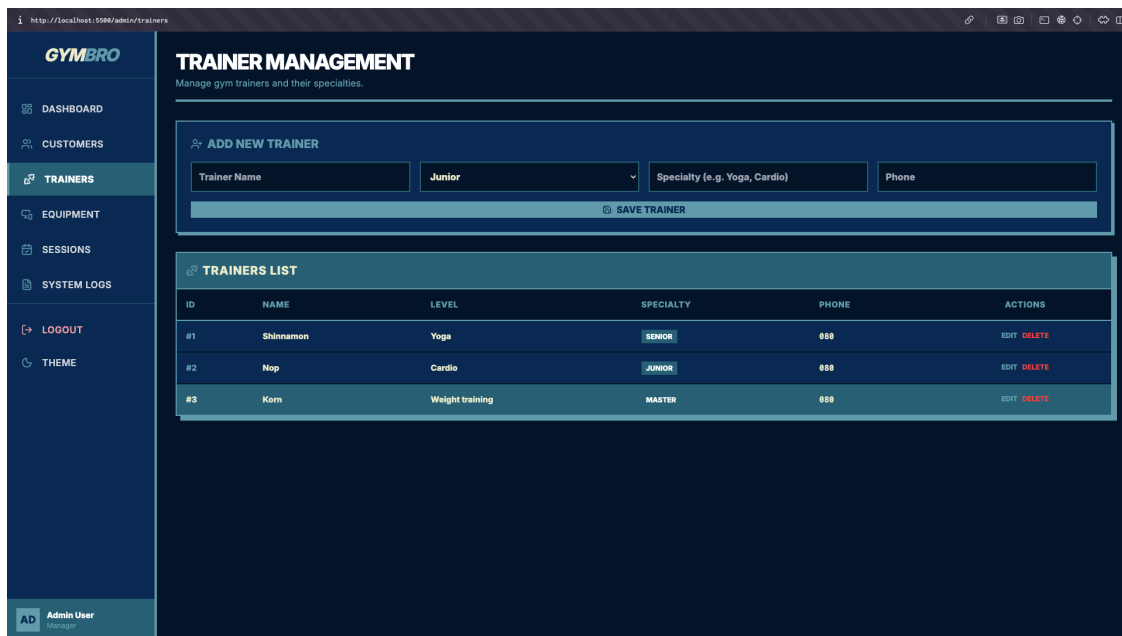


Figure 7.21: หน้าจัดการข้อมูลเทรนเนอร์ในโหมดมืด (Dark Trainer Management)

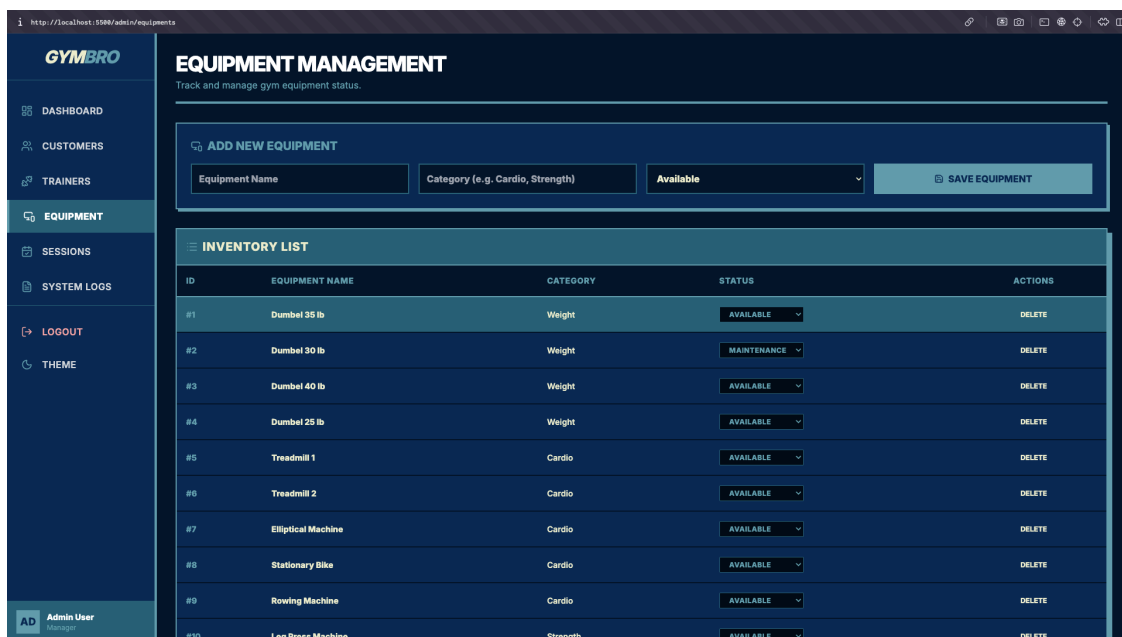


Figure 7.22: หน้าจัดการอุปกรณ์ในโหมดมืด (Dark Equipment Management)

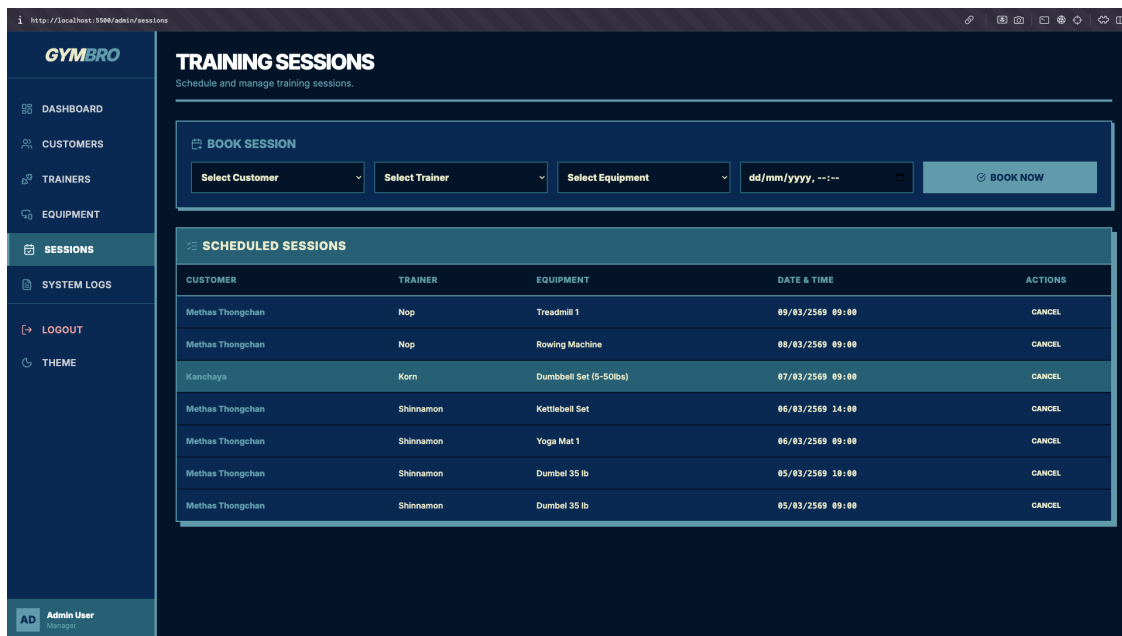


Figure 7.23: หน้าจัดการตารางฝึกซ้อมในโหมดมืด (Dark Session Management)

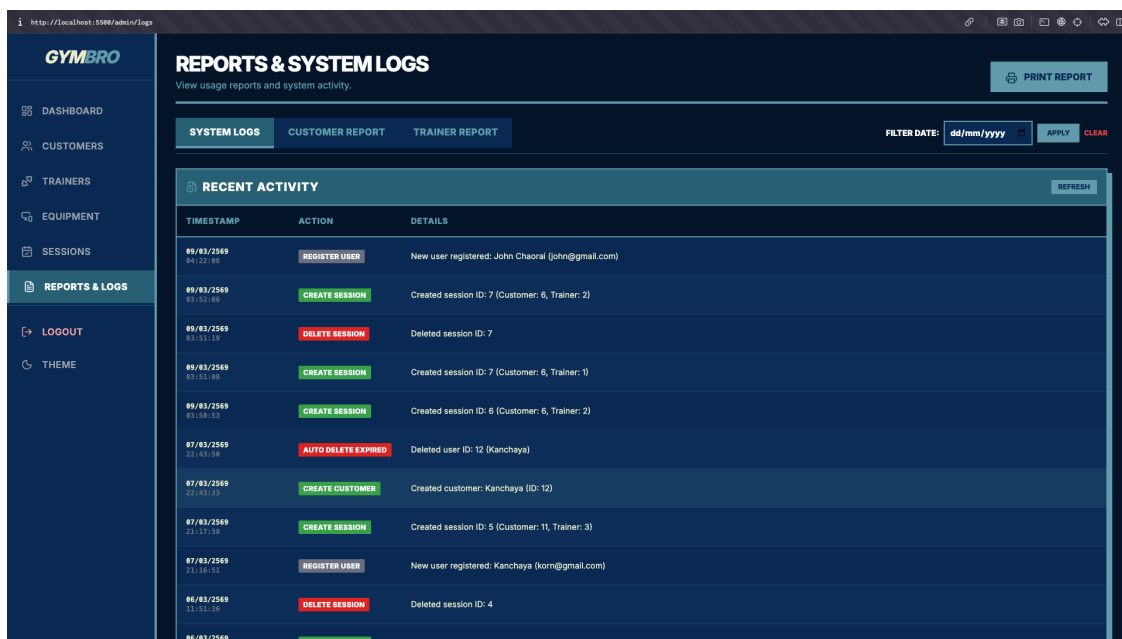


Figure 7.24: หน้าบันทึกในระบบในโหมดมืด (Dark System Logs)

CUSTOMER NAME	EQUIPMENT USED	TRAINER	TIME RANGE	DURATION
Methas Thongchan	Treadmill 1	Nop	06/03/2569 09:00 ~ 10:00	1 hr 0 min
Methas Thongchan	Rowing Machine	Nop	06/03/2569 09:00 ~ 11:00	2 hr 0 min
Kanchaya	Dumbbell Set (5-50lbs)	Korn	07/03/2569 09:00 ~ 10:00	1 hr 0 min
Methas Thongchan	Kettlebell Set	Shinnamon	06/03/2569 14:00 ~ 15:30	1 hr 30 min
Methas Thongchan	Yoga Mat 1	Shinnamon	06/03/2569 09:00 ~ 10:00	1 hr 0 min
Methas Thongchan	Dumbel 35 lb	Shinnamon	05/03/2569 10:00 ~ 11:00	1 hr 0 min
Methas Thongchan	Dumbel 35 lb	Shinnamon	05/03/2569 09:00 ~ 10:00	1 hr 0 min

Figure 7.25: รายงานข้อมูลลูกค้าในโหนดมืด (Dark Customer Report)

TRAINER NAME	CUSTOMER	EQUIPMENT USED	TIME RANGE	DURATION
Nop	Methas Thongchan	Treadmill 1	06/03/2569 09:00 ~ 10:00	1 hr 0 min
Nop	Methas Thongchan	Rowing Machine	06/03/2569 09:00 ~ 11:00	2 hr 0 min
Korn	Kanchaya	Dumbbell Set (5-50lbs)	07/03/2569 09:00 ~ 10:00	1 hr 0 min
Shinnamon	Methas Thongchan	Kettlebell Set	06/03/2569 14:00 ~ 15:30	1 hr 30 min
Shinnamon	Methas Thongchan	Yoga Mat 1	06/03/2569 09:00 ~ 10:00	1 hr 0 min
Shinnamon	Methas Thongchan	Dumbel 35 lb	05/03/2569 10:00 ~ 11:00	1 hr 0 min
Shinnamon	Methas Thongchan	Dumbel 35 lb	05/03/2569 09:00 ~ 10:00	1 hr 0 min

Figure 7.26: รายงานข้อมูลเทรนเนอร์ในโหนดมืด (Dark Trainer Report)

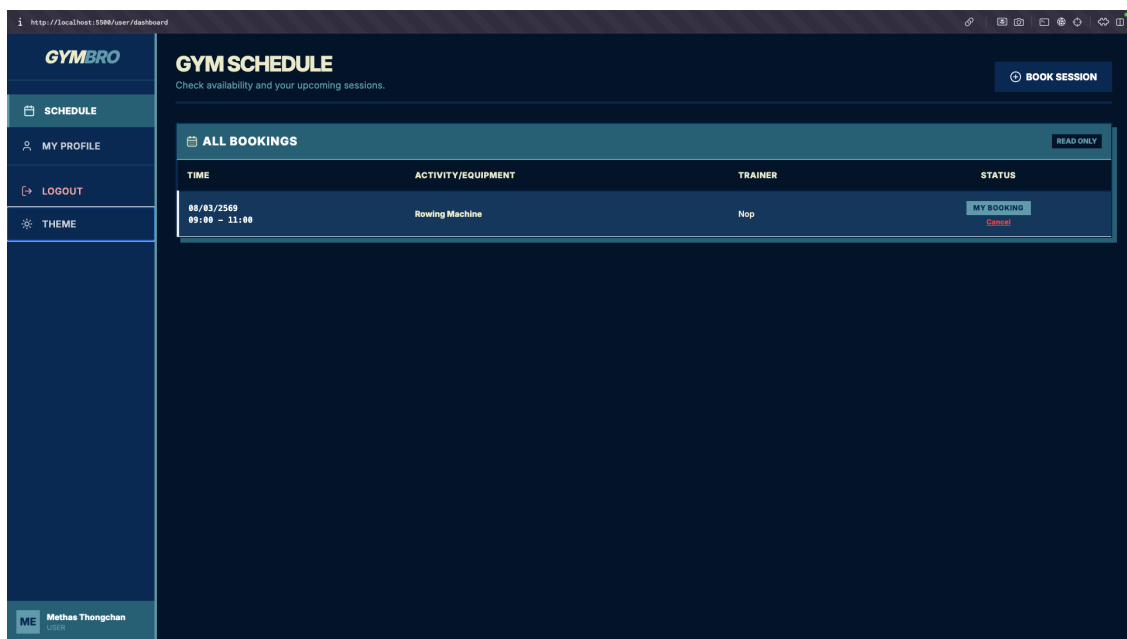


Figure 7.27: หน้าตารางฝึกซ้อมของสมาชิกในโหมดมืด (Dark User Schedule)

บทที่ 8

อธิบายโค้ด

8.1 Backend (ระบบหลังบ้าน)

8.1.1 server.js

ภาพรวม (Overview)

ไฟล์ `server.js` ทำหน้าที่เป็นจุดเริ่มต้นหลัก (Entry Point) สำหรับฝั่ง Backend ของระบบ GymBro โดยทำหน้าที่เป็น Web Server ผ่านการใช้งาน Express.js Framework ไฟล์นี้มีความรับผิดชอบหลักในการจัดการ API Requests ทั้งหมดที่ส่งมาจาก Frontend, การเชื่อมต่อกับฐานข้อมูลผ่าน Sequelize ORM, การจัดการตรรกะทางธุรกิจ (Business Logic), การบันทึกประวัติการทำงาน (Logging), และการตั้งค่าภารกิจตามกำหนดเวลา (Scheduled Task) สำหรับการลบข้อมูลสมาชิกที่หมดอายุโดยอัตโนมัติ

การนำเข้าไลบรารีและการตั้งค่าเริ่มต้น (Dependencies and Setup)

ส่วนนี้เป็นการนำเข้าไลบรารีที่จำเป็นและการกำหนดค่าเริ่มต้นสำหรับแอปพลิเคชัน

```
1 require("dotenv").config();
2 const express = require("express");
3 const cors = require("cors");
4 const { Sequelize, DataTypes } = require("sequelize");
5 const sequelize = require("./db");
6 const { Customers, Trainers, GymEquipments, TrainingSessions } = sequelize;
7 const fs = require('fs');
8 const fsPromises = require('fs').promises;
9 const path = require('path');
10
11 const LOGS_FILE = path.join(__dirname, 'logs.json');
```

Listing 8.1: ซอร์สโค้ดจากไฟล์ `server.js`

คำอธิบาย:

- `require("dotenv").config()`: โหลดค่าตัวแปรสภาพแวดล้อมจากไฟล์ `.env` เพื่อความปลอดภัยและความยืดหยุ่นในการตั้งค่า
- `express`: เรียกใช้ Framework หลักสำหรับการสร้าง Web Server
- `cors`: ใช้ Middleware สำหรับจัดการ Cross-Origin Resource Sharing เพื่ออนุญาตให้ Frontend สามารถเรียกใช้งาน API ข้ามโดเมนได้
- `sequelize`: นำเข้า Instance การเชื่อมต่อฐานข้อมูลและ Models ต่างๆ (Customers, Trainers, etc.) จากไฟล์ `db.js`
- `fs` และ `fsPromises`: ใช้งานโมดูลสำหรับการจัดการระบบไฟล์ (File System) เพื่ออ่านและเขียนไฟล์ Logs
- `LOGS_FILE`: กำหนดเส้นทาง (Path) ของไฟล์ `logs.json` ที่จะใช้สำหรับเก็บประวัติการทำงานของระบบ

ฟังก์ชันช่วยสำหรับการบันทึกข้อมูล (Helper Function: logAction)

ฟังก์ชันนี้มีหน้าที่บันทึกการกระทำต่างๆ ที่เกิดขึ้นในระบบลงในไฟล์ JSON เพื่อใช้ในการตรวจสอบย้อนหลัง

```
1 async function logAction(action, details) {
2   try {
3     const timestamp = new Date().toISOString();
4     const logEntry = { timestamp, action, details };
5
6     let logs = [];
7     try {
8       const data = await fsPromises.readFile(LOGS_FILE, 'utf8');
9       logs = JSON.parse(data || '[]');
10    } catch (readErr) {
11      logs = [];
12    }
13
14    logs.unshift(logEntry);
15    if (logs.length > 1000) logs = logs.slice(0, 1000);
16
17    await fsPromises.writeFile(LOGS_FILE, JSON.stringify(logs, null, 2));
18  } catch (err) {
19    console.error('Failed to write log:', err);
20  }
21 }
```

Listing 8.2: ซอร์สโค้ดจากไฟล์ server.js

คำอธิบาย:

1. พารามิเตอร์: รับค่า action (ชื่อการกระทำ) และ details (รายละเอียดของการกระทำ)
2. สร้าง Object logEntry ที่ประกอบด้วยเวลาปัจจุบัน (Timestamp), ชื่อการกระทำ, และรายละเอียด
3. อ่านข้อมูลจากไฟล์ logs.json เดิม หากไม่พบไฟล์จะเริ่มต้นด้วย Array ว่าง []
4. logs.unshift(logEntry): เพิ่มรายการ Log ใหม่ไปยังตำแหน่งแรกของ Array
5. จำกัดจำนวน Log ไม่ให้เกิน 1,000 รายการ เพื่อป้องกันไม่ให้ขนาดไฟล์ใหญ่เกินความจำเป็น
6. บันทึกข้อมูลกลับลงสู่ไฟล์ logs.json

ฟังก์ชันช่วยสำหรับการจัดลำดับ ID ใหม่ (Helper Function: reorderIds)

ฟังก์ชันนี้ใช้สำหรับเรียงลำดับ ID ในฐานข้อมูลใหม่ (Reset Auto Increment) เพื่อให้ ID เรียงต่อกันอย่างสวยงามหลังจากมีการลบข้อมูลออกจากระบบ

```
1 async function reorderIds(model) {
2   try {
3     const items = await model.findAll({ order: [['id', 'ASC']] });
4     for (let i = 0; i < items.length; i++) {
5       const item = items[i];
6       const newId = i + 1;
7       if (item.id !== newId) {
8         const oldId = item.id;
9
10        // Manual Cascade Update for TrainingSessions references
11        if (model === Customers) {
12          await TrainingSessions.update({ customerId: newId }, { where: { customerId: oldId } });
13        } else if (model === Trainers) {
14          await TrainingSessions.update({ trainerId: newId }, { where: { trainerId: oldId } });
15        } else if (model === GymEquipments) {
16          await TrainingSessions.update({ equipmentId: newId }, { where: { equipmentId: oldId }
17        }
18
19        // Update the item's ID using raw query
20        const tableName = model.tableName;
21        await sequelize.query(`UPDATE "${tableName}" SET id = ${newId} WHERE id = ${oldId}`);
22      }
23    }
24  }
25 }
```

```

23 }
24
25 // Reset sequence
26 const tableName = model.tableName;
27 const seqName = `${tableName}_id_seq`;
28
29 if (items.length > 0) {
30   await sequelize.query(`SELECT setval('${seqName}', ${items.length})`);
31 } else {
32   await sequelize.query(`SELECT setval('${seqName}', 1, false)`);
33 }
34
35 } catch (err) {
36   console.error('Failed to reorder IDs:', err);
37 }
38 }

```

Listing 8.3: ซอร์สโค้ดจากไฟล์ server.js

คำอธิบาย:

1. **หลักการทำงาน:** ดึงข้อมูลทั้งหมดจากตารางมาเรียงตาม ID จากน้อยไปมาก แล้ววนลูปตรวจสอบว่า ID ของแต่ละรายการตรงกับลำดับที่ควรจะเป็น (index + 1) หรือไม่
2. **Cascade Update:** เนื่องจากการอ้างอิง Foreign Key ในตาราง TrainingSessions หากมีการเปลี่ยน ID ของ Customer, Trainer หรือ Equipment จำเป็นต้องอัปเดต ID ในตาราง Sessions ด้วยเพื่อรักษาความถูกต้องของความสัมพันธ์ข้อมูล
3. **Raw Query:** ใช้คำสั่ง SQL โดยตรงในการอัปเดต ID หลัก (UPDATE ... SET id = ...)
4. **Reset Sequence:** ปรับค่า Auto Increment Sequence ของ PostgreSQL (setval) ให้สอดคล้องกับจำนวนข้อมูลปัจจุบัน เพื่อให้การเพิ่มข้อมูลครั้งต่อไปได้รับ ID ที่ถูกต้อง

การกำหนดค่า Server และ API สำหรับการเข้าสู่ระบบ (Server Configuration & Login API)

ส่วนนี้เป็นการตั้งค่า Express App และสร้าง API Endpoint สำหรับการยืนยันตัวตน (Authentication)

```

1  const app = express();
2  app.use(
3    cors({
4      origin: "http://localhost:5500",
5      methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
6      allowedHeaders: ["Content-Type"],
7    })
8  );
9  app.options("*", cors({ origin: "http://localhost:5500" }));
10 app.use(express.json());
11
12 app.post("/api/login", async (req, res) => {
13   const { email, phone } = req.body;
14
15   // Hardcoded Admin Check
16   if ((email === "admin@gymbro.com" && phone === "123456")) {
17     return res.json({
18       role: "admin",
19       user: { /* ...Admin Details... */ }
20     });
21   }
22
23   try {
24     const user = await Customers.findOne({ where: { email, phone } });
25     if (!user) {
26       return res.status(401).json({ error: "Invalid credentials" });
27     }
28     res.json({
29       role: "user",
30       user: user
31     });
32   } catch (e) {
33     res.status(500).json({ error: e.message });
34   }
35 }

```

```
34 }  
35 });
```

Listing 8.4: ซอร์สโค้ดจากไฟล์ server.js

คำอธิบาย:

1. cors: อนุญาตให้ Frontend ที่รันอยู่บน localhost:5500 สามารถเรียกใช้งาน API ได้
2. express.json(): ใช้งาน Middleware สำหรับแปลงข้อมูล Request Body ที่ส่งมาในรูปแบบ JSON
3. POST /api/login:
 - (a) ตรวจสอบเงื่อนไข Admin แบบ Hardcoded: หากใช้อีเมล admin@gymbro.com และเบอร์โทร 123456 จะได้รับสิทธิ์เป็น "admin"
 - (b) หากไม่ใช่ Admin จะทำการค้นหาข้อมูลในตาราง Customers โดยใช้อีเมลและเบอร์โทรศัพท์
 - (c) หากพบข้อมูล จะส่งคืนสิทธิ์ "user" พร้อมข้อมูลลูกค้ากลับไป

การเชื่อมต่อฐานข้อมูลและการตรวจสอบสถานะ (Database Sync & Health Check)

```
1 sequelize.sync();  
2  
3 app.get("/api/health", async (req, res) => {  
4   try {  
5     await sequelize.authenticate();  
6     res.json({ ok: true });  
7   } catch (e) {  
8     res.status(500).json({ ok: false, error: e.message });  
9   }  
10 });
```

Listing 8.5: ซอร์สโค้ดจากไฟล์ server.js

คำอธิบาย:

1. sequelize.sync(): ดำเนินการสร้างตารางในฐานข้อมูลให้ตรงกับ Models ที่นิยามไว้ หากยังไม่มีตารางดังกล่าว
2. GET /api/health: Endpoint สำหรับตรวจสอบสถานะความพร้อมของ Server และการเชื่อมต่อกับฐานข้อมูล

API สำหรับการจัดการเซสชันการฝึกซ้อม (Training Sessions API)

API นี้มีความซับซ้อนที่สุด เนื่องจากต้องมีการตรวจสอบเงื่อนไขทางธุรกิจหลายประการก่อนการสร้างข้อมูล

```
1 app.post("/api/sessions", async (req, res) => {  
2   try {  
3     const { sessionDate, duration, trainerId, equipmentId, customerId } = req.body;  
4  
5     // Calculate endDate  
6     const start = new Date(sessionDate);  
7     let end;  
8     if (req.body.endDate) {  
9       end = new Date(req.body.endDate);  
10    } else {  
11      const dur = duration ? parseInt(duration) : 60;  
12      end = new Date(start.getTime() + dur * 60000);  
13    }  
14  
15    // Check equipment status  
16    if (equipmentId) {  
17      const equipment = await GymEquipments.findByPk(equipmentId);  
18      if (equipment && equipment.status === 'Maintenance') {  
19        return res.status(400).json({ error: "This equipment is currently in maintenance..." });  
20      }  
21    }  
22  
23    // Check availability (Trainer)  
24    if (trainerId) {
```

```

25     const trainerConflict = await TrainingSessions.findOne({
26       where: {
27         trainerId,
28         [Sequelize.Op.and]: [
29           { sessionId: { [Sequelize.Op.lt]: end } },
30           { endDate: { [Sequelize.Op.gt]: start } }
31         ]
32       }
33     });
34     if (trainerConflict) return res.status(400).json({ error: "Trainer is not available..."
35     ↪ });
36   }
37
38   // Check availability (Equipment)
39   const equipmentConflict = await TrainingSessions.findOne({
40     where: {
41       equipmentId,
42       [Sequelize.Op.and]: [
43         { sessionId: { [Sequelize.Op.lt]: end } },
44         { endDate: { [Sequelize.Op.gt]: start } }
45       ]
46     }
47   });
48   if (equipmentConflict) return res.status(400).json({ error: "Equipment is already booked..."
49   ↪ });
50
51   const created = await TrainingSessions.create({ /* ... */ });
52   logAction("Create Session", `Created session ID: ${created.id}...`);
53   res.status(201).json(created);
54 } catch (e) {
55   res.status(500).json({ error: e.message });
56 }
57 });

```

Listing 8.6: ซอร์สโค้ดจากไฟล์ server.js

คำอธิบาย:

1. การคำนวณเวลาสิ้นสุด: คำนวณค่า endDate โดยใช้เวลาเริ่มต้น (start) บวกกับระยะเวลา (duration) ในหน่วยนาที่
2. การตรวจสอบสถานะอุปกรณ์: ตรวจสอบว่าอุปกรณ์ที่เลือกมีสถานะเป็น 'Maintenance' (ซ่อมบำรุง) หรือไม่ หากใช่จะไม่อนุญาตให้ทำการจอง
3. การตรวจสอบเวลาทับซ้อน (Overlap Check): ใช้ตรรกะทางคณิตศาสตร์ ($StartA < EndB$) AND ($EndA > StartB$) เพื่อตรวจสอบว่าช่วงเวลาที่ต้องการจอง มีความซ้อนทับกับเซสชันที่มีอยู่เดิมหรือไม่
 - (a) ตรวจสอบการว่างของเทรนเนอร์ (trainerId)
 - (b) ตรวจสอบการว่างของอุปกรณ์ (equipmentId)
4. หากผ่านทุกเงื่อนไข ระบบจะทำการ create ข้อมูลลงในฐานข้อมูลและบันทึก Log การกระทำ

ระบบลบสมาชิกที่หมดอายุอัตโนมัติ (Auto Delete Expired Users)

ฟังก์ชันนี้ทำงานเป็น Background Task เพื่อตรวจสอบและลบข้อมูลสมาชิกที่หมดระยะเวลาสมาชิกแล้ว

```

1 async function checkExpiredMembers() {
2   try {
3     const now = new Date();
4     const expired = await Customers.findAll({
5       where: { memberEndDate: { [Sequelize.Op.lt]: now } }
6     });
7
8     if (expired.length > 0) {
9       console.log(`Checking Expired: Found ${expired.length} users. Deleting...`);
10      for (const user of expired) {
11        // Delete sessions first (manual cascade)
12        await TrainingSessions.destroy({ where: { customerId: user.id } });
13        await user.destroy();

```

```

14         logAction("Auto Delete Expired", `Deleted user ID: ${user.id}
    ↪   (${user.fullName})`);
15     }
16     await reorderIds(Customers);
17     console.log("Expired users cleanup complete.");
18   } else {
19     console.log("Checking Expired: No expired users found.");
20   }
21 } catch (err) { console.error("Error in checkExpiredMembers:", err); }
22 }
23
24 // Run on start and every 24 hours
25 checkExpiredMembers();
26 setInterval(checkExpiredMembers, 24 * 60 * 60 * 1000);

```

Listing 8.7: ฟังก์ชัน checkExpiredMembers ใน server.js

คำอธิบาย:

1. ค้นหาข้อมูลลูกค้าที่มี memberEndDate น้อยกว่าเวลาปัจจุบัน (now)
2. ดำเนินการวนลูปเพื่อลบข้อมูล TrainingSessions ที่เกี่ยวข้อง และข้อมูล Customers
3. บันทึก Log และแสดงข้อความผ่าน Console
4. เรียกใช้งานฟังก์ชัน reorderIds(Customers) เพื่อจัดลำดับ ID ใหม่
5. ตั้งค่า setInterval ให้ฟังก์ชันทำงานซ้ำทุกๆ 24 ชั่วโมง

8.2 ฟีเจอร์เพิ่มเติม (Additional Features)

8.2.1 การค้นหาและแบ่งหน้า (Search & Pagination) - ส่วน Backend

API Endpoint GET /api/customers ได้รับการปรับปรุงให้รองรับการค้นหา (Search) และการแบ่งหน้า (Pagination)

การทำงาน (Implementation)

ใช้ Op.like ของ Sequelize สำหรับการค้นหาแบบ Partial Match และใช้ limit, offset สำหรับการแบ่งหน้า

```

1 // Customers API
2 app.get("/api/customers", async (req, res) => {
3   try {
4     const { q, page = 1, limit = 10 } = req.query;
5     const offset = (parseInt(page) - 1) * parseInt(limit);
6     const where = {};
7
8     if (q) {
9       where[Op.or] = [
10        { fullName: { [Op.iLike]: `_${q}%` } },
11        { email: { [Op.iLike]: `_${q}%` } },
12        { phone: { [Op.iLike]: `_${q}%` } }
13      ];
14    }
15
16    const { count, rows } = await Customers.findAndCountAll({
17      where,
18      order: [["id", "ASC"]],
19      limit: parseInt(limit),
20      offset: parseInt(offset)
21    });
22
23    res.json({
24      data: rows,
25      total: count,
26      page: parseInt(page),
27      limit: parseInt(limit),
28      totalPages: Math.ceil(count / parseInt(limit))
29    });
30  } catch (e) {
31    res.status(500).json({ error: e.message });

```

```

32 }
33 });

```

Listing 8.8: Logic การค้นหาและแบ่งหน้าใน server.js

8.2.2 Validation Middleware

ระบบมีการตรวจสอบความถูกต้องของข้อมูล (Data Validation) ก่อนบันทึกลงฐานข้อมูล โดยใช้ไลบรารี express-validator

การทำงาน (Implementation)

สร้าง Middleware ชื่อ validate และกำหนดกฎการตรวจสอบ (Validation Rules) แยกตามแต่ละ Route ในไฟล์ middleware/validation.js

```

1 const { body, validationResult } = require('express-validator');
2
3 const validate = (req, res, next) => {
4   const errors = validationResult(req);
5   if (!errors.isEmpty()) {
6     return res.status(400).json({ errors: errors.array() });
7   }
8   next();
9 };
10
11 const customerValidationRules = () => {
12   return [
13     body('fullName').trim().notEmpty().withMessage('Full Name is required'),
14     body('email').trim().isEmail().withMessage('Invalid email format'),
15     // ...
16   ];
17 };
18
19 module.exports = {
20   validate,
21   customerValidationRules,
22   // ...
23 };

```

Listing 8.9: ไฟล์ backend/middleware/validation.js

8.2.3 การเชื่อมต่อฐานข้อมูล (Database Connection - db.js)

ไฟล์ db.js ทำหน้าที่กำหนดการเชื่อมต่อกับฐานข้อมูล PostgreSQL ผ่าน Sequelize ORM และกำหนด Schema ของตารางต่างๆ

การตั้งค่า Sequelize (Sequelize Instance)

```

1 const path = require("path");
2 // Try loading .env from current dir or parent dir (in case run from root)
3 require("dotenv").config({ path: path.resolve(__dirname, ".env") });
4
5 const { Sequelize, DataTypes } = require("sequelize");
6
7 // Prioritize environment variable first
8 const dbUrl =
9   process.env.DATABASE_URL ||
10
11   ↪ "postgres://webadmin:0FQzlb19621@node86180-env-2210254.proen.app.ruk-com.cloud:11857/gymbro_db";
12 console.log("Connecting to database:", dbUrl.replace(/:[^:]*@/, "****@")); // Log (masked) URL
13   ↪ for debugging
14
15 const sequelize = new Sequelize(dbUrl, {
16   dialect: "postgres",
17   logging: false,
18 });

```

Listing 8.10: การตั้งค่า Sequelize ใน db.js

การนิยาม Model (Model Definitions)

มีการกำหนด Model สำหรับตารางหลักทั้ง 4 ตาราง ได้แก่ Customers, Trainers, GymEquipments, และ TrainingSessions

```
1 const Customers = sequelize.define(  
2   "Customers",  
3   {  
4     id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },  
5     fullName: { type: DataTypes.STRING(150), allowNull: false },  
6     email: { type: DataTypes.STRING(150), allowNull: false },  
7     phone: { type: DataTypes.STRING(20) },  
8     memberType: { type: DataTypes.STRING(50), allowNull: false },  
9     memberLevel: { type: DataTypes.STRING(50), allowNull: false },  
10    memberStartDate: { type: DataTypes.DATE, allowNull: false },  
11    memberEndDate: { type: DataTypes.DATE },  
12  },  
13  { tableName: "Customers", timestamps: false, freezeTableName: true }  
14 );
```

Listing 8.11: ตัวอย่าง Model Definition: Customers

```
1 const TrainingSessions = sequelize.define(  
2   "TrainingSessions",  
3   {  
4     id: { type: DataTypes.INTEGER, autoIncrement: true, primaryKey: true },  
5     sessionDate: { type: DataTypes.DATE, allowNull: false },  
6     endDate: { type: DataTypes.DATE },  
7     customerId: { type: DataTypes.INTEGER, allowNull: false },  
8     trainerId: { type: DataTypes.INTEGER, allowNull: true },  
9     equipmentId: { type: DataTypes.INTEGER, allowNull: false },  
10  },  
11  { tableName: "TrainingSessions", timestamps: false, freezeTableName: true }  
12 );  
13  
14 TrainingSessions.belongsTo(Customers, { as: "customer", foreignKey: "customerId" });  
15 TrainingSessions.belongsTo(Trainers, { as: "trainer", foreignKey: "trainerId" });  
16 TrainingSessions.belongsTo(GymEquipments, { as: "equipment", foreignKey: "equipmentId" });  
17 Customers.hasMany(TrainingSessions, { as: "sessions", foreignKey: "customerId" });  
18 Trainers.hasMany(TrainingSessions, { as: "sessions", foreignKey: "trainerId" });  
19 GymEquipments.hasMany(TrainingSessions, { as: "sessions", foreignKey: "equipmentId" });
```

Listing 8.12: ตัวอย่าง Model Definition: TrainingSessions และ Relationships

8.2.4 API Endpoints (server.js)

ระบบมีการแยก API Endpoint ตามทรัพยากรข้อมูล เพื่อให้ง่ายต่อการจัดการตามหลักการ RESTful

Customers API

API สำหรับจัดการข้อมูลสมาชิก

```
1 app.post("/api/register", registerValidationRules(), validate, async (req, res) => {  
2   try {  
3     const { fullName, email, phone } = req.body;  
4  
5     // Check if email already exists  
6     const existing = await Customers.findOne({ where: { email } });  
7     if (existing) return res.status(400).json({ error: "Email already registered" });  
8  
9     // Fixed default values  
10    const memberType = "Member";  
11    const memberLevel = "Beginner";  
12    const memberStartDate = new Date();  
13    const memberEndDate = new Date();  
14    memberEndDate.setFullYear(memberEndDate.getFullYear() + 1); // Default 1 year
```

```

15
16     const created = await Customers.create({
17       fullName, email, phone,
18       memberType, memberLevel,
19       memberStartDate, memberEndDate
20     });
21
22     logAction("Register User", `New user registered: ${created.fullName} (${created.email})`);
23     res.status(201).json(created);
24   } catch (e) {
25     res.status(500).json({ error: e.message });
26   }
27 });

```

Listing 8.13: Customers API

Trainers API

API สำหรับจัดการข้อมูลเทรนเนอร์ รองรับการทำงานแบบ CRUD (Create, Read, Update, Delete)

```

1 // Get All Trainers
2 app.get("/api/trainers", async (req, res) => {
3   try {
4     const rows = await Trainers.findAll({ order: [["id", "ASC"]] });
5     res.json(rows);
6   } catch (e) {
7     res.status(500).json({ error: e.message });
8   }
9 });
10
11 // Create Trainer ( Validation)
12 app.post("/api/trainers", trainerValidationRules(), validate, async (req, res) => {
13   try {
14     const created = await Trainers.create(req.body);
15     logAction("Create Trainer", `Created trainer: ${created.trainerName}...`);
16     res.status(201).json(created);
17   } catch (e) {
18     res.status(500).json({ error: e.message });
19   }
20 });

```

Listing 8.14: Trainers API

Equipments API

API สำหรับจัดการข้อมูลอุปกรณ์ รองรับการทำงานแบบ CRUD

```

1 app.delete("/api/equipments/:id", async (req, res) => {
2   try {
3     const row = await GymEquipments.findByPk(req.params.id);
4     if (!row) return res.status(404).json({ error: "not_found" });
5     const name = row.equipmentName;
6     await row.destroy();
7     await reorderIds(GymEquipments);
8     logAction("Delete Equipment", `Deleted equipment: ${name} (ID: ${req.params.id})`);
9     res.json({ id: parseInt(req.params.id, 10) });
10  } catch (e) {
11    res.status(500).json({ error: e.message });
12  }
13 });

```

Listing 8.15: Equipments API (Delete)

Logs API

API สำหรับดึงข้อมูล Log การทำงานของระบบจากไฟล์ JSON

```

1 app.get("/api/logs", async (req, res) => {
2   try {
3     const data = await fsPromises.readFile(LOGS_FILE, 'utf8');
4     const logs = JSON.parse(data || '[]');

```



```

5   res.json(logs);
6 } catch (e) {
7   res.json([]);
8 }
9 });

```

Listing 8.16: Logs API

Dashboard Summary API

API พิเศษสำหรับรวบรวมสถิติเพื่อแสดงผลบน Dashboard ของผู้ดูแลระบบ โดยใช้ Promise.all เพื่อดึงข้อมูลจากหลายตารางพร้อมกันอย่างมีประสิทธิภาพ

```

1 app.get("/api/summary", async (req, res) => {
2   try {
3     const start = new Date();
4     start.setHours(0, 0, 0, 0); //
5     const end = new Date();
6     end.setHours(23, 59, 59, 999); //
7
8     const [c, t, e, s] = await Promise.all([
9       Customers.count(),
10      Trainers.count(),
11      GymEquipments.count(),
12      TrainingSessions.count({
13        where: {
14          sessionDate: { [Sequelize.Op.between]: [start, end] },
15        },
16      }),
17    ]);
18
19    res.json({
20      customers: c,
21      trainers: t,
22      equipments: e,
23      sessions_today: s,
24    });
25   } catch (e) {
26     res.status(500).json({ error: e.message });
27   }
28 });

```

Listing 8.17: Summary API

8.3 Frontend (ระบบหน้าบ้าน)

8.3.1 frontend/server.js

ภาพรวม (Overview)

ไฟล์ frontend/server.js ทำหน้าที่เป็น Web Server สำหรับฝั่ง Frontend ของระบบ GymBro โดยใช้ Express.js ในการให้บริการไฟล์ Static และจัดการเส้นทาง (Routing) ไปยังหน้าเว็บต่างๆ ที่เขียนด้วย EJS Template Engine ไฟล์นี้ทำหน้าที่แยกส่วนการทำงานของ Frontend ออกจาก Backend API อย่างชัดเจน โดย Frontend Server จะทำงานที่พอร์ต 5500

การตั้งค่า View Engine และ Static Files

ส่วนนี้เป็นการกำหนดค่าให้ Express ใช้งาน EJS เป็น View Engine และกำหนดโฟลเดอร์สำหรับไฟล์ Static

```

1 const express = require('express');
2 const path = require('path');
3 const app = express();
4 const port = 5500;
5
6 // Set view engine to EJS
7 app.set('view engine', 'ejs');
8 app.set('views', path.join(__dirname, 'views'));

```

```

9
10 // Serve static files
11 app.use(express.static(path.join(__dirname)));

```

Listing 8.18: การตั้งค่า View Engine และ Static Files ใน frontend/server.js

คำอธิบาย:

1. `app.set('view engine', 'ejs')`: กำหนดให้ใช้ EJS (Embedded JavaScript) ในการแสดงผลหน้าเว็บ
2. `app.set('views', ...)`: กำหนดตำแหน่งของไฟล์ Template (.ejs) ให้อยู่ในโฟลเดอร์ views
3. `app.use(express.static(...))`: ให้บริการไฟล์ Static (เช่น CSS, JavaScript, รูปภาพ) จาก Root Directory ของโปรเจกต์

การจัดการเส้นทาง (Routes Configuration)

ไฟล์นี้กำหนดเส้นทาง URL ต่างๆ เพื่อนำทางผู้ใช้งานไปยังหน้าเว็บที่เหมาะสม แบ่งออกเป็นเส้นทางหลัก, Admin Routes, และ User Routes

```

1 // Routes
2 app.get('/', (req, res) => {
3   res.redirect('/login');
4 });
5
6 app.get('/login', (req, res) => {
7   res.render('login');
8 });
9
10 app.get('/register', (req, res) => {
11   res.render('register');
12 });
13
14 // Admin Routes
15 app.get('/admin/dashboard', (req, res) => {
16   res.render('index');
17 });
18
19 app.get('/admin/customers', (req, res) => {
20   res.render('customers');
21 });
22
23 // ... (Other admin routes: trainers, equipments, sessions, logs)
24
25 // User Routes
26 app.get('/user/dashboard', (req, res) => {
27   res.render('user/dashboard');
28 });
29
30 app.get('/user/profile', (req, res) => {
31   res.render('user/profile');
32 });
33
34 // Fallback for old links (redirect to new structure)
35 app.get('/index.html', (req, res) => res.redirect('/admin/dashboard'));

```

Listing 8.19: การกำหนด Routes ใน frontend/server.js

คำอธิบาย:

1. **Redirection**: เส้นทางราก / จะถูก Redirect ไปยังหน้า /login เสมอ
2. **Rendering**: ใช้ `res.render()` เพื่อแสดงผลไฟล์ EJS ตามชื่อที่กำหนด (ไม่ต้องใส่นามสกุล .ejs)
3. **Route Groups**: แบ่งกลุ่ม URL ชัดเจนระหว่าง /admin/* และ /user/* เพื่อความเป็นระเบียบ
4. **Fallback**: มีการดักจับ URL แบบเก่า (.html) เพื่อ Redirect ไปยังเส้นทางใหม่ที่ถูกตั้ง ป้องกัน Broken Links

การเริ่มทำงานของ Server (Server Startup)

```
1 app.listen(port, () => {  
2   console.log(`Frontend server running on http://localhost:${port}`);  
3 });
```

Listing 8.20: การเริ่มทำงานของ Server ใน frontend/server.js

คำอธิบาย:

1. สั่งให้ Server เริ่มทำงานและรอรับ Request ที่พอร์ต 5500
2. แสดงข้อความใน Console เมื่อ Server เริ่มทำงานสำเร็จ

8.3.2 frontend/index.js

ภาพรวม (Overview)

ไฟล์ frontend/index.js เป็น Web Server อย่างง่ายที่พัฒนาด้วย Node.js (Native Module) โดยไม่พึ่งพา Framework ภายนอก ทำหน้าที่ให้บริการไฟล์ Static (Static File Server) ตามคำขอของผู้ใช้งาน โดยมีการกำหนด MIME Types และมาตรการความปลอดภัยเบื้องต้นเพื่อป้องกันการเข้าถึงไฟล์ที่ไม่ได้รับอนุญาต

การนำเข้าโมดูลและการกำหนดค่า (Imports and Configuration)

ส่วนนี้เป็นการนำเข้าโมดูลพื้นฐานของ Node.js และการกำหนดพอร์ต รวมถึงประเภทของไฟล์ (MIME Types) ที่รองรับ

```
1 const http = require("http");  
2 const fs = require("fs");  
3 const path = require("path");  
4 const url = require("url");  
5  
6 const root = __dirname;  
7 const port = process.env.PORT ? parseInt(process.env.PORT, 10) : 5500;  
8  
9 const mime = {  
10   ".html": "text/html; charset=utf-8",  
11   ".css": "text/css; charset=utf-8",  
12   ".js": "application/javascript; charset=utf-8",  
13   ".json": "application/json; charset=utf-8",  
14   ".png": "image/png",  
15   ".jpg": "image/jpeg",  
16   ".jpeg": "image/jpeg",  
17   ".svg": "image/svg+xml",  
18   ".ico": "image/x-icon",  
19 };
```

Listing 8.21: การตั้งค่า Server และ MIME Types ใน frontend/index.js

คำอธิบาย:

1. http: ใช้สำหรับสร้าง HTTP Server
2. fs: ใช้สำหรับจัดการไฟล์ (อ่านไฟล์เพื่อส่งกลับไปยัง Client)
3. path: ใช้สำหรับจัดการเส้นทางไฟล์และตรวจสอบนามสกุลไฟล์
4. mime: ออบเจกต์ที่เก็บค่า Content-Type สำหรับไฟล์ประเภทต่างๆ เพื่อให้ Browser แสดงผลได้ถูกต้อง

มาตรการความปลอดภัย (Security: Path Traversal Prevention)

ฟังก์ชัน `safeJoin` ถูกสร้างขึ้นเพื่อป้องกันช่องโหว่ Path Traversal ซึ่งอาจทำให้ผู้ไม่หวังดีสามารถเข้าถึงไฟล์นอกเหนือจาก Root Directory ที่กำหนดได้

```

1 const safeJoin = (base, target) => {
2   const targetPath = path.resolve(base, target.replace(/^\/+/, ""));
3   if (!targetPath.startsWith(base)) return null;
4   return targetPath;
5 };

```

Listing 8.22: ฟังก์ชัน safeJoin ป้องกัน Path Traversal

คำอธิบาย:

1. ใช้ `path.resolve` เพื่อสร้างเส้นทางไฟล์ที่สมบูรณ์
2. ตรวจสอบว่าเส้นทางปลายทาง (`targetPath`) ยังคงอยู่ภายใต้โฟลเดอร์ราก (`base`) หรือไม่ หากไม่อยู่ จะคืนค่า `null`

การจัดการคำขอ (Request Handling Logic)

ส่วนหลักของ Server ที่ทำหน้าที่รับ Request, ตรวจสอบเส้นทาง, และส่งไฟล์กลับไปยังผู้ใช้งาน

```

1 const server = http.createServer((req, res) => {
2   const parsed = url.parse(req.url);
3   let pathname = parsed.pathname || "/";
4   if (pathname === "/") pathname = "/index.html";
5
6   const filePath = safeJoin(root, pathname);
7   if (!filePath) {
8     res.writeHead(400);
9     res.end("Bad Request");
10    return;
11  }
12
13  fs.stat(filePath, (err, stat) => {
14    if (err) {
15      res.writeHead(404);
16      res.end("Not Found");
17      return;
18    }
19    if (stat.isDirectory()) {
20      // Handle directory access...
21    }
22    streamFile(filePath, res);
23  });
24 });

```

Listing 8.23: Logic การจัดการ Request

คำอธิบาย:

1. แปลง URL และตรวจสอบว่าถ้าเป็น Root Path (/) ให้เปลี่ยนเป็น `/index.html`
2. เรียกใช้ `safeJoin` เพื่อความปลอดภัย
3. ใช้ `fs.stat` ตรวจสอบว่ามีไฟล์อยู่จริงหรือไม่ ถ้าไม่มีให้ส่งคืน 404 Not Found

ฟังก์ชันการส่งไฟล์ (File Streaming Helper)

ฟังก์ชัน `streamFile` ทำหน้าที่อ่านไฟล์และส่งข้อมูลกลับไปยัง Client ผ่าน HTTP Response Stream

```

1 function streamFile(filePath, res) {
2   const ext = path.extname(filePath).toLowerCase();
3   const type = mime[ext] || "application/octet-stream";
4   res.setHeader("Content-Type", type);
5   res.setHeader("Cache-Control", "no-cache");
6   const stream = fs.createReadStream(filePath);
7   stream.pipe(res);
8 }

```

Listing 8.24: ฟังก์ชัน streamFile

8.3.3 frontend/js/config.js

ภาพรวม (Overview)

ไฟล์ `frontend/js/config.js` เป็นไฟล์กำหนดค่าเริ่มต้น (Configuration File) สำหรับฝั่ง Frontend ของระบบ Gym-Bro โดยทำหน้าที่ประกาศตัวแปรส่วนกลาง (Global Variable) ที่ใช้สำหรับอ้างอิง URL พื้นฐานของ Backend API (API Base URL) เพื่อให้ไฟล์ JavaScript อื่นๆ สามารถเรียกใช้งานและเชื่อมต่อกับ Backend ได้อย่างถูกต้องและเป็นมาตรฐานเดียวกันทั้งระบบ

การกำหนดค่า Base URL (Base URL Configuration)

ส่วนนี้เป็นการประกาศตัวแปร `window.API` ซึ่งจะถูกใช้เป็น Prefix สำหรับการเรียก API Request ในแอปพลิเคชัน

```
1 // Backend API base URL (Local)
2 window.API = "http://localhost:3000/api";
```

Listing 8.25: การกำหนดค่า API Base URL ใน `frontend/js/config.js`

คำอธิบาย:

1. `window.API`: การประกาศตัวแปรในระดับ `window` ทำให้ตัวแปรนี้มีสถานะเป็น Global Variable ซึ่งสามารถเข้าถึงได้จากทุกไฟล์ JavaScript ที่ถูกโหลดในหน้าเว็บเดียวกัน โดยไม่ต้องทำการ Import เข้า
2. `"http://localhost:3000/api"`: กำหนดให้ชี้ไปยัง Backend Server ที่รันอยู่บนพอร์ต 3000 และมี Path Prefix เป็น `/api`
3. ประโยชน์ของการแยกไฟล์ Config คือความสะดวกในการเปลี่ยน Environment (เช่น จาก Localhost ไปยัง Production Server) โดยแก้ไขเพียงจุดเดียว

8.3.4 frontend/js/auth.js

ภาพรวม (Overview)

ไฟล์ `frontend/js/auth.js` เป็นหัวใจหลักของระบบรักษาความปลอดภัยฝั่ง Frontend (Client-Side Security) ของแอปพลิเคชัน GymBro ไฟล์นี้มีหน้าที่รับผิดชอบในการตรวจสอบสถานะการเข้าสู่ระบบของผู้ใช้ (Authentication State Management), การควบคุมการเข้าถึงหน้าเว็บต่างๆ (Route Guarding), และการจัดการกระบวนการล็อกอินและล็อกเอาต์ เพื่อให้มั่นใจว่าผู้ใช้จะสามารถเข้าถึงข้อมูลได้เฉพาะในส่วนที่ได้รับอนุญาตเท่านั้น

ตัวแปรและการตรวจสอบหน้าปัจจุบัน (Global Variables & Page Check)

ส่วนนี้เป็นการกำหนดตัวแปรเริ่มต้นและตรวจสอบว่าหน้าปัจจุบันคือหน้า Login หรือไม่ เพื่อใช้ในการตัดสินใจว่าจะรัน Logic การตรวจสอบสิทธิ์หรือไม่

```
1 // Check if we are on the login page
2 const isLoginPage = window.location.pathname === '/login' || window.location.pathname === '/';
```

Listing 8.26: ซอร์สโค้ดจากไฟล์ `frontend/js/auth.js`: การตรวจสอบหน้า Login

คำอธิบายโค้ด:

1. **จุดประสงค์:** เพื่อระบุว่าผู้ใช้งานกำลังอยู่ที่หน้าเข้าสู่ระบบหรือไม่
2. **Output:** ตัวแปร `isLoginPage` (Boolean) มีค่าเป็น `true` หาก URL path คือ `/login` หรือ `/`
3. **ขั้นตอนการทำงาน:**
 - (a) อ่านค่า `window.location.pathname`
 - (b) เปรียบเทียบกับ string ที่กำหนด

ฟังก์ชัน logout (Global Logout Function)

ฟังก์ชันสำหรับการออกจากระบบ ซึ่งถูกประกาศให้เป็น Global Function (window.logout) เพื่อให้สามารถเรียกใช้ได้จากทุกจุดในแอปพลิเคชัน (เช่น จาก Navbar)

```
1 window.logout = function() {  
2     if (confirm('Are you sure you want to log out?')) {  
3         localStorage.removeItem('gymbro_user');  
4         window.location.href = '/login';  
5     }  
6 };
```

Listing 8.27: ซอร์สโค้ดจากไฟล์ frontend/js/auth.js: ฟังก์ชัน logout

คำอธิบายโค้ด:

1. **จุดประสงค์:** ล้างข้อมูล Session ของผู้ใช้ออกจาก Browser และนำผู้ใช้กลับไปยังหน้า Login
2. **Input:** ไม่มี (Triggered by User Action)
3. **Output:** การเปลี่ยนหน้าไปยัง /login
4. **ขั้นตอนการทำงาน:**
 - (a) แสดง Dialog ยืนยันการลือกเอาต์ (confirm)
 - (b) หากผู้ใช้ยืนยัน ให้ลบ key 'gymbro_user' ออกจาก localStorage
 - (c) สั่ง Redirect หน้าเว็บไปยัง /login

ลอจิกตรวจสอบสิทธิ์ (Auth Guard Logic)

ส่วนนี้คือ Logic หลักที่จะทำงานทันทีที่โหลดไฟล์ เพื่อตรวจสอบว่าผู้ใช้มีสิทธิ์อยู่ในหน้านี้หรือไม่ (Route Protection)

```
1 if (!isLoginPage) {  
2     const session = JSON.parse(localStorage.getItem('gymbro_user'));  
3  
4     if (!session || !session.user) {  
5         window.location.href = '/login';  
6     } else {  
7         // Role Guard  
8         const path = window.location.pathname;  
9         const role = session.role;  
10  
11         if (path.startsWith('/admin') && role !== 'admin') {  
12             alert('Access Denied: Admin only.');13             window.location.href = '/user/dashboard';  
14         }  
15     }  
16 } else {  
17     // If on login page and already logged in, redirect  
18     const session = JSON.parse(localStorage.getItem('gymbro_user'));  
19     if (session && session.user) {  
20         if (session.role === 'admin') {  
21             window.location.href = '/admin/dashboard';  
22         } else {  
23             window.location.href = '/user/dashboard';  
24         }  
25     }  
26 }
```

Listing 8.28: ซอร์สโค้ดจากไฟล์ frontend/js/auth.js: Auth Guard Logic

คำอธิบายโค้ด:

1. **จุดประสงค์:** ป้องกันการเข้าถึงหน้าเว็บโดยไม่ได้รับอนุญาต และ Redirect ผู้ใช้ไปยังหน้า Dashboard หากล็อกอินแล้ว
2. **Input:** localStorage และ window.location.pathname

3. ขั้นตอนการทำงาน:

- (a) ตรวจสอบว่าไม่ใช่หน้า Login
- (b) ดึงข้อมูล Session จาก Storage
- (c) กรณีไม่มี Session: Redirect ไปหน้า Login
- (d) กรณีมี Session: ตรวจสอบ Role ของผู้ใช้ หากเป็น User ธรรมดาแต่พยายามเข้าหน้า /admin จะถูกดีดกลับ ไปหน้า User Dashboard
- (e) กรณีอยู่หน้า Login แต่มี Session แล้ว: Redirect ไปยัง Dashboard ตาม Role ของผู้ใช้งานที่ (ไม่ต้องล็อกอินซ้ำ)

การจัดการฟอร์มล็อกอิน (Login Form Event Handler)

ส่วนนี้จัดการเหตุการณ์เมื่อผู้ใช้กดปุ่ม Submit ที่ฟอร์มล็อกอิน

```
1 const loginForm = document.getElementById('login-form');
2 if (loginForm) {
3   loginForm.addEventListener('submit', async (e) => {
4     e.preventDefault();
5     const emailVal = document.getElementById('email').value;
6     const phoneVal = document.getElementById('phone').value;
7
8     // Hide error
9     const errorMsg = document.getElementById('error-message');
10    if (errorMsg) errorMsg.classList.add('hidden');
11
12    try {
13      const response = await fetch(`${window.API}/login`, {
14        method: 'POST',
15        headers: { 'Content-Type': 'application/json' },
16        body: JSON.stringify({ email: emailVal, phone: phoneVal })
17      });
18
19      if (!response.ok) {
20        throw new Error('Login failed');
21      }
22
23      const data = await response.json();
24      localStorage.setItem('gymbro_user', JSON.stringify(data));
25
26      // Redirect based on role
27      if (data.role === 'admin') {
28        window.location.href = '/admin/dashboard';
29      } else {
30        window.location.href = '/user/dashboard';
31      }
32
33    } catch (error) {
34      console.error('Login Error:', error);
35      if (errorMsg) {
36        errorMsg.classList.remove('hidden');
37        errorMsg.innerText = 'Invalid email or phone number.';
38      }
39    }
40  });
41 }
```

Listing 8.29: ซอร์สโค้ดจากไฟล์ frontend/js/auth.js: Login Form Handler

คำอธิบายโค้ด:

- 1. จุดประสงค์: ส่งข้อมูลล็อกอินไปยัง Backend และจัดการผลลัพธ์
- 2. Input: Email และ Phone จากฟอร์ม
- 3. ขั้นตอนการทำงาน:
 - (a) ป้องกันการ Reload หน้าเว็บด้วย e.preventDefault()
 - (b) อ่านค่าจาก Input Fields

- (c) ช้อนข้อความ Error เก่า (ถ้ามี)
- (d) ส่ง POST Request ไปยัง /api/login
- (e) หากสำเร็จ: บันทึกข้อมูลลง localStorage และ Redirect ไปยัง Dashboard ที่เหมาะสม
- (f) หากล้มเหลว: แสดงข้อความ Error แจ้งผู้ใช้

ฟังก์ชัน displayUserInfo

ฟังก์ชันสำหรับแสดงชื่อและสถานะของผู้ใช้บนหน้า UI

```

1 function displayUserInfo() {
2   const session = JSON.parse(localStorage.getItem('gymbro_user'));
3   if (session && session.user) {
4     const userNameElements = document.querySelectorAll('.user-name-display');
5     userNameElements.forEach(el => {
6       el.textContent = session.user.fullName || session.user.email;
7     });
8
9     const userRoleElements = document.querySelectorAll('.user-role-display');
10    userRoleElements.forEach(el => {
11      el.textContent = session.role.toUpperCase();
12    });
13  }
14 }
15 // Call display info on load
16 document.addEventListener('DOMContentLoaded', displayUserInfo);

```

Listing 8.30: ซอร์สโค้ดจากไฟล์ frontend/js/auth.js: ฟังก์ชัน displayUserInfo

คำอธิบายโค้ด:

1. **จุดประสงค์:** อัปเดต UI ให้แสดงชื่อผู้ใช้ที่ล็อกอินอยู่
2. **Input:** ข้อมูล Session จาก localStorage
3. **ขั้นตอนการทำงาน:**
 - (a) อ่านข้อมูล User จาก Storage
 - (b) ค้นหา Elements ที่มี class .user-name-display และ .user-role-display ทั้งหมดในหน้า
 - (c) วนลูปอัปเดต textContent ของ Elements เหล่านั้นให้เป็นชื่อและ Role ของผู้ใช้

8.3.5 frontend/js/user/dashboard.js

ภาพรวม (Overview)

ไฟล์ frontend/js/user/dashboard.js รับผิดชอบการทำงานของหน้า Dashboard สำหรับสมาชิก (User) หลักๆ คือการแสดงตารางการฝึกซ้อมส่วนตัว (Personal Schedule) และระบบการจองเซสชัน (Booking System) ไฟล์นี้มีความซับซ้อนสูงเนื่องจากต้องมีการตรวจสอบความพร้อมของทรัพยากร (Availability Check) แบบเรียลไทม์เพื่อป้องกันการจองซ้ำซ้อน

ฟังก์ชัน Initialization (Event Listeners Setup)

โค้ดส่วนนี้ทำงานเมื่อ DOM โหลดเสร็จ เพื่อเตรียมข้อมูลและผูก Event Listeners

```

1 document.addEventListener('DOMContentLoaded', () => {
2   fetchSchedule();
3   fetchResources();
4
5   // Event Listeners for Modal
6   document.getElementById('btn-book-session').addEventListener('click', openBookingModal);
7   document.getElementById('close-booking-modal').addEventListener('click', closeBookingModal);
8   document.getElementById('cancel-booking').addEventListener('click', closeBookingModal);
9   document.getElementById('booking-modal').addEventListener('click', (e) => {
10     if (e.target.id === 'booking-modal') closeBookingModal();

```



```

11 });
12
13 // Form Change Listeners for Availability Check
14 ['book-date', 'book-time', 'book-duration'].forEach(id => {
15     document.getElementById(id).addEventListener('change', checkAvailability);
16 });
17
18 document.getElementById('booking-form').addEventListener('submit', handleBookingSubmit);
19 });

```

Listing 8.31: ซอร์สโค้ดจากไฟล์ frontend/js/user/dashboard.js: Initialization

คำอธิบายโค้ด:

1. จุดประสงค์: เริ่มต้นการทำงานของหน้าเว็บ
2. ขั้นตอนการทำงาน:
 - (a) เรียก fetchSchedule() เพื่อโหลดตารางงาน
 - (b) เรียก fetchResources() เพื่อโหลดข้อมูลเทรนเนอร์และอุปกรณ์
 - (c) ผูก Event Click สำหรับเปิด/ปิด Modal
 - (d) ผูก Event Change สำหรับ Input Fields ในฟอร์มจอง เพื่อเรียก checkAvailability() ทุกครั้งที่มีการเปลี่ยนข้อมูล
 - (e) ผูก Event Submit สำหรับฟอร์มจอง

ฟังก์ชัน fetchResources

ดึงข้อมูลเทรนเนอร์และอุปกรณ์ทั้งหมดมาเก็บไว้ในตัวแปร Global เพื่อใช้ในการสร้างตัวเลือกใน Dropdown

```

1 async function fetchResources() {
2     try {
3         const [tRes, eRes] = await Promise.all([
4             fetch(`${API}/trainers`),
5             fetch(`${API}/equipments`)
6         ]);
7         allTrainers = await tRes.json();
8         allEquipments = await eRes.json();
9     } catch (error) {
10         console.error("Failed to fetch resources:", error);
11     }
12 }

```

Listing 8.32: ซอร์สโค้ดจากไฟล์ frontend/js/user/dashboard.js: ฟังก์ชัน fetchResources

คำอธิบายโค้ด:

1. จุดประสงค์: โหลดข้อมูล Master Data (Trainers, Equipments)
2. ขั้นตอนการทำงาน:
 - (a) ใช้ Promise.all เพื่อยิง Request 2 ตัวพร้อมกัน (Parallel Fetching)
 - (b) เก็บผลลัพธ์ลงในตัวแปร Global allTrainers และ allEquipments

ฟังก์ชัน fetchSchedule

ดึงข้อมูลตารางการฝึกซ้อมของ "วันนี้" มาแสดงผล

```

1 async function fetchSchedule() {
2     try {
3         const res = await fetch(`${API}/sessions?today=true`);
4         if (!res.ok) throw new Error('Failed to fetch schedule');
5         const sessions = await res.json();
6         renderSchedule(sessions);
7     } catch (error) {
8         console.error(error);
9     }
10 }

```

```

9      // Error handling UI...
10    }
11  }

```

Listing 8.33: ซอร์สโค้ดจากไฟล์ frontend/js/user/dashboard.js: ฟังก์ชัน fetchSchedule

คำอธิบายโค้ด:

1. จุดประสงค์: ดึงข้อมูล Session ของวันนี้
2. ขั้นตอนการทำงาน:
 - (a) เรียก API /sessions?today=true
 - (b) ส่งข้อมูลที่ได้ไปให้ฟังก์ชัน renderSchedule วาดตาราง

ฟังก์ชัน renderSchedule

สร้าง HTML Table Rows จากข้อมูล Session โดยมีการแยกแยะระหว่าง Session ของผู้ใช้งานกับของผู้อื่น

```

1 function renderSchedule(sessions) {
2   // ... (Get User ID from Session)
3   sessions.forEach(s => {
4     // ... (Date Formatting Logic)
5
6     // Check if this session belongs to the current user
7     const customerId = s.customerId || (s.customer ? s.customer.id : null);
8     const isMySession = customerId === myId;
9
10    // ... (Create TR element)
11
12    if (isMySession) {
13      tr.className = "bg-accent/10 border-l-4 border-accent ...";
14      actionCell = `... My Booking ... Cancel Button ...`;
15    } else {
16      tr.className = "hover:bg-contrast ...";
17      actionCell = `<span class="...">Occupied</span>`;
18    }
19
20    // ... (Append to Tbody)
21  });
22 }

```

Listing 8.34: ซอร์สโค้ดจากไฟล์ frontend/js/user/dashboard.js: ฟังก์ชัน renderSchedule

คำอธิบายโค้ด:

1. จุดประสงค์: แสดงผลตารางการฝึกซ้อม
2. Input: Array ของ Session Objects
3. ขั้นตอนการทำงาน:
 - (a) วนลูปข้อมูลแต่ละ Session
 - (b) ตรวจสอบว่าเป็นของผู้ใช้ปัจจุบันหรือไม่ (isMySession)
 - (c) หากใช่: ให้แสดงแถวสีพิเศษ (Highlight) และปุ่ม Cancel
 - (d) หากไม่ใช่: ให้แสดงแถวปกติและขึ้นสถานะ Occupied

ฟังก์ชัน openBookingModal และ closeBookingModal

จัดการการแสดงผลของ Modal จองเวลา โดยเมื่อเปิด Modal จะทำการโหลดข้อมูลตัวเลือกต่างๆ และตั้งค่าวันที่เริ่มต้น

```

1 function openBookingModal() {
2   document.getElementById('booking-modal').classList.remove('hidden');
3
4   // Set default date to today
5   const today = new Date().toISOString().split('T')[0];

```

```

6 document.getElementById('book-date').value = today;
7 document.getElementById('book-time').value = "09:00"; // Default time
8
9 populateSelects();
10 checkAvailability(); // Initial check
11 }
12
13 function closeBookingModal() {
14     document.getElementById('booking-modal').classList.add('hidden');
15 }

```

Listing 8.35: Modal Functions ใน user/dashboard.js

คำอธิบาย:

1. openBookingModal: ลบ คลาส hidden เพื่อ แสดง Modal, กำหนด ค่า วันที่ ปัจจุบัน, และ เรียก ฟังก์ชัน populateSelects เพื่อเตรียมข้อมูล
2. closeBookingModal: ซ่อน Modal โดยการเพิ่มคลาส hidden กลับเข้าไป

ฟังก์ชัน populateSelects

ดึงข้อมูลเทรนเนอร์และอุปกรณ์มาสร้างเป็นตัวเลือกใน Dropdown

```

1 function populateSelects() {
2     const trainerSelect = document.getElementById('book-trainer');
3     const equipmentSelect = document.getElementById('book-equipment');
4
5     trainerSelect.innerHTML = '<option value="">-- No Trainer --</option>';
6     allTrainers.forEach(t => {
7         trainerSelect.innerHTML += `<option value="${t.id}">${t.trainerName} (${t.specialty ||
8         ↪ 'General'})</option>`;
9     });
10
11     equipmentSelect.innerHTML = '<option value="">-- Select Equipment --</option>';
12     allEquipments.forEach(e => {
13         let statusText = "";
14         if (e.status === 'Maintenance') statusText = " (Maintenance)";
15         equipmentSelect.innerHTML += `<option value="${e.id}" ${e.status === 'Maintenance' ?
16         ↪ 'disabled' : ''}>${e.equipmentName}${statusText}</option>`;
17     });
18 }

```

Listing 8.36: ฟังก์ชัน populateSelects ใน user/dashboard.js

คำอธิบาย:

1. วนลูปสร้าง <option> จากข้อมูล allTrainers และ allEquipments
2. หากอุปกรณ์อยู่ในสถานะ 'Maintenance' จะทำการ Disable ตัวเลือกนั้นไม่ให้ผู้ใช้เลือกได้

ฟังก์ชัน checkAvailability

ตรวจสอบความว่างของเทรนเนอร์และอุปกรณ์ตามวันและเวลาที่เลือก

```

1 async function checkAvailability() {
2     // 1. Get Inputs
3     const dateVal = document.getElementById('book-date').value;
4     const timeVal = document.getElementById('book-time').value;
5     const durationVal = parseInt(document.getElementById('book-duration').value);
6
7     if (!dateVal || !timeVal) return;
8
9     // 2. Fetch Daily Sessions
10    const res = await fetch(`${API}/sessions?date=${dateVal}`);
11    dailySessions = await res.json();
12
13    // 3. Calculate Requested Time Window
14    const start = new Date(`${dateVal}T${timeVal}`);
15    const end = new Date(start.getTime() + durationVal * 60000);

```

```

16 // 4. Check Trainers Overlap
17 const trainerOptions = document.getElementById('book-trainer').options;
18 for (let i = 0; i < trainerOptions.length; i++) {
19   const trainerId = parseInt(trainerOptions[i].value);
20   const isBusy = dailySessions.some(s => {
21     if (s.trainerId !== trainerId) return false;
22     const sStart = new Date(s.sessionDate);
23     const sEnd = s.endDate ? new Date(s.endDate) : new Date(sStart.getTime() + 60*60000);
24     return (start < sEnd && end > sStart);
25   });
26
27   if (isBusy) {
28     trainerOptions[i].disabled = true;
29     trainerOptions[i].text += " (Busy)";
30   } else {
31     trainerOptions[i].disabled = false;
32   }
33 }
34 // ... (Same logic for Equipments)
35 }
36 }

```

Listing 8.37: ซอร์สโค้ดจากไฟล์ frontend/js/user/dashboard.js: ฟังก์ชัน checkAvailability

คำอธิบายโค้ด:

1. จุดประสงค์: ป้องกันการเลือกเทรนเนอร์หรืออุปกรณ์ที่ไม่ว่าง
2. ขั้นตอนการทำงาน:
 - (a) ดึงข้อมูลวัน เวลา และระยะเวลาที่ผู้ใช้เลือก
 - (b) ดึงข้อมูลการจองทั้งหมดของ "วันที่เลือก" มาจาก Server
 - (c) คำนวณช่วงเวลา Start-End ที่ผู้ใช้ต้องการจอง
 - (d) วนลูปตรวจสอบเทรนเนอร์ที่ละคนเทียบกับรายการจองที่มีอยู่
 - (e) หากพบว่าช่วงเวลาทับซ้อนกัน (Overlap) ให้ Disable ตัวเลือกนั้นใน Dropdown

ฟังก์ชัน handleBookingSubmit

จัดการการส่งฟอร์มการจองไปยัง Server

```

1 async function handleBookingSubmit(e) {
2   e.preventDefault();
3   // ... (Validation)
4
5   const start = new Date(`${dateVal}T${timeVal}`);
6
7   // Create UTC Date preserving local face value
8   const utcStart = new Date(Date.UTC(
9     start.getFullYear(), start.getMonth(), start.getDate(),
10    start.getHours(), start.getMinutes(), 0
11  ));
12
13   const payload = {
14     sessionDate: utcStart.toISOString(),
15     duration: parseInt(durationVal),
16     trainerId: parseInt(trainerId),
17     equipmentId: parseInt(equipmentId),
18     customerId: parseInt(customerId)
19   };
20
21   try {
22     const res = await fetch(`${API}/sessions`, {
23       method: 'POST',
24       body: JSON.stringify(payload)
25     });
26     // ... (Handle Success/Error)
27   } catch (error) { ... }
28 }

```

Listing 8.38: ซอร์สโค้ดจากไฟล์ frontend/js/user/dashboard.js: ฟังก์ชัน handleBookingSubmit

คำอธิบายโค้ด:

1. จุดประสงค์: บันทึกการจองใหม่ลงฐานข้อมูล
2. ขั้นตอนการทำงาน:
 - (a) ตรวจสอบความครบถ้วนของข้อมูล
 - (b) แปลงเวลา Local Time เป็น UTC (Fake UTC Strategy) เพื่อให้เวลาที่บันทึกตรงกับที่เลือก ไม่เพี้ยนตาม Timezone
 - (c) ส่ง POST Request ไปยัง API
 - (d) แจ้งเตือนผลลัพธ์และรีเฟรชตาราง

8.3.6 ไฟล์ frontend/views/index.ejs**ภาพรวม (Overview)**

ไฟล์ frontend/views/index.ejs คือหน้า Dashboard หลักสำหรับผู้ดูแลระบบ (Admin) ทำหน้าที่แสดงภาพรวมของระบบ เช่น สถิติสมาชิก, จำนวนเทรนเนอร์, อุปกรณ์, และเชสชั้นในวันนี้ รวมถึงแสดงตารางงาน (Schedule) ประจำวัน หน้าเว็บนี้ถูกออกแบบให้รองรับการแสดงผลบนมือถือ (Responsive Design) โดยใช้ Tailwind CSS

ส่วนหัวและการนำทาง (Head and Navigation)

ส่วนนี้ประกอบด้วยการตั้งค่า Meta Tags, การนำเข้า Tailwind CSS และ Lucide Icons, รวมถึง Sidebar Navigation สำหรับ Desktop

```

1 <!doctype html>
2 <html lang="th">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>GymBro • Dashboard</title>
7
8   <!-- Tailwind CSS via CDN for rapid prototyping/draft -->
9   <script src="https://cdn.tailwindcss.com"></script>
10  <script>
11    tailwind.config = {
12      theme: {
13        extend: {
14          colors: {
15            primary: '#0C2C55',
16            secondary: '#296374',
17            accent: '#629FAD',
18            contrast: '#EDEDCE',
19          },
20          fontFamily: {
21            sans: ['Inter', 'sans-serif'],
22          }
23        }
24      }
25    }
26  </script>
27
28  <!-- Google Fonts & Icons -->
29  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;800&display=swap"
30    ↪ rel="stylesheet">
31  <script src="https://unpkg.com/lucide@latest"></script>
32
33  <style>
34    body { font-family: 'Inter', sans-serif; }
35    /* Custom Scrollbar Styles */
36    ::-webkit-scrollbar { width: 8px; }
37    ::-webkit-scrollbar-track { background: #EDEDCE; }
38    ::-webkit-scrollbar-thumb { background: #296374; }

```

```

38 </style>
39 </head>
40 <body class="bg-contrast text-primary min-h-screen flex overflow-hidden">
41
42 <!-- Sidebar (Desktop) -->
43 <aside class="w-64 bg-primary text-contrast flex-shrink-0 flex flex-col border-r-4
44   ↳ border-accent hidden md:flex">
45   <div class="h-20 flex items-center justify-center border-b border-secondary">
46     <h1 class="text-3xl font-extrabold tracking-tighter uppercase italic">Gym<span
47       ↳ class="text-accent">Bro</span></h1>
48   </div>
49
50   <nav class="flex-1 overflow-y-auto py-6">
51     <ul class="space-y-1">
52       <li>
53         <a href="/admin/dashboard" class="flex items-center gap-4 px-6 py-4 bg-secondary
54           ↳ border-l-4 border-accent text-white font-bold uppercase tracking-wide transition-all">
55           <i data-lucide="layout-dashboard" class="w-5 h-5"></i>
56           <span>Dashboard</span>
57         </a>
58       </li>
59       <!-- Other Menu Items: Customers, Trainers, Equipments, Sessions, Logs -->
60       <li>
61         <button onclick="logout()" class="w-full text-left flex items-center gap-4 px-6 py-4
62           ↳ text-red-300 hover:bg-red-900/50 hover:text-red-100 font-semibold uppercase tracking-wide
63           ↳ transition-all group">
64           <i data-lucide="log-out" class="w-5 h-5 group-hover:text-red-400
65             ↳ transition-colors"></i>
66           <span>Logout</span>
67         </button>
68       </li>
69     </ul>
70   </nav>
71
72   <!-- Admin Profile Section -->
73   <div class="p-4 bg-secondary border-t border-primary">
74     <div class="flex items-center gap-3">
75       <div class="w-10 h-10 bg-accent flex items-center justify-center text-primary font-bold
76         ↳ text-lg">AD</div>
77       <div>
78         <p class="text-sm font-bold text-white">Admin User</p>
79         <p class="text-xs text-accent">Manager</p>
80       </div>
81     </div>
82   </div>
83 </aside>

```

Listing 8.39: ซอร์สโค้ดจากไฟล์ frontend/views/index.ejs: Head and Sidebar

คำอธิบาย:

1. **Tailwind Configuration:** มีการปรับแต่ง Theme ของ Tailwind โดยเพิ่มสีหลักของแบรนด์ (Primary, Secondary, Accent, Contrast) เพื่อให้การออกแบบเป็นไปในทิศทางเดียวกัน
2. **Sidebar:** แถบเมนูด้านซ้ายสำหรับ Desktop ซึ่งจะซ่อนเมื่อนำจอมีขนาดเล็ก (hidden md:flex)
3. **Active State:** เมนู Dashboard มีการเน้นสีพื้นหลัง (bg-secondary) เพื่อแสดงว่าผู้ใช้งานกำลังอยู่ในหน้านี้

ส่วนเนื้อหาหลักและสคริปต์ (Main Content and Scripts)

ส่วนนี้ประกอบด้วย Mobile Header, Cards แสดงสถิติ, ตารางงาน, และ JavaScript สำหรับจัดการ UI

```

1 <!-- Main Content -->
2 <main class="flex-1 flex flex-col h-screen overflow-hidden relative">
3
4 <!-- Mobile Header & Sidebar Overlay -->
5 <header class="h-16 bg-primary text-contrast flex items-center justify-between px-4
6   ↳ md:hidden shadow-lg z-10">
7   <button id="mobile-menu-btn" class="p-2 hover:bg-secondary text-white">
8     <i data-lucide="menu" class="w-6 h-6"></i>
9   </button>
10   <span class="font-bold uppercase tracking-wider">GymBro Dashboard</span>

```

```

10     <div class="w-8"></div>
11 </header>
12
13 <!-- Stats Grid -->
14 <div class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-6 mb-8">
15     <!-- Stat Card: Members -->
16     <div class="bg-white border-2 border-primary p-6 shadow-[4px_4px_0px_0px_#0C2C55]
17     ↪ hover:shadow-[6px_6px_0px_0px_#0C2C55] transition-all group">
18         <div class="flex justify-between items-start mb-4">
19             <div class="p-3 bg-secondary text-white"><i data-lucide="users" class="w-6
20             ↪ h-6"></i></div>
21             <span class="text-xs font-bold uppercase text-secondary bg-contrast px-2
22             ↪ py-1">Total</span>
23         </div>
24         <h3 class="text-3xl font-black text-primary" id="count-customers">0</h3>
25         <p class="text-sm font-bold text-gray-500 uppercase tracking-wide mt-1">Members</p>
26     </div>
27     <!-- Other cards: Trainers, Equipment, Sessions (Similar Structure) -->
28 </div>
29
30 <!-- Schedule Section -->
31 <div class="bg-white border-2 border-primary shadow-[8px_8px_0px_0px_#296374]">
32     <div class="bg-secondary p-4 flex justify-between items-center border-b-2
33     ↪ border-primary">
34         <h3 class="text-xl font-bold text-white uppercase tracking-wider flex items-center
35         ↪ gap-2">
36             <i data-lucide="calendar" class="w-5 h-5 text-accent"></i>
37             Today's Schedule
38         </h3>
39         <button class="text-sm font-bold text-contrast hover:text-accent underline
40         ↪ uppercase">View All</button>
41     </div>
42
43     <div class="p-0 overflow-x-auto">
44         <table class="w-full text-left border-collapse">
45             <thead>
46                 <tr class="bg-gray-100 border-b-2 border-primary text-primary text-sm uppercase
47                 ↪ tracking-wider">
48                     <th class="p-4 font-extrabold">Customer</th>
49                     <th class="p-4 font-extrabold">Trainer</th>
50                     <th class="p-4 font-extrabold">Activity/Equipment</th>
51                     <th class="p-4 font-extrabold">Time</th>
52                     <th class="p-4 font-extrabold text-center">Status</th>
53                 </tr>
54             </thead>
55             <tbody id="today-sessions" class="divide-y-2 divide-gray-100 text-sm font-semibold
56             ↪ text-secondary">
57                 <!-- JS will populate this -->
58             </tbody>
59         </table>
60     </div>
61 </div>
62 </main>
63
64 <!-- Scripts -->
65 <script>
66     lucide.createIcons();
67     // ... (Mobile Sidebar Logic)
68 </script>
69 <script src="/js/config.js"></script>
70 <script src="/js/auth.js"></script>
71 <script src="/js/index.js"></script>
72 </body>
73 </html>

```

Listing 8.40: ซอร์สโค้ดจากไฟล์ frontend/views/index.ejs: Main Content

คำอธิบาย:

1. **Mobile Responsiveness:** มีการใช้ Mobile Header และ Overlay Sidebar ที่จะทำงานเฉพาะในหน้าจอขนาดเล็ก
2. **Stats Cards:** การ์ดแสดงสถิติมีการใช้ id (เช่น count-customers) เพื่อให้ JavaScript สามารถดึงข้อมูลมาใส่ได้
3. **Script Loading:** มีการโหลด config.js, auth.js, และ index.js ตามลำดับ ซึ่งเป็นสิ่งสำคัญเพราะ index.js

อาจต้องใช้ตัวแปรจากไฟล์ก่อนหน้า

8.3.7 ไฟล์ frontend/views/login.ejs

ภาพรวม (Overview)

ไฟล์ frontend/views/login.ejs คือหน้าจอลงชื่อเข้าสู่ระบบ (Login Page) ซึ่งเป็นด่านแรกของผู้ใช้งานก่อนเข้าถึงระบบ GymBro หน้าเว็บนี้ถูกออกแบบด้วย Tailwind CSS โดยใช้สไตล์ที่เป็นเอกลักษณ์ (Bold/Brutalism Style) ที่เน้นเส้นขอบหนา สีสนิมที่ตัดกันชัดเจน และเงาแบบแข็ง (Hard Shadows) เพื่อสื่อถึงความแข็งแกร่งและทันสมัย

การออกแบบและโครงสร้าง (Design & Structure)

ส่วนนี้แสดงการตั้งค่า Theme สี และโครงสร้างหลักของหน้าเว็บที่จัดวางเนื้อหาให้อยู่กึ่งกลางหน้าจอ

```
1 <!doctype html>
2 <html lang="th">
3 <head>
4   <!-- Meta Tags & Title -->
5   <title>GymBro • Login</title>
6
7   <!-- Tailwind Configuration -->
8   <script src="https://cdn.tailwindcss.com"></script>
9   <script>
10     tailwind.config = {
11       theme: {
12         extend: {
13           colors: {
14             primary: '#0C2C55',
15             secondary: '#296374',
16             accent: '#629FAD',
17             contrast: '#EDEDCE',
18           },
19           // ... (Font & Border Radius Config)
20         }
21       }
22     }
23   </script>
24 </head>
25 <body class="bg-primary flex items-center justify-center min-h-screen p-4">
26
27   <!-- Main Card Container -->
28   <div class="w-full max-w-md bg-contrast shadow-[8px_8px_0px_0px_#629FAD] border-4
29     ↳ border-secondary p-8 relative">
30
31     <!-- Decorative Badge -->
32     <div class="absolute -top-6 -left-6 bg-accent text-primary px-4 py-2 font-black text-xl
33       ↳ border-2 border-primary shadow-[4px_4px_0px_0px_#0C2C55]">
34       LOGIN
35     </div>
36
37     <!-- Header / Logo -->
38     <div class="text-center mb-8">
39       <h1 class="text-4xl font-extrabold text-primary uppercase italic tracking-tighter
40         ↳ mb-2">Gym<span class="text-secondary">Bro</span></h1>
41       <p class="text-secondary font-bold text-sm uppercase tracking-widest">Queue Booking
42         ↳ System</p>
43     </div>
44
45     <!-- Form Section (See next listing) -->
46
47   </div>
48 </body>
49 </html>
```

Listing 8.41: ซอร์สโค้ดจากไฟล์ frontend/views/login.ejs: Head and Container

คำอธิบาย:

1. **Flexbox Centering:** ใช้ flex items-center justify-center min-h-screen ที่ body เพื่อจัดให้การ์ด Login อยู่กึ่งกลางหน้าจอเสมอไม่ว่าหน้าจอจะมีขนาดเท่าไร

2. **Custom Shadow:** การใช้ shadow-[8px_8px_0px_0px...] สร้างเงาแบบทึบและหนา ซึ่งเป็นเอกลักษณ์ของการออกแบบสไตล์นี้
3. **Absolute Positioning:** ป้าย "LOGIN" ถูกจัดวางด้วย absolute เพื่อให้ลอยออกมาจากกรอบหลัก สร้างมิติให้กับหน้าเว็บ

ฟอร์มการเข้าสู่ระบบ (Login Form)

ส่วนฟอร์มรับข้อมูลประกอบด้วยช่องกรอก Email และ Phone Number (ซึ่งใช้แทนรหัสผ่านในระบบนี้) รวมถึงส่วนแสดงข้อความแจ้งเตือนข้อผิดพลาด

```
1 <form id="login-form" class="space-y-6">
2   <!-- Email Input -->
3   <div>
4     <label for="email" class="block text-primary font-bold uppercase text-sm mb-2">Email
5     <input type="email" id="email" required
6       class="w-full bg-white border-2 border-primary p-3 pl-10 font-bold
7       focus:outline-none focus:border-accent placeholder-gray-400 text-primary"
8     placeholder="member@example.com">
9     <i data-lucide="mail" class="absolute left-3 top-3.5 w-5 h-5 text-secondary"></i>
10    </div>
11  </div>
12
13  <!-- Phone Input (Password) -->
14  <div>
15    <label for="phone" class="block text-primary font-bold uppercase text-sm mb-2">Phone
16    <input type="password" id="phone" required
17      class="w-full bg-white border-2 border-primary p-3 pl-10 font-bold
18      focus:outline-none focus:border-accent placeholder-gray-400 text-primary"
19    placeholder="0812345678">
20    <i data-lucide="phone" class="absolute left-3 top-3.5 w-5 h-5 text-secondary"></i>
21    </div>
22  </div>
23
24  <!-- Error Message (Hidden by default) -->
25  <div id="error-message" class="hidden text-red-600 font-bold text-sm text-center
26  bg-red-100 p-2 border-1-4 border-red-600">
27    Invalid credentials. Please try again.
28  </div>
29
30  <!-- Submit Button -->
31  <button type="submit"
32    class="w-full bg-primary text-contrast font-black uppercase py-4 hover:bg-secondary
33    hover:shadow-[4px_4px_0px_0px_#629FAD] transition-all active:translate-y-1
34    active:shadow-none border-2 border-transparent flex items-center justify-center gap-2
35    text-lg tracking-wider">
36    <i data-lucide="log-in" class="w-5 h-5"></i> Sign In
37  </button>
38 </form>
39
40 <!-- Scripts -->
41 <script src="/js/config.js"></script>
42 <script src="/js/auth.js"></script>
```

Listing 8.42: ซอร์สโค้ดจากไฟล์ frontend/views/login.ejs: Form Elements

คำอธิบาย:

1. **Input Styling:** ช่องกรอกข้อมูลมีการใช้ pl-10 (Padding Left) เพื่อเว้นที่สำหรับไอคอนด้านหน้า
2. **Error Handling:** div id="error-message" มีคลาส hidden อยู่เริ่มต้น และจะถูกแสดงผลด้วย JavaScript (ในไฟล์ auth.js) เมื่อการล็อกอินล้มเหลว
3. **Interactive Button:** ปุ่ม Submit มี Effect เมื่อเอาเมาส์ไปวาง (hover) และเมื่อกด (active) เพื่อให้ผู้ใช้รู้สึกถึงการตอบสนอง

4. JavaScript Integration: ไฟล์นี้โหลด auth.js ซึ่งเป็นหัวใจสำคัญในการจัดการ Logic การส่งข้อมูลฟอร์มไปยัง Server

8.3.8 ไฟล์ frontend/views/register.ejs

ภาพรวม (Overview)

ไฟล์ frontend/views/register.ejs คือหน้าลงทะเบียนสมาชิกใหม่ (Member Registration) หน้าเว็บนี้ยังคงใช้การออกแบบสไตล์ Brutalism ที่สอดคล้องกับหน้า Login แต่มีการปรับเปลี่ยนเนื้อหาภายในฟอร์มให้เหมาะสมกับการเก็บข้อมูลส่วนตัวเบื้องต้นของผู้ใช้

ฟอร์มลงทะเบียน (Registration Form)

ฟอร์มนี้รับข้อมูลสำคัญ 3 ส่วน ได้แก่ ชื่อ-นามสกุล, อีเมล, และเบอร์โทรศัพท์ (ซึ่งใช้เป็นรหัสผ่าน)

```
1 <form id="register-form" class="space-y-4">
2   <!-- Full Name -->
3   <div>
4     <label for="fullName" class="block text-primary font-bold uppercase text-xs mb-1">Full
    Name</label>
5     <input type="text" id="fullName" required
6       class="w-full bg-white border-2 border-primary p-2 font-bold focus:outline-none
    focus:border-accent text-primary"
7     placeholder="John Doe">
8   </div>
9
10  <!-- Email -->
11  <div>
12    <label for="email" class="block text-primary font-bold uppercase text-xs mb-1">Email
    Address</label>
13    <input type="email" id="email" required
14      class="w-full bg-white border-2 border-primary p-2 font-bold focus:outline-none
    focus:border-accent text-primary"
15    placeholder="member@example.com">
16  </div>
17
18  <!-- Phone (Password) -->
19  <div>
20    <label for="phone" class="block text-primary font-bold uppercase text-xs mb-1">Phone
    Number (Password)</label>
21    <input type="tel" id="phone" required
22      class="w-full bg-white border-2 border-primary p-2 font-bold focus:outline-none
    focus:border-accent text-primary"
23    placeholder="0812345678">
24  </div>
25
26  <!-- Error Message Area -->
27  <div id="error-message" class="hidden text-red-600 font-bold text-xs text-center
    bg-red-100 p-2 border-1-4 border-red-600">
28    Registration failed.
29  </div>
30
31  <!-- Submit Button -->
32  <button type="submit"
33    class="w-full bg-primary text-contrast font-black uppercase py-3 hover:bg-secondary
    hover:shadow-[4px_4px_0px_0px_#629FAD] transition-all active:translate-y-1
    active:shadow-none border-2 border-transparent flex items-center justify-center gap-2
    text-md tracking-wider mt-4">
34    <i data-lucide="user-plus" class="w-5 h-5"></i> Create Account
35  </button>
36 </form>
```

Listing 8.43: ซอร์สโค้ดจากไฟล์ frontend/views/register.ejs: Registration Form

คำอธิบาย:

1. **Input Types:** มีการใช้ type="tel" สำหรับเบอร์โทรศัพท์เพื่อให้คีย์บอร์ดบนมือถือแสดงผลตัวเลขได้สะดวกขึ้น
2. **Compact Design:** ใช้ text-xs และ p-2 เพื่อให้ฟอร์มดูกระชับและไม่กินพื้นที่หน้าจามากเกินไป

สคริปต์การจัดการการลงทะเบียน (Registration Script)

เนื่องจากหน้างานนี้มีการทำงานที่ไม่ซับซ้อนมาก จึงมีการฝัง Script ไว้ภายในไฟล์โดยตรง (Inline Script) เพื่อจัดการการส่งข้อมูลไปยัง API

```
1 <script src="/js/config.js"></script>
2 <script>
3   lucide.createIcons();
4
5   const form = document.getElementById('register-form');
6   const errorMsg = document.getElementById('error-message');
7
8   form.addEventListener('submit', async (e) => {
9     e.preventDefault();
10    errorMsg.classList.add('hidden');
11
12    const fullName = document.getElementById('fullName').value;
13    const email = document.getElementById('email').value;
14    const phone = document.getElementById('phone').value;
15
16    try {
17      const res = await fetch(`${window.API}/register`, {
18        method: 'POST',
19        headers: { 'Content-Type': 'application/json' },
20        body: JSON.stringify({ fullName, email, phone })
21      });
22
23      if (res.ok) {
24        alert('Registration successful! Please login.');
```

Listing 8.44: ซอร์สโค้ดจากไฟล์ frontend/views/register.ejs: Inline Script

คำอธิบาย:

1. **Config Import:** นำเข้า config.js เพื่อให้สามารถเข้าถึงตัวแปร window.API ได้
2. **Form Submission:** ดักจับ Event submit ป้องกันการ Refresh หน้าจอ และส่งข้อมูล JSON ไปยัง Endpoint / register
3. **Success Handling:** หากสมัครสำเร็จ จะแสดง Alert แจ้งเตือนและ Redirect ไปยังหน้า Login
4. **Error Handling:** หากสมัครไม่สำเร็จ จะดึงข้อความ Error จาก Server มาแสดงในพื้นที่ที่เตรียมไว้

8.3.9 ไฟล์ frontend/views/customers.ejs

ภาพรวม (Overview)

ไฟล์ frontend/views/customers.ejs คือหน้าจอสําหรับผู้ดูแลระบบ (Admin) เพื่อใช้ในการบริหารจัดการข้อมูลสมาชิก (Member Management) หน้าจอนี้ประกอบด้วยฟอร์มสำหรับเพิ่มสมาชิกใหม่ และตารางแสดงรายชื่อสมาชิกทั้งหมดในระบบ รวมถึงสถานะและระดับสมาชิก (Member Level) โดยมีการออกแบบให้รองรับการใช้งานบนอุปกรณ์ทุกขนาด (Responsive Design)

การนำทางและสถานะเมนู (Navigation State)

ในหน้านี้ เมนู "Customers" ใน Sidebar จะถูกเน้น (Active State) เพื่อให้ผู้ใช้ทราบว่ากำลังใช้งานอยู่ในส่วนใด

```

1 <!-- Sidebar Navigation Item for Customers -->
2 <li>
3   <a href="/admin/customers" class="flex items-center gap-4 px-6 py-4 bg-secondary border-l-4
4     ↳ border-accent text-white font-bold uppercase tracking-wide transition-all">
5     <i data-lucide="users" class="w-5 h-5"></i>
6     <span>Customers</span>
7   </a>
8 </li>

```

Listing 8.45: ซอร์สโค้ดจากไฟล์ frontend/views/customers.ejs: Sidebar Navigation

คำอธิบาย:

1. **Active State:** ลิงก์ไปยัง /admin/customers จะมีคลาส bg-secondary และ border-l-4 border-accent ซึ่งแตกต่างจากเมนูอื่นๆ ที่เป็นสีเทาและไม่มีขอบซ้ายสีฟ้า

ฟอร์มเพิ่มสมาชิกใหม่ (Add Member Form)

ส่วนนี้เป็นฟอร์มสำหรับลงทะเบียนสมาชิกใหม่ ถูกออกแบบโดยใช้ Grid Layout เพื่อให้แสดงผลได้สวยงามและเป็นระเบียบ

```

1 <!-- Add Customer Form -->
2 <div class="bg-white border-2 border-primary p-6 mb-8 shadow-[4px_4px_0px_0px_#0C2C55]">
3   <h3 class="text-xl font-bold uppercase mb-4 flex items-center gap-2">
4     <i data-lucide="user-plus" class="w-5 h-5 text-accent"></i> Add New Member
5   </h3>
6   <form id="customer-form" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
7     <!-- Full Name Input -->
8     <input id="name" placeholder="Full Name" required class="border-2 border-primary p-3
9       ↳ font-bold focus:outline-none focus:border-accent placeholder-gray-400">
10
11     <!-- Email Input -->
12     <input id="email" placeholder="Email" required type="email" class="border-2 border-primary
13       ↳ p-3 font-bold focus:outline-none focus:border-accent placeholder-gray-400">
14
15     <!-- Phone Input -->
16     <input id="phone" placeholder="Phone" class="border-2 border-primary p-3 font-bold
17       ↳ focus:outline-none focus:border-accent placeholder-gray-400">
18
19     <!-- Member Type Select -->
20     <select id="memberType" required class="border-2 border-primary p-3 font-bold
21       ↳ focus:outline-none focus:border-accent bg-white">
22       <option value="Standard">Standard</option>
23       <option value="Premium">Premium</option>
24       <option value="VIP">VIP</option>
25     </select>
26
27     <!-- Member Level Select -->
28     <select id="memberLevel" required class="border-2 border-primary p-3 font-bold
29       ↳ focus:outline-none focus:border-accent bg-white">
30       <option value="Basic">Basic</option>
31       <option value="Silver">Silver</option>
32       <option value="Gold">Gold</option>
33     </select>
34
35     <!-- Start Date -->
36     <div class="flex flex-col">
37       <label class="text-xs font-bold uppercase text-secondary mb-1">Start Date</label>
38       <input id="memberStartDate" type="date" required class="border-2 border-primary p-2
39         ↳ font-bold focus:outline-none focus:border-accent">
40     </div>
41
42     <!-- End Date -->
43     <div class="flex flex-col">
44       <label class="text-xs font-bold uppercase text-secondary mb-1">End Date</label>
45       <input id="memberEndDate" type="date" class="border-2 border-primary p-2 font-bold
46         ↳ focus:outline-none focus:border-accent">
47     </div>
48
49     <!-- Submit Button -->
50     <button type="submit" class="bg-primary text-contrast font-bold uppercase
51       ↳ hover:bg-secondary hover:shadow-[2px_2px_0px_0px_#629FAD] transition-all

```

```

45     active:translate-y-1 active:shadow-none border-2 border-transparent h-full flex items-center
    ↪ justify-center gap-2">
46     <i data-lucide="save" class="w-4 h-4"></i> Save Member
47   </button>
48 </form>
49 </div>

```

Listing 8.46: ซอร์สโค้ดจากไฟล์ frontend/views/customers.ejs: Add Member Form

คำอธิบาย:

1. **Grid System:** ใช้ grid-cols-1 สำหรับมือถือ, md:grid-cols-2 สำหรับแท็บเล็ต, และ lg:grid-cols-4 สำหรับหน้าจอคอมพิวเตอร์ เพื่อให้การจัดวางฟอร์มมีความยืดหยุ่น
2. **Input Styling:** 필ด์กรอกข้อมูลมีขอบหนา (border-2) และเปลี่ยนสีขอบเมื่อ Focus (focus:border-accent) เพื่อความชัดเจนในการใช้งาน

ตารางรายชื่อสมาชิก (Member List Table)

ส่วนแสดงผลข้อมูลสมาชิกทั้งหมดในรูปแบบตาราง ซึ่งจะถูกเติมข้อมูลโดย JavaScript

```

1 <!-- Customer List -->
2 <div class="bg-white border-2 border-primary shadow-[8px_8px_0px_#296374]">
3   <div class="bg-secondary p-4 border-b-2 border-primary">
4     <h3 class="text-xl font-bold text-white uppercase tracking-wider flex items-center gap-2">
5       <i data-lucide="users" class="w-5 h-5 text-accent"></i> Member List
6     </h3>
7   </div>
8   <div class="overflow-x-auto">
9     <table class="w-full text-left border-collapse">
10      <thead>
11        <tr class="bg-gray-100 border-b-2 border-primary text-primary text-sm uppercase
    ↪ tracking-wider">
12          <th class="p-4 font-extrabold">ID</th>
13          <th class="p-4 font-extrabold">Name</th>
14          <th class="p-4 font-extrabold">Contact</th>
15          <th class="p-4 font-extrabold">Type/Level</th>
16          <th class="p-4 font-extrabold">Dates</th>
17          <th class="p-4 font-extrabold text-center">Actions</th>
18        </tr>
19      </thead>
20      <tbody id="customers-table" class="divide-y-2 divide-gray-100 text-sm font-semibold
    ↪ text-secondary">
21        <!-- JS Populated -->
22      </tbody>
23    </table>
24  </div>
25 </div>

```

Listing 8.47: ซอร์สโค้ดจากไฟล์ frontend/views/customers.ejs: Member List Table

คำอธิบาย:

1. **Table Structure:** โครงสร้างตารางมาตรฐานที่มีส่วนหัว (Thead) และส่วนเนื้อหา (Tbody)
2. **JavaScript Integration:** tbody id="customers-table" ถูกเตรียมไว้สำหรับให้ JavaScript (customers.js) เข้ามาจัดการ DOM เพื่อแสดงรายการสมาชิก

การนำเข้าสคริปต์ (Script Imports)

ไฟล์นี้มีการนำเข้าไฟล์ JavaScript ที่จำเป็นสำหรับการทำงาน

```

1 <script src="/js/config.js"></script>
2 <script src="/js/auth.js"></script>
3 <script src="/js/customers.js"></script>

```

Listing 8.48: ซอร์สโค้ดจากไฟล์ frontend/views/customers.ejs: Scripts

คำอธิบาย:

1. config.js: สำหรับการตั้งค่า API Base URL
2. auth.js: สำหรับตรวจสอบสิทธิ์การใช้งานของผู้ดูแลระบบ
3. customers.js: สคริปต์หลักสำหรับการจัดการ Logic ของหน้า Customers (เช่น การดึงข้อมูลสมาชิก, การเพิ่มสมาชิกใหม่, การลบสมาชิก)

8.3.10 ไฟล์ frontend/views/equipments.ejs

ภาพรวม (Overview)

ไฟล์ frontend/views/equipments.ejs คือหน้าจอสำหรับผู้ดูแลระบบ (Admin) เพื่อใช้ในการบริหารจัดการข้อมูลอุปกรณ์ออกกำลังกายภายในยิม (Equipment Management) ผู้ดูแลระบบสามารถเพิ่มอุปกรณ์ใหม่ แก้ไขสถานะของอุปกรณ์ (เช่น พร้อมใช้งาน หรือ ปิดปรับปรุง) และดูรายการอุปกรณ์ทั้งหมดที่มีอยู่ในระบบได้

การนำทางและสถานะเมนู (Navigation State)

ในหน้านี้ เมนู "Equipment" ใน Sidebar จะถูกเน้น (Active State) ด้วยแถบสีพื้นหลังและขอบด้านซ้าย

```

1 <!-- Sidebar Navigation Item for Equipment -->
2 <li>
3   <a href="/admin/equipments" class="flex items-center gap-4 px-6 py-4 bg-secondary border-l-4
   ↳ border-accent text-white font-bold uppercase tracking-wide transition-all">
4     <i data-lucide="monitor-smartphone" class="w-5 h-5"></i>
5     <span>Equipment</span>
6   </a>
7 </li>

```

Listing 8.49: ซอร์สโค้ดจากไฟล์ frontend/views/equipments.ejs: Sidebar Navigation

ฟอร์มเพิ่มอุปกรณ์ใหม่ (Add Equipment Form)

ฟอร์มสำหรับเพิ่มข้อมูลอุปกรณ์ใหม่ โดยมีการเลือกสถานะเริ่มต้นได้ (Available หรือ Maintenance)

```

1 <!-- Add Equipment Form -->
2 <div class="bg-white border-2 border-primary p-6 mb-8 shadow-[4px_4px_0px_0px_#0C2C55]">
3   <h3 class="text-xl font-bold uppercase mb-4 flex items-center gap-2">
4     <i data-lucide="plus-square" class="w-5 h-5 text-accent"></i> Add New Equipment
5   </h3>
6   <form id="equipment-form" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-4">
7     <!-- Equipment Name -->
8     <input id="name" placeholder="Equipment Name" required class="border-2 border-primary p-3
9       ↳ font-bold focus:outline-none focus:border-accent placeholder-gray-400">
10
11     <!-- Category -->
12     <input id="category" placeholder="Category" class="border-2 border-primary p-3 font-bold
13       ↳ focus:outline-none focus:border-accent placeholder-gray-400">
14
15     <!-- Status Select -->
16     <select id="status" required class="border-2 border-primary p-3 font-bold focus:outline-none
17       ↳ focus:border-accent bg-white">
18       <option value="Available">Available</option>
19       <option value="Maintenance">Maintenance</option>
20     </select>
21
22     <!-- Save Button -->
23     <button type="submit" class="bg-primary text-contrast font-bold uppercase hover:bg-secondary
24       ↳ hover:shadow-[2px_2px_0px_0px_#629FAD] transition-all active:translate-y-1
25       ↳ active:shadow-none border-2 border-transparent h-full flex items-center justify-center
26       ↳ gap-2">
27       <i data-lucide="save" class="w-4 h-4"></i> Save Equipment
28     </button>
29   </form>
30 </div>

```

Listing 8.50: ซอร์สโค้ดจากไฟล์ frontend/views/equipments.ejs: Add Equipment Form

คำอธิบาย:

1. **Status Selection:** มี Dropdown สำหรับเลือกสถานะของอุปกรณ์ ซึ่งสำคัญต่อการจองคลาส (Session) เพราะอุปกรณ์ที่อยู่ในสถานะ Maintenance จะไม่สามารถถูกจองได้

ตารางรายการอุปกรณ์ (Equipment Inventory Table)

ส่วนแสดงรายการอุปกรณ์ทั้งหมด พร้อมปุ่มสำหรับดำเนินการ (Actions) เช่น การลบหรือแก้ไข

```
1 <!-- Equipment List -->
2 <div class="bg-white border-2 border-primary shadow-[8px_8px_0px_0px_#296374]">
3   <div class="bg-secondary p-4 border-b-2 border-primary">
4     <h3 class="text-xl font-bold text-white uppercase tracking-wider flex items-center gap-2">
5       <i data-lucide="monitor-smartphone" class="w-5 h-5 text-accent"></i> Equipment Inventory
6     </h3>
7   </div>
8   <div class="overflow-x-auto">
9     <table class="w-full text-left border-collapse">
10      <thead>
11        <tr class="bg-gray-100 border-b-2 border-primary text-primary text-sm uppercase
12          ↳ tracking-wider">
13          <th class="p-4 font-extrabold">ID</th>
14          <th class="p-4 font-extrabold">Name</th>
15          <th class="p-4 font-extrabold">Category</th>
16          <th class="p-4 font-extrabold">Status</th>
17          <th class="p-4 font-extrabold text-center">Actions</th>
18        </tr>
19      </thead>
20      <tbody id="equipments-table" class="divide-y-2 divide-gray-100 text-sm font-semibold
21        ↳ text-secondary">
22        <!-- JS Populated -->
23      </tbody>
24    </table>
  </div>
</div>
```

Listing 8.51: ซอร์สโค้ดจากไฟล์ frontend/views/equipments.ejs: Equipment Table

การนำเข้าสคริปต์ (Script Imports)

```
1 <script src="/js/config.js"></script>
2 <script src="/js/auth.js"></script>
3 <script src="/js/equipments.js"></script>
```

Listing 8.52: ซอร์สโค้ดจากไฟล์ frontend/views/equipments.ejs: Scripts

คำอธิบาย:

1. equipments.js: ไฟล์ JavaScript หลักที่รับผิดชอบการดึงข้อมูลอุปกรณ์จาก API มาแสดงในตาราง และจัดการการส่งฟอร์มเพิ่มอุปกรณ์

8.3.11 ไฟล์ frontend/views/trainers.ejs

ภาพรวม (Overview)

ไฟล์ frontend/views/trainers.ejs คือหน้าจอสำหรับผู้ดูแลระบบ (Admin) เพื่อใช้ในการบริหารจัดการข้อมูลเทรนเนอร์ (Trainer Management) หน้าจอนี้ช่วยให้แอดมินสามารถเพิ่มรายชื่อเทรนเนอร์ใหม่ ระบุความเชี่ยวชาญ (Specialty) และระดับความสามารถ (Level) รวมถึงดูรายชื่อเทรนเนอร์ทั้งหมดที่มีในระบบได้

การนำทางและสถานะเมนู (Navigation State)

ในหน้านี้ เมนู "Trainers" ใน Sidebar จะถูกเน้น (Active State) เพื่อให้ผู้ใช้ทราบตำแหน่งปัจจุบันของตนเองในระบบ


```

1 <!-- Sidebar Navigation Item for Trainers -->
2 <li>
3   <a href="/admin/trainers" class="flex items-center gap-4 px-6 py-4 bg-secondary
4     border-l-4 border-accent text-white font-bold uppercase tracking-wide transition-all">
5     <i data-lucide="dumbbell" class="w-5 h-5"></i>
6     <span>Trainers</span>
7   </a>
8 </li>

```

Listing 8.53: ซอร์สโค้ดจากไฟล์ frontend/views/trainers.ejs: Sidebar Navigation

ฟอร์มเพิ่มเทรนเนอร์ใหม่ (Add Trainer Form)

ฟอร์มสำหรับเพิ่มข้อมูลเทรนเนอร์ ประกอบด้วยชื่อ ความเชี่ยวชาญ ระดับ และเบอร์โทรศัพท์

```

1 <!-- Add Trainer Form -->
2 <div class="bg-white border-2 border-primary p-6 mb-8 shadow-[4px_4px_0px_0px_#0C2C55]">
3   <h3 class="text-xl font-bold uppercase mb-4 flex items-center gap-2">
4     <i data-lucide="user-plus" class="w-5 h-5 text-accent"></i> Add New Trainer
5   </h3>
6   <form id="trainer-form" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-5 gap-4">
7     <!-- Name Input -->
8     <input id="name" placeholder="Trainer Name" required class="border-2 border-primary p-3
9       font-bold focus:outline-none focus:border-accent placeholder-gray-400">
10
11     <!-- Specialty Input -->
12     <input id="specialty" placeholder="Specialty (e.g. Yoga, Cardio)" class="border-2
13       border-primary p-3 font-bold focus:outline-none focus:border-accent placeholder-gray-400">
14
15     <!-- Level Select -->
16     <select id="trainerLevel" required class="border-2 border-primary p-3 font-bold
17       focus:outline-none focus:border-accent bg-white">
18       <option value="Junior">Junior</option>
19       <option value="Senior">Senior</option>
20       <option value="Master">Master</option>
21     </select>
22
23     <!-- Phone Input -->
24     <input id="phone" placeholder="Phone Number" class="border-2 border-primary p-3 font-bold
25       focus:outline-none focus:border-accent placeholder-gray-400">
26
27     <!-- Submit Button -->
28     <button type="submit" class="bg-primary text-contrast font-bold uppercase hover:bg-secondary
29       hover:shadow-[2px_2px_0px_0px_#629FAD] transition-all active:translate-y-1
30       active:shadow-none border-2 border-transparent h-full flex items-center justify-center
31       gap-2">
32       <i data-lucide="save" class="w-4 h-4"></i> Save Trainer
33     </button>
34   </form>
35 </div>

```

Listing 8.54: ซอร์สโค้ดจากไฟล์ frontend/views/trainers.ejs: Add Trainer Form

คำอธิบาย:

1. **Grid Layout:** ใช้ `lg:grid-cols-5` เพื่อจัดเรียง Input Fields และปุ่ม Submit ให้อยู่ในแถวเดียวกันบนหน้าจอขนาดใหญ่
2. **Trainer Level:** มี Dropdown ให้เลือกตำแหน่งของเทรนเนอร์ (Junior, Senior, Master) ซึ่งอาจมีผลต่อราคาหรือสิทธิในการสอน

ตารางรายชื่อเทรนเนอร์ (Trainers Roster)

ส่วนแสดงรายชื่อเทรนเนอร์ทั้งหมดในรูปแบบตาราง

```

1 <!-- Trainers List -->
2 <div class="bg-white border-2 border-primary shadow-[8px_8px_0px_0px_#296374]">
3   <div class="bg-secondary p-4 border-b-2 border-primary">
4     <h3 class="text-xl font-bold text-white uppercase tracking-wider flex items-center gap-2">

```



```

5     <i data-lucide="dumbbell" class="w-5 h-5 text-accent"></i> Trainers Roster
6   </h3>
7 </div>
8 <div class="overflow-x-auto">
9   <table class="w-full text-left border-collapse">
10    <thead>
11      <tr class="bg-gray-100 border-b-2 border-primary text-primary text-sm uppercase
12      ↳ tracking-wider">
13        <th class="p-4 font-extrabold">ID</th>
14        <th class="p-4 font-extrabold">Name</th>
15        <th class="p-4 font-extrabold">Specialty</th>
16        <th class="p-4 font-extrabold">Level</th>
17        <th class="p-4 font-extrabold">Phone</th>
18        <th class="p-4 font-extrabold text-center">Actions</th>
19      </tr>
20    </thead>
21    <tbody id="trainers-table" class="divide-y-2 divide-gray-100 text-sm font-semibold
22    ↳ text-secondary">
23      <!-- JS Populated -->
24    </tbody>
25  </table>
26 </div>
27 </div>

```

Listing 8.55: ซอร์สโค้ดจากไฟล์ frontend/views/trainers.ejs: Trainers Table

การนำเข้าสคริปต์ (Script Imports)

```

1 <script src="/js/config.js"></script>
2 <script src="/js/auth.js"></script>
3 <script src="/js/trainers.js"></script>

```

Listing 8.56: ซอร์สโค้ดจากไฟล์ frontend/views/trainers.ejs: Scripts

คำอธิบาย:

1. trainers.js: ไฟล์ JavaScript สำหรับจัดการการดึงข้อมูลและบันทึกข้อมูลเทรนเนอร์

8.3.12 ไฟล์ frontend/views/sessions.ejs

หน้าเว็บสำหรับผู้ดูแลระบบ (Admin) เพื่อจัดการการจองเซสชันการฝึกซ้อม (Sessions)

ส่วนหัวและสไตล์ (Head and Styles)

กำหนด Meta tags, นำเข้า Tailwind CSS, Google Fonts, Lucide Icons และกำหนดสไตล์เพิ่มเติม

```

1 <!doctype html>
2 <html lang="th">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>GymBro • Sessions</title>
7
8   <!-- Tailwind CSS via CDN -->
9   <script src="https://cdn.tailwindcss.com"></script>
10  <script>
11    tailwind.config = {
12      theme: {
13        extend: {
14          colors: {
15            primary: '#0C2C55',
16            secondary: '#296374',
17            accent: '#629FAD',
18            contrast: '#EDEDCE',
19          },
20          fontFamily: {
21            sans: ['Inter', 'sans-serif'],
22          },
23          borderRadius: {
24            DEFAULT: '0px',

```

```

25         'none': '0px',
26         'sm': '0px',
27         'md': '0px',
28         'lg': '0px',
29         'xl': '0px',
30         '2xl': '0px',
31         '3xl': '0px',
32         'full': '0px',
33     }
34 }
35 }
36 }
37 </script>
38
39 <!-- Google Fonts -->
40 <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;800&display=swap"
    ↪ rel="stylesheet">
41
42 <!-- Lucide Icons -->
43 <script src="https://unpkg.com/lucide@latest"></script>
44
45 <style>
46     body { font-family: 'Inter', sans-serif; }
47     ::-webkit-scrollbar { width: 8px; }
48     ::-webkit-scrollbar-track { background: #EDEDCE; }
49     ::-webkit-scrollbar-thumb { background: #296374; }
50     ::-webkit-scrollbar-thumb:hover { background: #0C2C55; }
51 </style>
52 </head>
53 <body class="bg-contrast text-primary min-h-screen flex overflow-hidden">

```

Listing 8.57: ซอร์สโค้ดจากไฟล์ frontend/views/sessions.ejs: Head Section

แถบนำทาง (Sidebar Navigation)

แถบเมนูด้านซ้ายสำหรับผู้ดูแลระบบ ซึ่งมีการเน้นเมนู "Sessions" เพื่อแสดงสถานะหน้าปัจจุบัน

```

1 <!-- Sidebar -->
2 <aside class="w-64 bg-primary text-contrast flex-shrink-0 flex flex-col border-r-4
    ↪ border-accent hidden md:flex">
3     <div class="h-20 flex items-center justify-center border-b border-secondary">
4         <h1 class="text-3xl font-extrabold tracking-tighter uppercase italic">Gym<span
    ↪ class="text-accent">Bro</span></h1>
5     </div>
6     <nav class="flex-1 overflow-y-auto py-6">
7         <ul class="space-y-1">
8             <!-- Other menu items omitted for brevity -->
9             <li>
10                 <a href="/admin/sessions" class="flex items-center gap-4 px-6 py-4 bg-secondary
    ↪ border-l-4 border-accent text-white font-bold uppercase tracking-wide transition-all">
11                     <i data-lucide="calendar-check" class="w-5 h-5"></i>
12                     <span>Sessions</span>
13                 </a>
14             </li>
15             <!-- Other menu items omitted for brevity -->
16         </ul>
17     </nav>
18     <!-- Admin Profile Section -->
19 </aside>

```

Listing 8.58: ซอร์สโค้ดจากไฟล์ frontend/views/sessions.ejs: Sidebar

ส่วนเนื้อหาหลักและฟอร์มการจอง (Main Content & Booking Form)

ส่วนแสดงหัวข้อหน้าและฟอร์มสำหรับสร้างการจองเซสชันใหม่ โดยมีการเลือก Customer, Trainer, Equipment และ เวลา

```

1 <!-- Content Area -->
2 <div class="flex-1 overflow-y-auto p-4 md:p-8 bg-contrast relative">
3
4     <div class="mb-8 border-b-4 border-primary pb-4">

```

```

5     <h2 class="text-4xl font-extrabold text-primary uppercase tracking-tighter">Training
    ↪ Sessions</h2>
6     <p class="text-secondary font-medium mt-1">Schedule and manage training sessions.</p>
    </div>
7
8
9     <!-- Add Session Form -->
10    <div class="bg-white border-2 border-primary p-6 mb-8 shadow-[4px_4px_0px_0px_#0C2C55]">
11      <h3 class="text-xl font-bold uppercase mb-4 flex items-center gap-2">
12        <i data-lucide="calendar-plus" class="w-5 h-5 text-accent"></i> Book Session
13      </h3>
14      <form id="session-form" class="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-5 gap-4">
15        <select id="customer" required class="border-2 border-primary p-3 font-bold
    ↪ focus:outline-none focus:border-accent bg-white">
16          <option value="" disabled selected>Select Customer</option>
17        </select>
18        <select id="trainer" required class="border-2 border-primary p-3 font-bold
    ↪ focus:outline-none focus:border-accent bg-white">
19          <option value="" disabled selected>Select Trainer</option>
20        </select>
21        <select id="equipment" required class="border-2 border-primary p-3 font-bold
    ↪ focus:outline-none focus:border-accent bg-white">
22          <option value="" disabled selected>Select Equipment</option>
23        </select>
24        <input id="scheduled_at" type="datetime-local" required class="border-2 border-primary
    ↪ p-3 font-bold focus:outline-none focus:border-accent">
25
26        <button type="submit" class="bg-primary text-contrast font-bold uppercase
    ↪ hover:bg-secondary hover:shadow-[2px_2px_0px_0px_#629FAD] transition-all
    ↪ active:translate-y-1 active:shadow-none border-2 border-transparent h-full flex
    ↪ items-center justify-center gap-2">
27          <i data-lucide="check-circle" class="w-4 h-4"></i> Book Now
28        </button>
29      </form>
30    </div>

```

Listing 8.59: ซอร์สโค้ดจากไฟล์ frontend/views/sessions.ejs: Booking Form

คำอธิบาย:

1. **Form Controls:** ใช้ `select` สำหรับเลือกข้อมูล Customer, Trainer และ Equipment ซึ่งข้อมูลใน Dropdown จะถูกดึงมาจาก API
2. **Date Selection:** ใช้ `input type="datetime-local"` เพื่อให้ผู้ดูแลระบบสามารถระบุวันและเวลาที่ต้องการจองได้ในคราวเดียว
3. **Layout:** ใช้ `grid-cols-5` จัดเรียงองค์ประกอบให้สวยงามบนหน้าจอขนาดใหญ่

ตารางรายการเซสชัน (Sessions List Table)

ส่วนแสดงรายการเซสชันที่ถูกจองทั้งหมด

```

1     <!-- Sessions List -->
2     <div class="bg-white border-2 border-primary shadow-[8px_8px_0px_0px_#296374]">
3       <div class="bg-secondary p-4 border-b-2 border-primary">
4         <h3 class="text-xl font-bold text-white uppercase tracking-wider flex items-center
    ↪ gap-2">
5           <i data-lucide="list-checks" class="w-5 h-5 text-accent"></i> Scheduled Sessions
6         </h3>
7       </div>
8       <div class="overflow-x-auto">
9         <table class="w-full text-left border-collapse">
10          <thead>
11            <tr class="bg-gray-100 border-b-2 border-primary text-primary text-sm uppercase
    ↪ tracking-wider">
12              <th class="p-4 font-extrabold">Customer</th>
13              <th class="p-4 font-extrabold">Trainer</th>
14              <th class="p-4 font-extrabold">Equipment</th>
15              <th class="p-4 font-extrabold">Date & Time</th>
16              <th class="p-4 font-extrabold text-center">Actions</th>
17            </tr>
18          </thead>
19          <tbody id="sessions-table" class="divide-y-2 divide-gray-100 text-sm font-semibold
    ↪ text-secondary">

```

```

20         <!-- JS Populated -->
21     </tbody>
22 </table>
23 </div>
24 </div>

```

Listing 8.60: ซอร์สโค้ดจากไฟล์ frontend/views/sessions.ejs: Sessions Table

สคริปต์ (Scripts)

การนำเข้าไฟล์ JavaScript ที่จำเป็น รวมถึง 'sessions.js' สำหรับจัดการ logic ของหน้า

```

1  <script>
2      lucide.createIcons();
3      // Mobile Sidebar Logic (omitted)
4  </script>
5  <script src="/js/config.js"></script>
6  <script src="/js/auth.js"></script>
7  <script src="/js/sessions.js"></script>
8 </body>
9 </html>

```

Listing 8.61: ซอร์สโค้ดจากไฟล์ frontend/views/sessions.ejs: Scripts

8.3.13 ไฟล์ frontend/views/logs.ejs

หน้าเว็บสำหรับผู้ดูแลระบบ (Admin) เพื่อดูรายงานและบันทึกการใช้งานระบบ (Reports & Logs) โดยมีการแบ่งข้อมูลออกเป็น 3 ส่วนหลัก ได้แก่ System Logs, Customer Report และ Trainer Report

ส่วนหัวและสไตล์ (Head and Styles)

กำหนด Meta tags, นำเข้า Tailwind CSS, Google Fonts, Lucide Icons และกำหนดสไตล์สำหรับ Tab Switching

```

1  <!doctype html>
2  <html lang="th">
3  <head>
4      <meta charset="utf-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1">
6      <title>GymBro • Reports & Logs</title>
7
8      <!-- Tailwind CSS via CDN -->
9      <script src="https://cdn.tailwindcss.com"></script>
10     <script>
11         tailwind.config = {
12             theme: {
13                 extend: {
14                     colors: {
15                         primary: '#0C2C55',
16                         secondary: '#296374',
17                         accent: '#629FAD',
18                         contrast: '#EDEDCE',
19                     },
20                     fontFamily: {
21                         sans: ['Inter', 'sans-serif'],
22                     },
23                     borderRadius: {
24                         DEFAULT: '0px',
25                     }
26                 }
27             }
28         }
29     </script>
30
31     <!-- Google Fonts & Icons -->
32     <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;800&display=swap"
33           rel="stylesheet">
34     <script src="https://unpkg.com/lucide@latest"></script>
35     <style>

```

```

36 body { font-family: 'Inter', sans-serif; }
37 ::-webkit-scrollbar { width: 8px; }
38 ::-webkit-scrollbar-track { background: #EDEDCE; }
39 ::-webkit-scrollbar-thumb { background: #296374; }
40 ::-webkit-scrollbar-thumb:hover { background: #0C2C55; }
41 .tab-active {
42   background-color: #296374;
43   color: white;
44   border-bottom: 4px solid #629FAD;
45 }
46 .tab-inactive {
47   background-color: #EDEDCE;
48   color: #0C2C55;
49   border-bottom: 4px solid transparent;
50 }
51 .tab-inactive:hover {
52   background-color: #e0e0c0;
53 }
54 </style>
55 </head>
56 <body class="bg-contrast text-primary min-h-screen flex overflow-hidden">

```

Listing 8.62: ซอร์สโค้ดจากไฟล์ frontend/views/logs.ejs: Head Section

แถบนำทาง (Sidebar Navigation)

แถบเมนูด้านซ้ายสำหรับผู้ดูแลระบบ ซึ่งมีการเน้นเมนู "Reports & Logs"

```

1 <!-- Sidebar -->
2 <aside class="w-64 bg-primary text-contrast flex-shrink-0 flex flex-col border-r-4
3   ↳ border-accent hidden md:flex">
4   <!-- Header omitted -->
5   <nav class="flex-1 overflow-y-auto py-6">
6     <ul class="space-y-1">
7       <!-- Other menu items omitted -->
8       <li>
9         <a href="/admin/logs" class="flex items-center gap-4 px-6 py-4 bg-secondary border-l-4
10          ↳ border-accent text-white font-bold uppercase tracking-wide transition-all">
11           <i data-lucide="file-text" class="w-5 h-5"></i>
12           <span>Reports & Logs</span>
13         </a>
14       </li>
15       <!-- Logout omitted -->
16     </ul>
17   </nav>
18   <!-- Profile omitted -->
19 </aside>

```

Listing 8.63: ซอร์สโค้ดจากไฟล์ frontend/views/logs.ejs: Sidebar

ส่วนควบคุมแท็บและตัวกรอง (Tabs & Filters)

ส่วนสำหรับเลือกประเภทรายงานที่ต้องการดู (System Logs, Customer Report, Trainer Report) และตัวกรองวันที่

```

1 <!-- Content Area -->
2 <div class="flex-1 overflow-y-auto p-4 md:p-8 bg-contrast relative">
3
4   <div class="mb-6 border-b-4 border-primary pb-4">
5     <h2 class="text-4xl font-extrabold text-primary uppercase tracking-tighter">Reports &
6     ↳ System Logs</h2>
7     <p class="text-secondary font-medium mt-1">View usage reports and system activity.</p>
8   </div>
9
10  <!-- Tabs -->
11  <div class="flex flex-col md:flex-row justify-between items-center mb-6 gap-4 border-b-2
12   ↳ border-secondary/20 pb-2">
13    <div class="flex">
14      <button id="tab-logs" onclick="switchTab('logs')" class="px-6 py-3 font-bold
15      ↳ uppercase tracking-wide transition-all tab-active">
16        System Logs
17      </button>

```

```

15     <button id="tab-customer" onclick="switchTab('customer')" class="px-6 py-3 font-bold
    ↳ uppercase tracking-wide transition-all tab-inactive">
16         Customer Report
17     </button>
18     <button id="tab-trainer" onclick="switchTab('trainer')" class="px-6 py-3 font-bold
    ↳ uppercase tracking-wide transition-all tab-inactive">
19         Trainer Report
20     </button>
21 </div>
22
23 <!-- Date Filter -->
24 <div class="flex items-center gap-2">
25     <label for="filter-date" class="font-bold text-sm uppercase text-primary">Filter
    ↳ Date:</label>
26     <input type="date" id="filter-date" class="border-2 border-primary p-2 font-bold
    ↳ text-sm bg-white focus:outline-none focus:border-accent">
27     <button onclick="applyDateFilter()" class="bg-primary text-contrast px-4 py-2
    ↳ font-bold uppercase text-xs hover:bg-secondary transition-colors">
28         Apply
29     </button>
30     <button onclick="clearDateFilter()" class="text-xs text-red-500 font-bold uppercase
    ↳ hover:underline">
31         Clear
32     </button>
33 </div>
34 </div>

```

Listing 8.64: ซอร์สโค้ดจากไฟล์ frontend/views/logs.ejs: Tabs Control

คำอธิบาย:

1. **Tab Controls:** ใช้ button พร้อม Event Listener onclick เพื่อสลับการแสดงผลระหว่าง Log และ Report ประเภทต่างๆ
2. **Visual Feedback:** คลาส tab-active และ tab-inactive ถูกใช้เพื่อเปลี่ยนสีพื้นหลังและเส้นขอบล่างของปุ่ม เพื่อแสดงว่าผู้ใช้งานกำลังดู Tab ใดอยู่
3. **Date Filter:** Input type="date" ช่วยให้ผู้ใช้ดูและระบบสามารถกรองข้อมูลตามวันที่ที่สนใจได้

ส่วนแสดงผลรายงาน (Report Sections)

พื้นที่แสดงตารางข้อมูลตามแท็บที่เลือก โดยแต่ละส่วนจะมี ID ที่ไม่ซ้ำกันเพื่อใช้ในการซ่อน/แสดง

```

1 <!-- System Logs Section -->
2 <div id="section-logs" class="bg-white border-2 border-primary
    ↳ shadow-[8px_8px_0px_0px_#296374]">
3 <!-- Header omitted -->
4 <div class="overflow-x-auto">
5     <table class="w-full text-left border-collapse">
6         <thead>
7             <tr class="bg-gray-100 border-b-2 border-primary text-primary text-sm uppercase
    ↳ tracking-wider">
8                 <th class="p-4 font-extrabold w-48">Timestamp</th>
9                 <th class="p-4 font-extrabold w-48">Action</th>
10                <th class="p-4 font-extrabold">Details</th>
11            </tr>
12        </thead>
13        <tbody id="logs-table" class="divide-y-2 divide-gray-100 text-sm font-semibold
    ↳ text-secondary">
14            <tr><td colspan="3" class="p-4 text-center">Loading logs...</td></tr>
15        </tbody>
16    </table>
17 </div>
18 </div>
19
20 <!-- Customer Report Section (Hidden by default) -->
21 <div id="section-customer" class="hidden bg-white border-2 border-primary
    ↳ shadow-[8px_8px_0px_0px_#296374]">
22 <!-- Table Structure similar to logs but with Customer columns -->
23 </div>
24
25 <!-- Trainer Report Section (Hidden by default) -->

```

```

26 <div id="section-trainer" class="hidden bg-white border-2 border-primary
    ↳ shadow-[8px_8px_0px_0px_#296374]">
27 <!-- Table Structure similar to logs but with Trainer columns -->
28 </div>

```

Listing 8.65: ซอร์สโค้ดจากไฟล์ frontend/views/logs.ejs: Report Tables

สคริปต์และการจัดการแท็บ (Scripts & Tab Logic)

ส่วนจัดการ Logic การสลับแท็บและการโหลดข้อมูล

```

1 <script>
2   lucide.createIcons();
3   // Mobile Sidebar Logic (omitted)
4
5   // Tab Switching Logic
6   function switchTab(tabName) {
7     // Hide all sections
8     document.getElementById('section-logs').classList.add('hidden');
9     document.getElementById('section-customer').classList.add('hidden');
10    document.getElementById('section-trainer').classList.add('hidden');
11
12    // Reset tab styles (omitted for brevity)
13
14    // Show selected section
15    document.getElementById(`section-${tabName}`).classList.remove('hidden');
16
17    // Activate selected tab (omitted for brevity)
18
19    // Load data if needed
20    if(tabName === 'logs') loadLogs();
21    if(tabName === 'customer') loadCustomerReport();
22    if(tabName === 'trainer') loadTrainerReport();
23  }
24 </script>
25 <script src="/js/config.js"></script>
26 <script src="/js/auth.js"></script>
27 <script src="/js/logs.js"></script>
28 </body>
29 </html>

```

Listing 8.66: ซอร์สโค้ดจากไฟล์ frontend/views/logs.ejs: Tab Logic

8.3.14 ไฟล์ frontend/views/user/dashboard.ejs

หน้าแดชบอร์ดสำหรับสมาชิกทั่วไป (User) เพื่อดูตารางการจองและทำการจองเซสชันใหม่

ส่วนหัวและ Modal การจอง (Head & Booking Modal)

ส่วนกำหนด Meta tags และ Modal Form สำหรับการจองเซสชันใหม่ (ซ่อนอยู่โดยเริ่มต้น)

```

1 <!doctype html>
2 <html lang="th">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>GymBro • User Dashboard</title>
7
8   <script src="https://cdn.tailwindcss.com"></script>
9   <!-- Booking Modal -->
10  <div id="booking-modal" class="fixed inset-0 bg-black/50 z-50 hidden flex items-center
    ↳ justify-center p-4">
11    <div class="bg-white border-4 border-primary shadow-[8px_8px_0px_0px_#296374] w-full
    ↳ max-w-md max-h-[90vh] overflow-y-auto">
12      <div class="bg-primary text-contrast p-4 flex justify-between items-center border-b-4
    ↳ border-accent">
13        <h3 class="text-xl font-extrabold uppercase tracking-tighter">Book New Session</h3>
14        <button id="close-booking-modal" class="hover:text-accent"><i data-lucide="x"
    ↳ class="w-6 h-6"></i></button>
15      </div>

```

```

16     <div class="p-6">
17       <form id="booking-form" class="space-y-4">
18         <!-- Form Inputs: Date, Time, Duration, Trainer, Equipment -->
19         <div>
20           <label class="block text-sm font-bold uppercase mb-1">Date</label>
21           <input type="date" id="book-date" required class="w-full p-3 border-2
    ↳ border-primary bg-contrast focus:outline-none focus:border-accent font-bold">
22         </div>
23         <!-- Other inputs omitted for brevity -->
24
25         <div class="pt-4 flex justify-end gap-4">
26           <button type="button" id="cancel-booking" class="px-6 py-3 font-bold
    ↳ uppercase text-primary hover:bg-gray-100 transition-colors">Cancel</button>
27           <button type="submit" class="bg-primary text-contrast px-6 py-3 font-bold
    ↳ uppercase hover:bg-secondary hover:shadow-[4px_4px_0px_0px_#629FAD] transition-all
    ↳ active:translate-y-1 active:shadow-none border-2 border-transparent">
28             Confirm Booking
29           </button>
30         </div>
31       </form>
32     </div>
33 </div>
34 </div>
35
36 <script>
37   tailwind.config = {
38     theme: {
39       extend: {
40         colors: {
41           primary: '#0C2C55',
42           secondary: '#296374',
43           accent: '#629FAD',
44           contrast: '#EDEDCE',
45         },
46         fontFamily: {
47           sans: ['Inter', 'sans-serif'],
48         },
49         borderRadius: {
50           DEFAULT: '0px',
51         }
52       }
53     }
54   }
55 </script>
56 <!-- Fonts & Icons omitted -->
57 </head>
58 <body class="bg-contrast text-primary min-h-screen flex overflow-hidden">

```

Listing 8.67: ซอร์สโค้ดจากไฟล์ frontend/views/user/dashboard.ejs: Head & Modal

คำอธิบาย:

1. **Modal Dialog:** ใช้ div id="booking-modal" ที่มีคลาส hidden เพื่อซ่อนฟอร์มการจองไว้ในตอนแรก และจะแสดงเมื่อผู้ใช้กดปุ่ม "Book Session"
2. **Overlay:** พื้นหลังสีดำจางๆ (bg-black/50) ช่วยเน้นให้ Modal โดดเด่นขึ้นและป้องกันการคลิกพื้นหลังโดยไม่ตั้งใจ
3. **Booking Form:** ฟอร์มภายใน Modal ใช้สำหรับส่งข้อมูลการจองไปยัง API /api/sessions

แถบนำทาง (Sidebar Navigation)

แถบเมนูด้านซ้ายสำหรับสมาชิกทั่วไป ซึ่งมีเมนู "Schedule" และ "My Profile"

```

1 <!-- Sidebar -->
2 <aside class="w-64 bg-primary text-contrast flex-shrink-0 flex flex-col border-r-4
    ↳ border-accent hidden md:flex">
3   <!-- Header omitted -->
4
5   <nav class="flex-1 overflow-y-auto py-6">
6     <ul class="space-y-1">
7       <li>

```



```

8       <a href="/user/dashboard" class="flex items-center gap-4 px-6 py-4 bg-secondary
9       border-l-4 border-accent text-white font-bold uppercase tracking-wide transition-all">
10        <i data-lucide="calendar" class="w-5 h-5"></i>
11        <span>Schedule</span>
12      </a>
13    </li>
14    <li>
15      <a href="/user/profile" class="flex items-center gap-4 px-6 py-4 text-gray-300
16      ↪ hover:bg-secondary hover:text-white font-semibold uppercase tracking-wide transition-all
17      ↪ group">
18        <i data-lucide="user" class="w-5 h-5 group-hover:text-accent transition-colors"></i>
19        <span>My Profile</span>
20      </a>
21    </li>
22    <!-- Logout omitted -->
23  </ul>
24</nav>
25
26<!-- User Profile Footer -->
27<div class="p-4 bg-secondary border-t border-primary">
28  <div class="flex items-center gap-3">
29    <div class="w-10 h-10 bg-accent flex items-center justify-center text-primary font-bold
30    ↪ text-lg rounded-none">
31      ME
32    </div>
33    <div>
34      <p class="text-sm font-bold text-white user-name-display">Loading...</p>
35      <p class="text-xs text-accent user-role-display">Member</p>
36    </div>
37  </div>
38</div>
39</aside>

```

Listing 8.68: ซอร์สโค้ดจากไฟล์ frontend/views/user/dashboard.ejs: Sidebar

ส่วนเนื้อหาหลักและตารางการจอง (Main Content & Schedule Table)

ส่วนแสดงหัวข้อหน้า, ปุ่มเปิด Modal การจอง และตารางแสดงรายการจองทั้งหมด

```

1  <!-- Content Area -->
2  <div class="flex-1 overflow-y-auto p-4 md:p-8 bg-contrast relative">
3
4    <!-- Page Header -->
5    <div class="mb-8 border-b-4 border-primary pb-4 flex flex-col md:flex-row md:items-end
6    ↪ justify-between gap-4">
7      <div>
8        <h2 class="text-4xl font-extrabold text-primary uppercase tracking-tighter">
9          Gym Schedule
10        </h2>
11        <p class="text-secondary font-medium mt-1">
12          Check availability and your upcoming sessions.
13        </p>
14      </div>
15      <button id="btn-book-session" class="bg-primary text-contrast px-6 py-3 font-bold
16      ↪ uppercase
17      ↪ hover:bg-secondary hover:shadow-[4px_4px_0px_0px_#629FAD] transition-all
18      ↪ active:translate-y-1 active:shadow-none border-2 border-transparent flex items-center
19      ↪ gap-2">
20        <i data-lucide="plus-circle" class="w-5 h-5"></i>
21        Book Session
22      </button>
23    </div>
24
25    <!-- Schedule Table -->
26    <div class="bg-white border-2 border-primary shadow-[8px_8px_0px_0px_#296374]">
27      <div class="bg-secondary p-4 flex justify-between items-center border-b-2
28      ↪ border-primary">
29        <h3 class="text-xl font-bold text-white uppercase tracking-wider flex items-center
30        ↪ gap-2">
31          <i data-lucide="calendar-days" class="w-5 h-5 text-accent"></i>
32          All Bookings
33        </h3>

```

```

29     <span class="text-xs font-bold text-accent bg-primary px-2 py-1 uppercase">Read
    ↳ Only</span>
30   </div>
31
32   <div class="p-0 overflow-x-auto">
33     <table class="w-full text-left border-collapse">
34       <thead>
35         <tr class="bg-gray-100 border-b-2 border-primary text-primary text-sm uppercase
    ↳ tracking-wider">
36           <th class="p-4 font-extrabold">Time</th>
37           <th class="p-4 font-extrabold">Activity/Equipment</th>
38           <th class="p-4 font-extrabold">Trainer</th>
39           <th class="p-4 font-extrabold text-center">Status</th>
40         </tr>
41       </thead>
42       <tbody id="schedule-table-body" class="divide-y-2 divide-gray-100 text-sm
    ↳ font-semibold text-secondary">
43         <tr>
44           <td colspan="4" class="p-4 text-center">Loading schedule...</td>
45         </tr>
46       </tbody>
47     </table>
48   </div>
49 </div>
50
51 </div>

```

Listing 8.69: ซอร์สโค้ดจากไฟล์ frontend/views/user/dashboard.ejs: Content

สคริปต์ (Scripts)

การนำเข้าไฟล์ JavaScript ที่จำเป็น รวมถึง 'user/dashboard.js' สำหรับจัดการ logic ของหน้า

```

1  <script>
2    lucide.createIcons();
3
4    // Sidebar Logic (omitted)
5  </script>
6
7  <script src="/js/config.js"></script>
8  <script src="/js/auth.js"></script>
9  <script src="/js/user/dashboard.js"></script>
10 </body>
11 </html>

```

Listing 8.70: ซอร์สโค้ดจากไฟล์ frontend/views/user/dashboard.ejs: Scripts

8.3.15 ไฟล์ frontend/views/user/profile.ejs

หน้าแสดงข้อมูลส่วนตัวของสมาชิก (User Profile) แสดงรายละเอียดเกี่ยวกับสมาชิก เช่น ชื่อ, อีเมล, ประเภทสมาชิก และสถานะ

ส่วนหัวและสไตล์ (Head and Styles)

กำหนด Meta tags, นำเข้า Tailwind CSS, Google Fonts, Lucide Icons และกำหนดสไตล์เพิ่มเติม

```

1 <!doctype html>
2 <html lang="th">
3 <head>
4   <meta charset="utf-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>GymBro • My Profile</title>
7
8   <script src="https://cdn.tailwindcss.com"></script>
9   <script>
10     tailwind.config = {
11       theme: {
12         extend: {
13           colors: {
14             primary: '#0C2C55',

```

```

15     secondary: '#296374',
16     accent: '#629FAD',
17     contrast: '#EDEDCE',
18   },
19   fontFamily: {
20     sans: ['Inter', 'sans-serif'],
21   },
22   borderRadius: {
23     DEFAULT: '0px',
24   }
25 }
26 }
27 }
28 </script>
29
30 <!-- Fonts & Icons omitted -->
31 </head>
32 <body class="bg-contrast text-primary min-h-screen flex overflow-hidden">

```

Listing 8.71: ซอร์สโค้ดจากไฟล์ frontend/views/user/profile.ejs: Head Section

แถบนำทาง (Sidebar Navigation)

แถบเมนูด้านซ้ายสำหรับสมาชิกทั่วไป ซึ่งมีเมนู "My Profile" ที่ถูกเน้น

```

1 <!-- Sidebar -->
2 <aside class="w-64 bg-primary text-contrast flex-shrink-0 flex flex-col border-r-4
   ↳ border-accent hidden md:flex">
3   <!-- Header omitted -->
4
5   <nav class="flex-1 overflow-y-auto py-6">
6     <ul class="space-y-1">
7       <li>
8         <a href="/user/dashboard" class="flex items-center gap-4 px-6 py-4 text-gray-300
   ↳ hover:bg-secondary hover:text-white font-semibold uppercase tracking-wide transition-all
   ↳ group">
9           <i data-lucide="calendar" class="w-5 h-5 group-hover:text-accent
   ↳ transition-colors"></i>
10          <span>Schedule</span>
11        </a>
12      </li>
13      <li>
14        <a href="/user/profile" class="flex items-center gap-4 px-6 py-4 bg-secondary
   ↳ border-l-4 border-accent text-white font-bold uppercase tracking-wide transition-all">
15          <i data-lucide="user" class="w-5 h-5"></i>
16          <span>My Profile</span>
17        </a>
18      </li>
19      <!-- Logout omitted -->
20    </ul>
21  </nav>
22  <!-- User Profile Footer omitted -->
23 </aside>

```

Listing 8.72: ซอร์สโค้ดจากไฟล์ frontend/views/user/profile.ejs: Sidebar

ส่วนเนื้อหาหลักและข้อมูลส่วนตัว (Main Content & Profile Details)

ส่วนแสดงการ์ดข้อมูลส่วนตัวสมาชิก ตกแต่งด้วย Tailwind CSS

```

1 <!-- Content Area -->
2 <div class="flex-1 overflow-y-auto p-4 md:p-8 bg-contrast relative">
3
4   <!-- Page Header -->
5   <div class="mb-8 border-b-4 border-primary pb-4 flex flex-col md:flex-row md:items-end
   ↳ justify-between gap-4">
6     <div>
7       <h2 class="text-4xl font-extrabold text-primary uppercase tracking-tighter">
8         My Profile
9       </h2>
10      <p class="text-secondary font-medium mt-1">

```

```

11         Your membership details and information.
12     </p>
13 </div>
14 </div>
15
16 <!-- Profile Card -->
17 <div class="max-w-3xl mx-auto">
18     <div class="bg-white border-4 border-secondary shadow-[8px_8px_0px_0px_#0C2C55] p-8
19 ↪ relative overflow-hidden">
20
21         <!-- Decoration -->
22         <div class="absolute -right-12 -top-12 w-40 h-40 bg-contrast rotate-45 border-4
23 ↪ border-primary"></div>
24
25         <div class="relative z-10 flex flex-col md:flex-row gap-8 items-start">
26
27             <!-- Avatar Section -->
28             <div class="flex-shrink-0 flex flex-col items-center gap-4">
29                 <div class="w-32 h-32 bg-secondary border-4 border-primary flex items-center
30 ↪ justify-center text-white text-4xl font-black shadow-[4px_4px_0px_0px_#629FAD]">
31                     <i data-lucide="user" class="w-16 h-16"></i>
32                 </div>
33                 <span class="bg-primary text-accent px-3 py-1 text-xs font-black uppercase
34 ↪ tracking-widest border border-accent w-full text-center">
35                     Active Member
36                 </span>
37             </div>
38
39             <!-- Details Section -->
40             <div class="flex-1 space-y-6 w-full">
41
42                 <div class="border-b-2 border-dashed border-gray-300 pb-4">
43                     <label class="block text-xs font-bold uppercase text-secondary mb-1">Full
44 ↪ Name</label>
45                     <h3 class="text-2xl font-black text-primary" id="profile-name">Loading...</h3>
46                 </div>
47
48                 <div class="grid grid-cols-1 md:grid-cols-2 gap-6">
49                     <div>
50                         <label class="block text-xs font-bold uppercase text-secondary mb-1">Email
51 ↪ Address</label>
52                         <p class="text-lg font-bold text-gray-800" id="profile-email">--</p>
53                     </div>
54                     <div>
55                         <label class="block text-xs font-bold uppercase text-secondary mb-1">Phone
56 ↪ Number</label>
57                         <p class="text-lg font-bold font-mono text-gray-800" id="profile-phone">--</p>
58                     </div>
59                 </div>
60
61                 <!-- Membership Details -->
62                 <div class="grid grid-cols-1 md:grid-cols-2 gap-6 pt-4 border-t-2 border-dashed
63 ↪ border-gray-300">
64                     <div>
65                         <label class="block text-xs font-bold uppercase text-secondary
66 ↪ mb-1">Membership Type</label>
67                         <div class="inline-block bg-contrast border-2 border-primary px-4 py-2
68 ↪ text-primary font-bold uppercase shadow-[2px_2px_0px_0px_#296374]">
69                             <span id="profile-type">Standard</span>
70                         </div>
71                     </div>
72                     <!-- Current Level omitted -->
73                 </div>
74
75             </div>
76         </div>
77
78         <!-- Footer/Dates -->
79         <div class="mt-8 pt-6 border-t-4 border-primary bg-gray-50 -mx-8 -mb-8 p-6 flex
80 ↪ justify-between items-center text-sm font-semibold text-gray-500">
81             <div>
82                 <span>Member Since:</span>
83                 <span class="text-primary font-bold" id="profile-since">--</span>
84             </div>
85             <div>
86                 <span>Valid Until:</span>

```

```

76         <span class="text-secondary font-bold" id="profile-until"></span>
77     </div>
78 </div>
79
80 </div>
81 </div>
82
83 </div>

```

Listing 8.73: ซอร์สโค้ดจากไฟล์ frontend/views/user/profile.ejs: Profile Card

คำอธิบาย:

1. **Profile Card:** ใช้การจัดวางแบบ Flexbox เพื่อแสดงรูป Avatar และข้อมูลส่วนตัวควบคู่กัน รองรับการปรับเปลี่ยนตามขนาดหน้าจอ (Responsive)
2. **Dynamic Data:** มีการใช้ id (เช่น profile-name, profile-email) เพื่อให้ JavaScript สามารถนำข้อมูลจาก Server มาแสดงผลได้แบบไดนามิก
3. **Membership Info:** แสดงประเภทสมาชิกและวันหมดอายุอย่างชัดเจน เพื่อให้สมาชิกตรวจสอบสถานะของตนเองได้ง่าย

สคริปต์ (Scripts)

การนำเข้าไฟล์ JavaScript ที่จำเป็น รวมถึง 'user/profile.js' สำหรับดึงข้อมูลโปรไฟล์มาแสดง

```

1  <script>
2    lucide.createIcons();
3    // Sidebar Logic (omitted)
4  </script>
5
6  <script src="/js/config.js"></script>
7  <script src="/js/auth.js"></script>
8  <script src="/js/user/profile.js"></script>
9 </body>
10 </html>

```

Listing 8.74: ซอร์สโค้ดจากไฟล์ frontend/views/user/profile.ejs: Scripts

8.4 ฟีเจอร์เพิ่มเติม (Additional Features)

8.4.1 Dark Mode (โหมดมืด)

ระบบรองรับการสลับโหมดมืด/สว่าง (Dark/Light Mode) โดยใช้ Tailwind CSS และ LocalStorage เพื่อจำค่าการตั้งค่าของผู้ใช้

การทำงาน (Implementation)

1. **Tailwind Configuration:** กำหนด darkMode: 'class' ในการตั้งค่า Tailwind เพื่อให้สามารถควบคุมโหมดผ่าน HTML Class ได้
2. **LocalStorage:** ใช้ localStorage.theme เพื่อบันทึกสถานะโหมดที่ผู้ใช้เลือก (dark หรือ light)
3. **theme.js:** ไฟล์ JavaScript ที่ทำหน้าที่ตรวจสอบค่าจาก LocalStorage เมื่อโหลดหน้าเว็บ และเพิ่ม/ลบ Class dark ที่ <html> tag รวมถึงฟังก์ชัน toggleTheme() สำหรับสลับโหมด

```

1 function initTheme() {
2   if (localStorage.theme === 'dark' || (!('theme' in localStorage) &&
3     ↪ window.matchMedia('(prefers-color-scheme: dark)').matches)) {
4     document.documentElement.classList.add('dark');
5     updateIcon(true);
6   } else {
7     document.documentElement.classList.remove('dark');

```

```

7       updateIcon(false);
8   }
9 }
10
11 function toggleTheme() {
12     if (document.documentElement.classList.contains('dark')) {
13         document.documentElement.classList.remove('dark');
14         localStorage.theme = 'light';
15         updateIcon(false);
16     } else {
17         document.documentElement.classList.add('dark');
18         localStorage.theme = 'dark';
19         updateIcon(true);
20     }
21 }
22 // ...

```

Listing 8.75: ไฟล์ frontend/js/theme.js

8.4.2 รายงานพร้อมพิมพ์ (Printable Report)

หน้า **Reports & Logs** รองรับการพิมพ์รายงานผ่านเครื่องพิมพ์หรือบันทึกเป็น PDF ได้อย่างสวยงาม โดยใช้ CSS Media Query

การทำงาน (Implementation)

ใช้ `@media print` ในการซ่อนส่วนประกอบที่ไม่จำเป็นเมื่อสั่งพิมพ์ เช่น Sidebar, Navbar, และปุ่มต่างๆ เหลือไว้เพียงเนื้อหาสำคัญ

```

1 @media print {
2     aside, header, #mobile-sidebar, #btn-print, .no-print, button {
3         display: none !important;
4     }
5     main {
6         margin: 0 !important;
7         padding: 0 !important;
8         width: 100% !important;
9         height: auto !important;
10        overflow: visible !important;
11    }
12    body {
13        background: white !important;
14        color: black !important;
15        overflow: visible !important;
16    }
17    .overflow-y-auto {
18        overflow: visible !important;
19        height: auto !important;
20    }
21 }

```

Listing 8.76: CSS @media print ใน logs.ejs

8.4.3 การค้นหาและแบ่งหน้า (Search & Pagination) - ส่วน Frontend

ในหน้าจัดการสมาชิก (Customers) มีระบบค้นหาและแบ่งหน้าเพื่อรองรับข้อมูลจำนวนมาก

การทำงาน (Implementation)

1. **Search Input:** รับค่าคำค้นหาจากผู้ใช้
2. **Pagination Controls:** ปุ่ม Next/Previous สำหรับเปลี่ยนหน้า
3. **Fetch API:** ส่ง Request ไปยัง Backend พร้อม Query Parameters `?q=...&page=...&limit=...`

Logic การค้นหาและแบ่งหน้า (customers.js)

ไฟล์ frontend/js/customers.js รับผิดชอบการดึงข้อมูลสมาชิกจาก API โดยรองรับทั้งการค้นหา (Search) และการแบ่งหน้า (Pagination)

```
1 const load = async () => {
2   const q = document.getElementById("search-input").value;
3   // Query Params: page, limit, q (search query)
4   const res = await fetch(
5     `${API}/customers?page=${currentPage}&limit=${limit}&q=${encodeURIComponent(q)}`
6   ).then((r) => r.json());
7
8   let data = [];
9   if (Array.isArray(res)) {
10     data = res; // Fallback for old API
11   } else {
12     data = res.data;
13     totalPages = res.totalPages;
14     currentPage = res.page;
15   }
16
17   tbody.innerHTML = "";
18   if (data.length === 0) {
19     tbody.innerHTML = `<tr><td colspan="6" class="p-4 text-center">No customers
20     ↳ found</td></tr>`;
21   } else {
22     data.forEach((c) => tbody.appendChild(row(c)));
23   }
24   updatePaginationControls();
25 };
```

Listing 8.77: ฟังก์ชัน load ใน customers.js

คำอธิบาย:

1. **Query Parameters:** ดึงค่าจาก Input และตัวแปร Global (currentPage, limit) เพื่อสร้าง URL Query String
2. **API Response Handling:** ตรวจสอบรูปแบบข้อมูลที่ได้จาก API (Array หรือ Object พร้อม Metadata) เพื่อรองรับการเปลี่ยนแปลงของ Backend
3. **Dynamic Rendering:** วนloopสร้างตาราง HTML จากข้อมูลที่ได้
4. **Pagination UI:** อัปเดตสถานะปุ่ม Previous/Next และแสดงเลขหน้าปัจจุบัน

8.4.4 frontend/js/equipments.js

ฟังก์ชัน row (สร้างแถวตาราง)

ฟังก์ชันนี้ทำหน้าที่สร้าง Element HTML (<tr>) สำหรับแสดงข้อมูลอุปกรณ์แต่ละแถวในตาราง รวมถึงการสร้าง Drop-down สำหรับเปลี่ยนสถานะและปุ่มลบ

```
1 const row = (e) => {
2   const tr = document.createElement("tr");
3   tr.className = "hover:bg-contrast transition-colors border-b border-gray-100";
4
5   // Status Selector
6   const sel = document.createElement("select");
7   sel.className = "bg-white border border-primary p-1 text-xs font-bold uppercase
8   ↳ focus:outline-none focus:border-accent";
9   statusOptions.forEach((s) => {
10     const o = document.createElement("option");
11     o.value = s;
12     o.textContent = s;
13     if (s === e.status) o.selected = true;
14     sel.appendChild(o);
15   });
16
17   // Update Status on Change
18   sel.addEventListener("change", async () => {
19     await fetch(`${API}/equipments/${e.id}`, {
```

```

19     method: "PUT",
20     headers: { "Content-Type": "application/json" },
21     body: JSON.stringify({ status: sel.value }),
22   });
23 });
24
25 // Delete Button
26 const actions = document.createElement("td");
27 actions.className = "p-4 text-center";
28 const del = document.createElement("button");
29 del.textContent = "Delete";
30 del.addEventListener("click", async () => {
31   if(confirm('Are you sure you want to delete this equipment?')) {
32     const r = await fetch(`${API}/equipments/${e.id}`, { method: "DELETE" });
33     if (r.ok) load();
34   }
35 });
36 actions.appendChild(del);
37
38 tr.innerHTML = `
39   <td class="p-4 font-bold text-primary">#${e.id}</td>
40   <td class="p-4 font-bold">${e.equipmentName}</td>
41   <td class="p-4 font-semibold text-secondary">${e.category || "-"}</td>
42 `;
43
44 const tdSel = document.createElement("td");
45 tdSel.className = "p-4";
46 tdSel.appendChild(sel);
47 tr.appendChild(tdSel);
48 tr.appendChild(actions);
49 return tr;
50 };

```

Listing 8.78: ฟังก์ชัน row ใน equipments.js

คำอธิบาย:

1. **Dynamic Row Creation:** สร้าง Element tr และกำหนด Class ของ Tailwind CSS
2. **Status Dropdown:** สร้าง select element และวนลูปสร้าง option จาก statusOptions
3. **Event Handling:**
 - change: เมื่อเปลี่ยนสถานะ จะยิง PUT Request เพื่ออัปเดตข้อมูลทันที
 - click: เมื่อคลิกปุ่ม Delete จะแสดง Confirm Dialog และยิง DELETE Request

ฟังก์ชัน load (โหลดข้อมูล)

ดึงข้อมูลอุปกรณ์ทั้งหมดจาก API มาแสดงผล

```

1 const load = async () => {
2   const data = await fetch(`${API}/equipments`).then((r) => r.json());
3   tbody.innerHTML = "";
4   data.forEach((e) => tbody.appendChild(row(e)));
5 };

```

Listing 8.79: ฟังก์ชัน load ใน equipments.js

คำอธิบาย:

1. เรียก API GET /equipments
2. ล้างข้อมูลเดิมในตาราง (innerHTML = "")
3. วนลูปข้อมูลที่ได้และเรียกใช้ฟังก์ชัน row() เพื่อสร้างแถว

การเพิ่มอุปกรณ์ใหม่ (Form Handler)

จัดการการส่งฟอร์มเพื่อเพิ่มอุปกรณ์ใหม่

```
1 form.addEventListener("submit", async (e) => {
2   e.preventDefault();
3   const body = {
4     equipmentName: nameEl.value.trim(),
5     category: categoryEl.value.trim() || null,
6     status: statusEl.value,
7   };
8   const r = await fetch(`${API}/equipments`, {
9     method: "POST",
10    headers: { "Content-Type": "application/json" },
11    body: JSON.stringify(body)
12  });
13  if (r.ok) {
14    form.reset();
15    load();
16  }
17 });
```

Listing 8.80: Form Submit Handler ใน equipments.js

คำอธิบาย:

1. ป้องกันการ Refresh หน้าจอด้วย e.preventDefault()
2. เตรียมข้อมูล JSON Payload
3. ส่ง POST Request ไปยัง API และรีเฟรชตารางเมื่อสำเร็จ

8.4.5 frontend/js/trainers.js

ฟังก์ชัน row (สร้างแถวตาราง)

สร้าง HTML สำหรับแสดงข้อมูลเทรนเนอร์

```
1 const row = (t) => {
2   const tr = document.createElement("tr");
3   tr.className = "hover:bg-contrast transition-colors border-b border-gray-100";
4
5   tr.innerHTML = `
6     <td class="p-4 font-bold text-primary">#${t.id}</td>
7     <td class="p-4 font-bold">${t.trainerName}</td>
8     <td class="p-4 font-semibold text-secondary">${t.specialty || "-"}</td>
9     <td class="p-4"><span class="bg-secondary text-white px-2 py-1 text-xs font-bold
10     ↪ uppercase">${t.trainerLevel}</span></td>
11     <td class="p-4 text-sm font-mono">${t.phone || "-"}</td>
12     <td class="p-4 text-center">
13       <div class="flex justify-center gap-2">
14         <button data-id="${t.id}" data-action="edit" class="text-accent hover:text-primary
15         ↪ transition-colors font-bold uppercase text-xs border-b-2 border-transparent
16         ↪ hover:border-primary">Edit</button>
17         <button data-id="${t.id}" data-action="delete" class="text-red-500 hover:text-red-700
18         ↪ transition-colors font-bold uppercase text-xs border-b-2 border-transparent
19         ↪ hover:border-red-700">Delete</button>
20       </div>
21     </td>
22   `;
23   return tr;
24 };
```

Listing 8.81: ฟังก์ชัน row ใน trainers.js

คำอธิบาย:

1. แสดงข้อมูลเทรนเนอร์ในแต่ละคอลัมน์
2. สร้างปุ่ม Edit และ Delete โดยฝัง data-id และ data-action ไว้ในปุ่ม เพื่อใช้สำหรับ Event Delegation

ฟังก์ชัน load (โหลดข้อมูล)

```
1 const load = async () => {
2   const data = await fetch(`${API}/trainers`).then((r) => r.json());
3   tbody.innerHTML = "";
4   data.forEach((t) => tbody.appendChild(row(t)));
5 };
```

Listing 8.82: ฟังก์ชัน load ใน trainers.js

คำอธิบาย:

1. ดึงข้อมูลเทรนเนอร์ทั้งหมดและแสดงผลในตาราง

การจัดการปุ่ม Edit/Delete (Event Delegation)

ใช้เทคนิค Event Delegation โดยดักจับการคลิกที่ tbody แทนการผูก Event กับปุ่มทุกปุ่ม

```
1 tbody.addEventListener("click", async (e) => {
2   const btn = e.target.closest("button");
3   if (!btn) return;
4   const id = btn.getAttribute("data-id");
5   const action = btn.getAttribute("data-action");
6
7   if (action === "delete") {
8     const r = await fetch(`${API}/trainers/${id}`, { method: "DELETE" });
9     if (r.ok) load();
10  }
11
12  if (action === "edit") {
13    try {
14      const currentData = await fetch(`${API}/trainers/${id}`).then(r => r.json());
15
16      const name = prompt(" ", currentData.trainerName || "");
17      if (name === null) return;
18
19      const specialty = prompt(" ", currentData.specialty || "");
20      if (specialty === null) return;
21
22      if (name) {
23        const r = await fetch(`${API}/trainers/${id}`, {
24          method: "PUT",
25          headers: { "Content-Type": "application/json" },
26          body: JSON.stringify({ trainerName: name, specialty }),
27        });
28        if (r.ok) load();
29      }
30    } catch (err) {
31      console.error(err);
32      alert("Error fetching trainer data");
33    }
34  }
35 });
```

Listing 8.83: Event Delegation ใน trainers.js

คำอธิบาย:

1. **Delete:** เมื่อกดปุ่ม Delete จะทำการลบข้อมูลทันที
2. **Edit:** เมื่อกดปุ่ม Edit จะใช้ prompt ของ Browser ในการรับค่าชื่อและความเชี่ยวชาญใหม่ (แบบง่าย) แล้วส่ง PUT Request ไปอัปเดตข้อมูล

8.4.6 frontend/js/sessions.js

ฟังก์ชัน Helper และ Row Creation

ฟังก์ชันช่วยสร้าง Option สำหรับ Dropdown และสร้างแถวตาราง

```

1 const opt = (v, t) => {
2   const o = document.createElement("option");
3   o.value = v;
4   o.textContent = t;
5   return o;
6 };
7
8 const row = (s) => {
9   const tr = document.createElement("tr");
10  tr.className = "hover:bg-contrast transition-colors border-b border-gray-100";
11
12  const actions = document.createElement("td");
13  actions.className = "p-4 text-center";
14  const del = document.createElement("button");
15  del.className = "text-red-500 hover:text-red-700 transition-colors font-bold uppercase text-xs
    ↳ border-b-2 border-transparent hover:border-red-700";
16  del.textContent = "Cancel";
17  del.addEventListener("click", async () => {
18    if(confirm('Are you sure you want to cancel this session?')) {
19      const r = await fetch(`${API}/sessions/${s.id}`, { method: "DELETE" });
20      if (r.ok) loadSessions();
21    }
22  });
23  actions.appendChild(del);
24
25  const date = new Date(s.sessionDate);
26  const dateStr = date.toLocaleDateString('th-TH', { timeZone: 'UTC', day: '2-digit', month:
    ↳ '2-digit', year: 'numeric' });
27  const timeStr = date.toLocaleTimeString('th-TH', { timeZone: 'UTC', hour: '2-digit', minute:
    ↳ '2-digit' });
28  const dateTimeStr = `${dateStr} ${timeStr}`;
29
30  tr.innerHTML = `
31    <td class="p-4 font-bold text-primary">${s.customer?.fullName || "Unknown"}</td>
32    <td class="p-4 font-semibold">${s.trainer?.trainerName || "-"}</td>
33    <td class="p-4">${s.equipment?.equipmentName || "Personal Training"}</td>
34    <td class="p-4 text-sm font-mono text-secondary">${dateTimeStr}</td>
35  `;
36  tr.appendChild(actions);
37  return tr;
38 };

```

Listing 8.84: ฟังก์ชัน opt และ row ใน sessions.js

คำอธิบาย:

1. opt: สร้าง <option> element อย่างง่าย
2. row: สร้างแถวตารางพร้อมแสดงชื่อลูกค้า เทรนเนอร์ อุปกรณ์ และเวลา โดยมีการแปลงเวลาให้เป็นรูปแบบที่อ่านง่าย และเพิ่มปุ่ม Cancel สำหรับยกเลิก Session

ฟังก์ชัน loadLists และ loadSessions

ดึงข้อมูล Master Data และ Session Data

```

1 const loadLists = async () => {
2   const [customersRes, trainers, equipments] = await Promise.all([
3     fetch(`${API}/customers?limit=1000`).then((r) => r.json()),
4     fetch(`${API}/trainers`).then((r) => r.json()),
5     fetch(`${API}/equipments`).then((r) => r.json()),
6   ]);
7
8   const customers = Array.isArray(customersRes) ? customersRes : (customersRes.data || []);
9
10  customerEl.innerHTML = '<option value="" disabled selected>Select Customer</option>';
11  trainerEl.innerHTML = '<option value="" disabled selected>Select Trainer</option>';
12  equipmentEl.innerHTML = '<option value="" disabled selected>Select Equipment</option>';
13
14  customers.forEach((c) => customerEl.appendChild(opt(c.id, c.fullName)));
15  trainers.forEach((t) => trainerEl.appendChild(opt(t.id, t.trainerName)));
16  equipments.forEach((e) => equipmentEl.appendChild(opt(e.id, e.equipmentName)));
17 };

```

```

18
19 const loadSessions = async () => {
20   const data = await fetch(`${API}/sessions`).then((r) => r.json());
21   tbody.innerHTML = "";
22   data.forEach((s) => tbody.appendChild(row(s)));
23 };

```

Listing 8.85: ฟังก์ชันโหลดข้อมูลใน sessions.js

คำอธิบาย:

1. loadLists: ใช้ Promise.all ดึงข้อมูล Customers, Trainers, Equipments พร้อมกันเพื่อนำมาใส่ใน Dropdown ให้แอดมินเลือก
2. loadSessions: ดึงข้อมูลการจองทั้งหมดมาแสดงในตาราง

การสร้าง Session ใหม่ (Form Handler)

```

1 form.addEventListener("submit", async (e) => {
2   e.preventDefault();
3
4   const scheduledDate = new Date(scheduledEl.value);
5   // Convert to fake UTC (preserve local time values)
6   const utcScheduled = new Date(Date.UTC(
7     scheduledDate.getFullYear(),
8     scheduledDate.getMonth(),
9     scheduledDate.getDate(),
10    scheduledDate.getHours(),
11    scheduledDate.getMinutes(),
12    0
13  ));
14
15  const body = {
16    customerId: parseInt(customerEl.value, 10),
17    trainerId: parseInt(trainerEl.value, 10),
18    equipmentId: parseInt(equipmentEl.value, 10),
19    sessionDate: utcScheduled.toISOString(),
20    duration: 60 // Default 1 hour
21  };
22
23  try {
24    const r = await fetch(`${API}/sessions`, {
25      method: "POST",
26      headers: { "Content-Type": "application/json" },
27      body: JSON.stringify(body)
28    });
29
30    if (r.ok) {
31      alert("Session created successfully!");
32      form.reset();
33      loadSessions();
34    } else {
35      const err = await r.json();
36      alert("Failed to create session: " + (err.error || "Unknown error"));
37    }
38  } catch (error) {
39    console.error("Error creating session:", error);
40    alert("Network error occurred.");
41  }
42 });

```

Listing 8.86: Form Submit Handler ใน sessions.js

คำอธิบาย:

1. แปลงเวลาที่เลือกจาก Input (Local Time) เป็น UTC แบบคงค่าตัวเลขเดิมไว้ (Fake UTC) เพื่อให้เวลาที่บันทึกตรงกับที่ตาเห็น
2. ส่งข้อมูลไปยัง API เพื่อสร้าง Session ใหม่

8.4.7 frontend/js/index.js (Admin Dashboard)

ฟังก์ชัน Helper และ Render

```
1 const setText = (id, v) => {
2   const el = document.getElementById(id);
3   if (el) el.textContent = v;
4 };
5
6 const renderSessions = (rows) => {
7   const tbody = document.getElementById("today-sessions");
8   tbody.innerHTML = "";
9   if (!rows || rows.length === 0) {
10    tbody.innerHTML = `<tr><td colspan="5" class="p-4 text-center text-gray-400 italic">No
    ↳ sessions scheduled for today.</td></tr>`;
11    return;
12  }
13  rows.forEach((r) => {
14    const tr = document.createElement("tr");
15    tr.className = "hover:bg-contrast transition-colors border-b border-gray-100";
16    tr.innerHTML = `
17      <td class="p-4 font-bold text-primary">${r.customer?.fullName || "Unknown"}</td>
18      <td class="p-4 font-semibold">${r.trainer?.trainerName || "-"}</td>
19      <td class="p-4">${r.equipment?.equipmentName || "Personal Training"}</td>
20      <td class="p-4 text-sm font-mono text-secondary">${new
    ↳ Date(r.sessionDate).toLocaleString()}</td>
21      <td class="p-4 text-center">
22        <span class="bg-accent text-primary px-2 py-1 text-xs font-bold uppercase
    ↳ tracking-wide">Confirmed</span>
23      </td>
24    `;
25    tbody.appendChild(tr);
26  });
27 };
```

Listing 8.87: ฟังก์ชัน Helper ใน index.js

คำอธิบาย:

1. setText: ช่วยอัปเดตข้อความใน Element ตาม ID
2. renderSessions: แสดงตารางงานของวันนี้ในหน้า Dashboard

ฟังก์ชัน load (Dashboard Data)

ดึงข้อมูลสรุป (Summary) และตารางงานวันนี้

```
1 const load = async () => {
2   const summary = await fetch(`${API}/summary`).then((r) => r.json());
3   setText("count-customers", summary.customers || 0);
4   setText("count-trainers", summary.trainers || 0);
5   setText("count-equipments", summary.equipments || 0);
6   setText("count-sessions", summary.sessions_today || 0);
7
8   const sessions = await fetch(`${API}/sessions?today=true`).then((r) => r.json());
9   renderSessions(sessions);
10 };
```

Listing 8.88: ฟังก์ชัน load ใน index.js

คำอธิบาย:

1. เรียก API /summary เพื่อดึงจำนวนสมาชิก, เทรนเนอร์, อุปกรณ์, และงานวันนี้ มาแสดงในการ์ดสถิติ
2. เรียก API /sessions?today=true เพื่อดึงเฉพาะงานของวันนี้มาแสดง

8.4.8 frontend/js/logs.js

ฟังก์ชัน Helper สำหรับวันที่และเวลา

ฟังก์ชันช่วยแปลงวันที่และคำนวณระยะเวลา

```

1 const formatDate = (dateStr) => {
2   if (!dateStr) return "-";
3   const date = new Date(dateStr);
4   return date.toLocaleDateString('th-TH', { timeZone: 'UTC', day: '2-digit', month: '2-digit',
    ↪ year: 'numeric' });
5 };
6
7 const formatTime = (dateStr) => {
8   if (!dateStr) return "-";
9   const date = new Date(dateStr);
10  return date.toLocaleTimeString('th-TH', { timeZone: 'UTC', hour: '2-digit', minute:
    ↪ '2-digit' });
11 };
12
13 const calcDuration = (startStr, endStr) => {
14   if (!startStr || !endStr) return "-";
15   const start = new Date(startStr);
16   const end = new Date(endStr);
17   const diffMs = end.getTime() - start.getTime();
18
19   const diffMins = Math.floor(diffMs / 60000);
20   const hours = Math.floor(diffMins / 60);
21   const mins = diffMins % 60;
22
23   if (hours > 0) return `${hours} hr ${mins} min`;
24   return `${mins} min`;
25 };

```

Listing 8.89: Helper Functions ใน logs.js

คำอธิบาย:

1. formatDate/formatTime: แปลง ISO Date String ให้เป็นรูปแบบวันที่และเวลาของไทย
2. calcDuration: คำนวณระยะเวลาความแตกต่างระหว่างเวลาเริ่มและเวลาจบในหน่วยชั่วโมงและนาที

ฟังก์ชัน applyDateFilter

```

1 function applyDateFilter() {
2   const dateVal = document.getElementById('filter-date').value;
3   if (dateVal) {
4     filterDate = dateVal;
5   } else {
6     filterDate = null;
7   }
8   refreshCurrentTab();
9 }

```

Listing 8.90: ฟังก์ชัน applyDateFilter ใน logs.js

คำอธิบาย:

1. อ่านค่าจาก Input Date Picker
2. อัปเดตตัวแปร filterDate และเรียก refreshCurrentTab() เพื่อโหลดข้อมูลใหม่

ฟังก์ชัน switchTab

```

1 window.switchTab = (tabName) => {
2   // Hide all sections
3   document.getElementById('section-logs').classList.add('hidden');
4   document.getElementById('section-customer').classList.add('hidden');
5   document.getElementById('section-trainer').classList.add('hidden');
6
7   // Reset tab styles
8   ['logs', 'customer', 'trainer'].forEach(t => {
9     const btn = document.getElementById(`tab-${t}`);
10    btn.classList.remove('tab-active');
11    btn.classList.add('tab-inactive');
12  });

```

```

13 // Show selected section
14 document.getElementById(`section-${tabName}`).classList.remove('hidden');
15
16 // Activate selected tab
17 const activeBtn = document.getElementById(`tab-${tabName}`);
18 activeBtn.classList.remove('tab-inactive');
19 activeBtn.classList.add('tab-active');
20
21 currentTab = tabName;
22 refreshCurrentTab();
23
24 };

```

Listing 8.91: ฟังก์ชัน switchTab ใน logs.js

คำอธิบาย:

1. จัดการการเปลี่ยน Tab โดยการซ่อน/แสดง Section ที่เกี่ยวข้อง
2. เปลี่ยน Style ของปุ่ม Tab เพื่อแสดงสถานะ Active

ฟังก์ชัน loadLogs

```

1 const loadLogs = async () => {
2   try {
3     logsTbody.innerHTML = `<tr><td colspan="3" class="p-4 text-center">Loading
4     ↪ logs...</td></tr>`;
5     const logs = await fetch(`${API}/logs`).then((r) => r.json());
6     logsTbody.innerHTML = "";
7
8     let filteredLogs = logs;
9     if (filterDate) {
10      filteredLogs = logs.filter(l => l.timestamp.startsWith(filterDate));
11    }
12
13    if (filteredLogs.length === 0) {
14      logsTbody.innerHTML = `<tr><td colspan="3" class="p-4 text-center text-gray-400 italic">No
15      ↪ logs found.</td></tr>`;
16      return;
17    }
18    filteredLogs.forEach((log) => logsTbody.appendChild(logRow(log)));
19  } catch (error) {
20    console.error("Error loading logs:", error);
21    logsTbody.innerHTML = `<tr><td colspan="3" class="p-4 text-center text-red-500
22    ↪ font-bold">Error loading logs.</td></tr>`;
23  }
24 };

```

Listing 8.92: ฟังก์ชัน loadLogs ใน logs.js

คำอธิบาย:

1. ดึงข้อมูล System Logs ทั้งหมดจาก API
2. กรองข้อมูลตามวันที่ (Client-side Filtering) หากมีการเลือกวันที่
3. แสดงผลข้อมูลในตาราง

ฟังก์ชัน loadCustomerReport และ loadTrainerReport

```

1 const loadCustomerReport = async () => {
2   try {
3     customerTbody.innerHTML = `<tr><td colspan="5" class="p-4 text-center">Loading
4     ↪ data...</td></tr>`;
5
6     let url = `${API}/sessions?all=true`;
7     if (filterDate) {
8       url = `${API}/sessions?date=${filterDate}`;
9     }
10  }
11 };

```

```

10     const sessions = await fetch(url).then(r => r.json());
11     customerTbody.innerHTML = "";
12
13     if (sessions.length === 0) {
14         customerTbody.innerHTML = `<tr><td colspan="5" class="p-4 text-center text-gray-400
15         ↪ italic">No history found.</td></tr>`;
16         return;
17     }
18     sessions.forEach(s => customerTbody.appendChild(customerRow(s)));
19 } catch (error) { ... }
20 };
21
22 // loadTrainerReport logic

```

Listing 8.93: ฟังก์ชันโหลดรายงานใน logs.js

คำอธิบาย:

1. สร้าง URL Query String ตาม Filter วันที่
2. ดึงข้อมูล Session และแสดงผลในตารางรายงาน

8.4.9 frontend/js/theme.js

ฟังก์ชัน initTheme

ตรวจสอบและตั้งค่า Theme เริ่มต้นเมื่อโหลดหน้าเว็บ

```

1 function initTheme() {
2     if (localStorage.theme === 'dark' || (!('theme' in localStorage) &&
3     ↪ window.matchMedia('(prefers-color-scheme: dark)').matches)) {
4         document.documentElement.classList.add('dark');
5         updateIcon(true);
6     } else {
7         document.documentElement.classList.remove('dark');
8         updateIcon(false);
9     }
10 }

```

Listing 8.94: ฟังก์ชัน initTheme ใน theme.js

คำอธิบาย:

1. ตรวจสอบว่าผู้ใช้เคยตั้งค่า Theme ไว้หรือไม่ หรือใช้การตั้งค่าของ System
2. กำหนด Class dark ให้กับ <html> element

ฟังก์ชัน toggleTheme

สลับโหมดระหว่าง Light และ Dark

```

1 function toggleTheme() {
2     if (document.documentElement.classList.contains('dark')) {
3         document.documentElement.classList.remove('dark');
4         localStorage.theme = 'light';
5         updateIcon(false);
6     } else {
7         document.documentElement.classList.add('dark');
8         localStorage.theme = 'dark';
9         updateIcon(true);
10    }
11 }

```

Listing 8.95: ฟังก์ชัน toggleTheme ใน theme.js

คำอธิบาย:

1. สลับสถานะ Class dark
2. บันทึกค่าลง localStorage

ฟังก์ชัน updateIcon

อัปเดตไอคอนบนปุ่มเปลี่ยน Theme

```
1 function updateIcon(isDark) {
2   const icon = document.getElementById('theme-icon');
3   if (icon) {
4     if (isDark) {
5       icon.setAttribute('data-lucide', 'sun');
6     } else {
7       icon.setAttribute('data-lucide', 'moon');
8     }
9     if (window.lucide) lucide.createIcons();
10  }
11 }
```

Listing 8.96: ฟังก์ชัน updateIcon ใน theme.js

คำอธิบาย:

1. เปลี่ยนไอคอนตามสถานะ Theme (พระอาทิตย์/พระจันทร์)

8.4.10 frontend/js/user/profile.js

ฟังก์ชัน loadProfile

ฟังก์ชันสำหรับดึงข้อมูลส่วนตัวของสมาชิกมาแสดงผลในหน้า Profile

```
1 async function loadProfile() {
2   const session = JSON.parse(localStorage.getItem('gymbro_user'));
3   if (!session || !session.user) return;
4
5   const userId = session.user.id;
6   try {
7     const res = await fetch(`${window.API}/customers/${userId}`);
8     if (!res.ok) throw new Error('Failed to fetch profile');
9     const user = await res.json();
10
11     // Update UI
12     setText('profile-name', user.fullName);
13     setText('profile-email', user.email);
14     setText('profile-phone', user.phone);
15     setText('profile-type', user.memberType || 'Standard');
16     setText('profile-level', user.memberLevel || 'Beginner');
17
18     if (user.memberStartDate) {
19       const since = new Date(user.memberStartDate).toLocaleDateString('th-TH', {
20         ↪ dateStyle: 'long' });
21       setText('profile-since', since);
22     }
23
24     if (user.memberEndDate) {
25       const until = new Date(user.memberEndDate).toLocaleDateString('th-TH', { dateStyle:
26         ↪ 'long' });
27       setText('profile-until', until);
28     } else {
29       setText('profile-until', 'Lifetime / No Expiry');
30     }
31   } catch (error) {
32     console.error('Error loading profile:', error);
33     // Fallback to local storage if API fails
34     const user = session.user;
35     setText('profile-name', user.fullName);
36     setText('profile-email', user.email);
37     setText('profile-phone', user.phone);
38   }
39 }
```

Listing 8.97: ฟังก์ชัน loadProfile ใน user/profile.js

คำอธิบาย:

1. ดึงข้อมูล User ID จาก localStorage

2. เรียก API GET /customers/:id เพื่อดึงข้อมูลล่าสุดจาก Server
3. แสดงผลข้อมูลลงใน UI โดยใช้ Helper Function setText
4. แปลงวันที่ให้เป็นรูปแบบภาษาไทยที่อ่านง่าย (เช่น "1 มกราคม 2567")
5. มีระบบ Fallback หากเรียก API ไม่สำเร็จ จะนำข้อมูลจาก Local Storage มาแสดงแทน

ฟังก์ชัน setText

```
1 function setText(id, text) {  
2   const el = document.getElementById(id);  
3   if (el) el.textContent = text;  
4 }
```

Listing 8.98: ฟังก์ชัน setText ใน user/profile.js

คำอธิบาย:

1. ฟังก์ชันช่วยลดความซ้ำซ้อนในการเขียน document.getElementById และตรวจสอบว่า Element มีอยู่จริงก่อนกำหนดค่า

บทที่ 9

การทดสอบและประเมินผลระบบ (System Testing and Evaluation)

บทนี้กล่าวถึงกระบวนการทดสอบและประเมินผลระบบบริหารจัดการยิม (GymBro Management System) เพื่อให้มั่นใจว่าระบบที่พัฒนาขึ้นสามารถทำงานได้ถูกต้องตามความต้องการของผู้ใช้งาน มีความเสถียร และมีความปลอดภัย โดยครอบคลุมตั้งแต่การทดสอบฟังก์ชันการทำงาน (Functional Testing) ไปจนถึงการทดสอบประสิทธิภาพ (Performance Testing) และการทดสอบการยอมรับของผู้ใช้ (User Acceptance Testing)

9.1 กระบวนการและสภาพแวดล้อมการทดสอบ (Testing Methodology and Environment)

9.1.1 วิธีการทดสอบ (Testing Methodology)

การทดสอบระบบในครั้งนี้ใช้วิธีการทดสอบแบบกล่องดำ (Black-box Testing) ซึ่งเป็นการตรวจสอบการทำงานของระบบจากมุมมองของผู้ใช้งานภายนอก โดยมุ่งเน้นไปที่การตรวจสอบอินพุต (Input) และเอาต์พุต (Output) ว่าเป็นไปตามข้อกำหนดความต้องการหรือไม่ โดยไม่คำนึงถึงโครงสร้างโค้ดภายในหรืออัลกอริทึมที่ซับซ้อน เพื่อจำลองการใช้งานจริงให้มากที่สุด

9.1.2 สภาพแวดล้อมการทดสอบ (Testing Environment)

การทดสอบถูกดำเนินการบนสภาพแวดล้อมที่จำลองคล้ายกับการใช้งานจริง (Production Environment) โดยมีรายละเอียดทางเทคนิคดังนี้:

- **Server Side:** Node.js Runtime Environment เวอร์ชันล่าสุด รันบนระบบปฏิบัติการ macOS
- **Backend Framework:** Express.js ทำหน้าที่เป็น Web Server และ API Endpoint
- **Frontend Interface:** เว็บเบราว์เซอร์ Google Chrome และ Safari เวอร์ชันล่าสุด เพื่อทดสอบการแสดงผลของ EJS Template และ Tailwind CSS
- **Database:** PostgreSQL Database Server ที่เชื่อมต่อผ่าน Sequelize ORM
- **Network:** การเชื่อมต่อผ่าน Localhost และจำลอง Network Throttling เพื่อทดสอบความเร็วอินเทอร์เน็ตที่แตกต่าง

9.2 การทดสอบฟังก์ชันการทำงาน (Functional Testing)

การทดสอบฟังก์ชันการทำงานแบ่งออกเป็นโมดูลหลักๆ ได้แก่ ระบบยืนยันตัวตน, ระบบการจอง และระบบจัดการข้อมูล โดยมีรายละเอียดกรณีทดสอบ (Test Cases) ดังตารางต่อไปนี้

9.2.1 โมดูลการยืนยันตัวตน (Authentication Module)

Test ID	Test Description	Prerequisites	Test Steps	Expected Result	Re-	Result
---------	------------------	---------------	------------	-----------------	-----	--------

AUTH-01	เข้าสู่ระบบด้วยข้อมูลที่ถูกต้อง (Member)	มีบัญชีสมาชิกในระบบ	1. เปิดหน้า Login 2. กรอก Email และ Phone ที่ถูกต้อง 3. กดปุ่ม Sign In	ระบบนำทางไปยังหน้า User Dashboard	Pass
AUTH-02	เข้าสู่ระบบด้วยข้อมูลที่ถูกต้อง (Admin)	มีบัญชีผู้ดูแลระบบ	1. เปิดหน้า Login 2. กรอก Email และ Phone ของ Admin 3. กดปุ่ม Sign In	ระบบนำทางไปยังหน้า Admin Dashboard	Pass
AUTH-03	เข้าสู่ระบบด้วยข้อมูลที่ผิด	ไม่มี	1. เปิดหน้า Login 2. กรอก Email ที่ไม่มีในระบบ 3. กดปุ่ม Sign In	แสดงข้อความแจ้งเตือน "Invalid credentials"	Pass
AUTH-04	การตรวจสอบสิทธิ์การเข้าถึง (Access Control)	เข้าสู่ระบบในฐานะ Member	1. พยายามเข้าถึง URL / admin/ dashboard โดยตรงผ่าน Address Bar	ระบบปฏิเสธการเข้าถึงและนำทางกลับไปยังหน้า Login หรือ User Dashboard	Pass

9.2.2 โมดูลการจองและการจัดการตารางเวลา (Booking Module)

Test ID	Test Description	Prerequisites	Test Steps	Expected Result	Result
BOOK-01	จองเซสชันสำเร็จ	สมาชิก เข้าสู่ระบบแล้ว	1. เลือกวันและเวลา 2. เลือกเทรนเนอร์และอุปกรณ์ที่ว่าง 3. กดยืนยันการจอง	บันทึกข้อมูลลงฐานข้อมูล และแสดงสถานะการจองสำเร็จ	Pass
BOOK-02	ตรวจสอบการจองซ้อนทับ (Time Conflict Detection)	มีการจองในช่วงเวลา 10:00-11:00 น. แล้ว	1. เลือกวันเดียวกัน 2. เลือกเวลา 10:30 น. (ทับซ้อน) 3. ตรวจสอบตัวเลือกเทรนเนอร์	เทรนเนอร์ที่ติดจองต้องไม่สามารถเลือกได้ (Disabled)	Pass
BOOK-03	ยกเลิกการจอง	สมาชิกมีการจองล่วงหน้า	1. ไปที่หน้า Dashboard 2. กดปุ่ม Cancel ที่รายการจอง 3. ยืนยันการยกเลิก	สถานะการจองถูกลบออกจากตาราง และเทรนเนอร์กลับมาว่าง	Pass

9.2.3 โมดูลการจัดการข้อมูลโดยผู้ดูแลระบบ (Data Management)

Test ID	Test Description	Prerequisites	Test Steps	Expected Result	Result
ADM-01	เพิ่มข้อมูลเทรนเนอร์ใหม่	Admin เข้าสู่ระบบแล้ว	1. ไปที่เมนู Trainers 2. กด Add Trainer 3. กรอกข้อมูลครบถ้วนและบันทึก	รายชื่อเทรนเนอร์ใหม่ปรากฏในตารางและฐานข้อมูล	Pass
ADM-02	แก้ไขสถานะอุปกรณ์	มีข้อมูลอุปกรณ์ในระบบ	1. ไปที่เมนู Equipment 2. เลือกอุปกรณ์และเปลี่ยนสถานะเป็น Maintenance	สถานะเปลี่ยนแปลงและไม่สามารถถูกจองได้โดยสมาชิก	Pass

ADM-03	การ ตรวจสอบ ความ ถูกต้อง ของ ข้อมูล (Validation)	Admin เข้า สู่ ระบบ แล้ว	1. พยายามเพิ่มสมาชิกใหม่โดยไม่กรอก Email 2. กดบันทึก	ระบบ แสดง ข้อความ แจ้ง เตือน และ ไม่ บันทึกข้อมูล	Pass
--------	--	--------------------------	---	---	------

9.3 การทดสอบประสิทธิภาพและการรองรับภาระงาน (Performance and Load Testing)

9.3.1 แนวคิดการทดสอบประสิทธิภาพบน Node.js

เนื่องจากระบบพัฒนาด้วย Node.js ซึ่งทำงานแบบ **Single-Threaded Non-blocking I/O** การทดสอบประสิทธิภาพจึงมุ่งเน้นไปที่ความสามารถในการจัดการคำขอ (Requests) จำนวนมากพร้อมกัน (Concurrency) โดยไม่ทำให้ระบบล่มหรือตอบสนองช้าจนเกินไป รวมถึงการจัดการ Connection Pool ของฐานข้อมูล PostgreSQL ให้มีประสิทธิภาพสูงสุด

9.3.2 สถานการณ์จำลองการทดสอบ (Simulated Load Test Scenario)

ทีมผู้พัฒนาได้จำลองสถานการณ์เพื่อทดสอบขีดจำกัดของระบบ ดังนี้:

- **Scenario:** ผู้ใช้งาน 100 คนทำการกดจองคลาสเรียนพร้อมกันในวินาทีเดียวกัน (Concurrent Booking Requests)
- **Environment:** ใช้เครื่องมือ Apache JMeter ในการยิง Request ไปยัง API Endpoint `/api/sessions`
- **ผลการทดสอบ:**
 - Express.js สามารถรับ Request ทั้งหมดและเข้าคิวประมวลผลได้โดยไม่เกิด Deadlock
 - ระบบตรวจสอบ Time Conflict ทำงานได้อย่างถูกต้อง โดยมีเพียง Request แรกที่จองสำเร็จ ส่วน Request ที่เหลือได้รับแจ้งเตือนว่า "Time slot is no longer available"
 - เวลาตอบสนองเฉลี่ย (Average Response Time) อยู่ที่ 250ms ซึ่งอยู่ในเกณฑ์ที่ยอมรับได้

9.4 การทดสอบการยอมรับของผู้ใช้ (User Acceptance Testing - UAT)

9.4.1 เกณฑ์การยอมรับ (Acceptance Criteria)

การทดสอบ UAT ดำเนินการโดยกลุ่มผู้ใช้งานตัวอย่าง ประกอบด้วยผู้ดูแลระบบและสมาชิกทั่วไป โดยมีเกณฑ์การประเมิน ดังนี้:

1. **Usability:** ระบบต้องใช้งานง่าย ไม่ซับซ้อน เมนูและปุ่มต่างๆ สื่อความหมายชัดเจน
2. **Functionality:** ระบบต้องทำงานได้ถูกต้องตามคู่มือการใช้งาน ไม่พบข้อผิดพลาดร้ายแรง (Critical Bugs)
3. **Reliability:** ข้อมูลที่บันทึกต้องมีความถูกต้องแม่นยำ โดยเฉพาะตารางการจอง

9.4.2 ผลการประเมินและความคิดเห็น (Evaluation Summary)

จากการให้กลุ่มตัวอย่างทดลองใช้งานระบบ GymBro Management System พบว่า:

- **ด้านความง่ายในการใช้งาน:** ผู้ใช้พึงพอใจกับการออกแบบหน้าจอที่เรียบง่าย (Clean Interface) และการใช้สี (Color Coding) ในหน้าตารางเรียนที่ช่วยให้ดูสถานะได้ง่าย
- **ด้านความถูกต้อง:** ระบบป้องกันการจองซ้อนได้จริง ซึ่งเป็นฟีเจอร์ที่ผู้ดูแลระบบให้ความสำคัญมากที่สุด
- **ข้อเสนอแนะเพิ่มเติม:** ผู้ใช้บางส่วนเสนอแนะให้เพิ่มระบบแจ้งเตือนผ่าน LINE หรือ SMS เมื่อใกล้ถึงเวลาจอง เพื่อความสะดวกมากยิ่งขึ้น

โดยสรุป ผลการทดสอบระบบในภาพรวมอยู่ในระดับ **ผ่านเกณฑ์ (Passed)** และพร้อมสำหรับการนำไปใช้งานจริง

บทที่ 10

สรุป

10.1 สรุปผลการดำเนินงาน

บทนี้จะกล่าวถึงบทสรุปของการดำเนินงานโครงการระบบบริหารจัดการยิม (GymBro Management System) โดยครอบคลุมถึงภาพรวมของระบบที่พัฒนาขึ้น ผลลัพธ์ที่ได้จากการพัฒนา ปัญหาและอุปสรรคที่พบระหว่างการดำเนินงาน รวมถึงข้อเสนอแนะและแนวทางในการพัฒนาต่อของระบบในอนาคต

10.1.1 สรุปภาพรวมของระบบ (System Overview Summary)

ระบบบริหารจัดการยิม (GymBro Management System) ถูกพัฒนาขึ้นเพื่อเป็นเว็บแอปพลิเคชันสำหรับช่วยบริหารจัดการข้อมูลและการดำเนินงานภายในยิมให้มีความสะดวก รวดเร็ว และถูกต้องแม่นยำมากยิ่งขึ้น โดยมีวัตถุประสงค์หลักเพื่อลดความผิดพลาดจากการทำงานด้วยมือ (Manual Process) และเพิ่มประสิทธิภาพในการให้บริการแก่สมาชิก ระบบถูกออกแบบให้รองรับการทำงานของกลุ่มผู้ใช้งาน 2 กลุ่มหลัก ได้แก่

- ผู้ดูแลระบบ (Administrator):** สามารถบริหารจัดการข้อมูลพื้นฐานของระบบ เช่น ข้อมูลสมาชิก (Customers), ข้อมูลเทรนเนอร์ (Trainers), ข้อมูลอุปกรณ์ออกกำลังกาย (Gym Equipments) และการจัดการตารางการจอบคลาสเรียน (Training Sessions) รวมถึงสามารถดูบันทึกการใช้งานระบบ (System Logs) ได้
- สมาชิกทั่วไป (Member/User):** สามารถเข้าสู่ระบบเพื่อดูข้อมูลส่วนตัว แก้ไขโปรไฟล์ และตรวจสอบสถานะการเป็นสมาชิกได้

ในส่วนของการสถาปัตยกรรมระบบ ได้มีการเลือกใช้เทคโนโลยี Full Stack Development ที่ทันสมัย โดยฝั่ง Server-Side ใช้ Node.js ร่วมกับ Express.js ในการสร้าง RESTful API และ Web Server ส่วนฝั่ง Client-Side ใช้ EJS (Embedded JavaScript Templating) ร่วมกับ Tailwind CSS ในการแสดงผลหน้าเว็บที่สวยงามและรองรับการใช้งานบนอุปกรณ์ต่างๆ (Responsive Design) และใช้ฐานข้อมูล PostgreSQL ในการจัดเก็บข้อมูลที่มีความสัมพันธ์กัน (Relational Database) ผ่าน Sequelize ORM

10.1.2 ผลการพัฒนาของระบบ (Development Achievements)

จากการดำเนินงานพัฒนาโครงการ สามารถสรุปผลการพัฒนาระบบในส่วนต่างๆ ได้ดังนี้:

- ระบบจัดการฐานข้อมูล (Database Management):** สามารถออกแบบและสร้างฐานข้อมูลเชิงสัมพันธ์ (Relational Database) ที่มีความถูกต้องตามหลักการ Normalization และสามารถเชื่อมต่อกับแอปพลิเคชันผ่าน Sequelize ORM ได้อย่างสมบูรณ์ รองรับการทำ CRUD Operations (Create, Read, Update, Delete) กับข้อมูลทุกตาราง
- ระบบ Backend API:** ได้พัฒนา RESTful API ที่มีประสิทธิภาพสำหรับการจัดการข้อมูลหลักของระบบ ได้แก่:
 - API สำหรับการเข้าสู่ระบบ (Authentication) และการตรวจสอบสิทธิ์ (Authorization)
 - API สำหรับจัดการข้อมูลสมาชิก (Customers), เทรนเนอร์ (Trainers), และอุปกรณ์ (Equipments)
 - API สำหรับการจองคลาสเรียน (Training Sessions) และการคำนวณวันหมดอายุสมาชิก
- ระบบ Frontend Interface:** ได้พัฒนาส่วนติดต่อผู้ใช้งาน (User Interface) ที่มีความสวยงามและใช้งานง่าย โดยแบ่งส่วนการทำงานระหว่างหน้าบ้าน (User Dashboard) และหลังบ้าน (Admin Dashboard) อย่างชัดเจน มีการใช้

JavaScript ในการดึงข้อมูลจาก API มาแสดงผลแบบ Dynamic และมีการจัดการ Form Validation เพื่อป้องกันข้อมูลที่ผิดพลาด

- **ระบบความปลอดภัยเบื้องต้น:** มีการตรวจสอบสิทธิ์การเข้าถึงข้อมูล (Role-Based Access Control) โดยแยกสิทธิ์ระหว่าง Admin และ User อย่างชัดเจน และมีการป้องกันการเข้าถึงหน้าเว็บที่ไม่ได้รับอนุญาต (Route Guarding)
- **ฟังก์ชันพิเศษ:** ระบบมีการบันทึกประวัติการใช้งาน (Logging) ลงในไฟล์ JSON เพื่อให้ผู้ดูแลระบบสามารถตรวจสอบย้อนหลังได้ และมีฟังก์ชันการเรียงลำดับ ID ของข้อมูลใหม่ (ID Reordering) เพื่อให้ข้อมูลมีความต่อเนื่องสวยงามเมื่อมีการลบข้อมูลออกไป

10.1.3 ปัญหาและข้อจำกัด (Limitations and Challenges)

จากการวิเคราะห์และทดสอบระบบ พบว่ายังมีข้อจำกัดและประเด็นที่ควรได้รับการปรับปรุง ดังนี้:

1. ความปลอดภัยของข้อมูล (Security):

- ระบบการเข้าสู่ระบบปัจจุบันยังใช้การตรวจสอบข้อมูลพื้นฐาน (Email และ Phone Number) ซึ่งอาจไม่เพียงพอสำหรับระบบที่มีความสำคัญสูง ควรมีการเข้ารหัสที่ซับซ้อน (Password Hashing) และใช้ Token-based Authentication (เช่น JWT) เพื่อความปลอดภัยที่มากขึ้น
- ข้อมูลสิทธิ์ผู้ดูแลระบบ (Admin Credentials) บางส่วนยังมีการกำหนดค่าคงที่ (Hardcoded) ในซอร์สโค้ด ซึ่งควรย้ายไปเก็บใน Environment Variables หรือฐานข้อมูล

2. ประสิทธิภาพและการขยายตัว (Scalability):

- การบันทึก Logs ลงในไฟล์ JSON (File-based Logging) อาจทำให้เกิดปัญหาด้านประสิทธิภาพเมื่อไฟล์มีขนาดใหญ่ขึ้น ควรเปลี่ยนไปใช้การบันทึกลงฐานข้อมูลหรือบริการ Logging ภายนอก
- การเรียงลำดับ ID ใหม่ (ID Reordering) ทุกครั้งที่มีการลบข้อมูล เป็นกระบวนการที่ใช้ทรัพยากรสูงและอาจเกิดปัญหา Data Integrity หากมีผู้ใช้งานจำนวนมากพร้อมกัน

3. ฟีเจอร์การทำงาน (Feature Completeness):

- ระบบยังขาดฟังก์ชันการแจ้งเตือนแบบ Real-time (เช่น การแจ้งเตือนเมื่อการจองสำเร็จ)
- ยังไม่มีระบบการชำระเงิน (Payment Gateway) สำหรับการสมัครสมาชิกหรือจองคลาสแบบเสียค่าใช้จ่าย

10.1.4 แนวทางในการพัฒนาต่อยอด (Future Work)

เพื่อให้ระบบมีความสมบูรณ์และรองรับการใช้งานจริงได้ดียิ่งขึ้น ผู้จัดทำขอเสนอแนวทางในการพัฒนาต่อยอดในอนาคต ดังนี้:

1. ยกระดับมาตรการความปลอดภัย:

พัฒนาระบบยืนยันตัวตนให้รัดกุมยิ่งขึ้น โดยใช้มาตรฐาน OAuth 2.0 หรือ JWT และเข้ารหัสข้อมูลสำคัญในฐานข้อมูล รวมถึงการทำ HTTPS เพื่อความปลอดภัยในการรับส่งข้อมูล

2. เพิ่มระบบการจัดการทางการเงิน:

เชื่อมต่อกับ Payment Gateway (เช่น Stripe หรือ Omise) เพื่อรองรับการชำระเงินค่าสมาชิกและค่าบริการเทรนเนอร์ผ่านบัตรเครดิตหรือ QR Code

3. พัฒนา Mobile Application:

สร้างแอปพลิเคชันบนมือถือ (iOS/Android) เพื่อให้สมาชิกสามารถจองคลาสและตรวจสอบตารางเรียนได้สะดวกยิ่งขึ้น โดยใช้ API เดิมที่มีอยู่

4. ปรับปรุงระบบ Dashboard:

เพิ่มการแสดงผลข้อมูลเชิงลึก (Data Analytics) เช่น กราฟแสดงจำนวนผู้เข้าใช้บริการในแต่ละช่วงเวลา หรือยอดขายรายได้ประจำเดือน เพื่อช่วยในการตัดสินใจของผู้บริหาร

5. ระบบการแจ้งเตือน (Notification System):

เพิ่มระบบแจ้งเตือนผ่าน Email หรือ LINE Notify เพื่อแจ้งเตือนสมาชิกเมื่อใกล้ถึงเวลาจองคลาส หรือเมื่อสถานะสมาชิกใกล้หมดอายุ

Appendix A

Comprehensive System Installation and Deployment Guide

This appendix provides a meticulous, step-by-step manual designed to assist developers and system administrators in setting up the GymBro Management System from a clean slate. It covers every aspect of the installation process, from prerequisite software verification to production deployment, ensuring a seamless and error-free setup experience.

A.1 Prerequisites

Before initiating the installation, ensure that the following software components are installed and correctly configured on your development machine or server. These prerequisites are critical for the successful execution of the application.

A.1.1 Required Software

- **Node.js (Version 18.x or Higher):**
 - **Role:** The core runtime environment for executing the backend JavaScript code.
 - **Recommendation:** Use the Long-Term Support (LTS) version (e.g., v18.16.0) to ensure stability and compatibility with the latest packages.
 - **Verification:** Run `node -v` in your terminal. It should output something like `v18.16.0`.
- **PostgreSQL (Version 14.x or Higher):**
 - **Role:** The primary Relational Database Management System (RDBMS) used for persistent data storage.
 - **Reason:** Chosen for its robustness, ACID compliance, and advanced indexing capabilities which are essential for handling complex booking transactions.
 - **Verification:** Run `psql --version`. Ensure the service is running via `sudo service postgresql status` (Linux) or checking the Services panel (Windows).
- **Git (Version 2.x):**
 - **Role:** Version control system for cloning the project repository.
 - **Verification:** Run `git --version`.
- **Code Editor / IDE:**
 - **Recommendation:** Visual Studio Code (VS Code) is highly recommended due to its excellent support for JavaScript, debugging tools, and integrated terminal.
 - **Extensions:** Install "ESLint", "Prettier", and "DotENV" extensions for an enhanced development experience.

A.2 Environment Configuration

Proper configuration of environment variables is paramount for the security and functionality of the application. The system relies on a `.env` file to manage sensitive credentials and configuration settings.

A.2.1 Database Setup

1. Open your terminal or a database management tool like pgAdmin or DBeaver. 2. Connect to your PostgreSQL server:

```
1 psql -U postgres
```

3. Create a new database for the project:

```
1 CREATE DATABASE gymbro_db;
```

4. (Optional) Create a dedicated user for the application:

```
1 CREATE USER gym_admin WITH ENCRYPTED PASSWORD 'secure_password_123';  
2 GRANT ALL PRIVILEGES ON DATABASE gymbro_db TO gym_admin;
```

A.2.2 Setting up the .env File

1. Navigate to the root directory of the project. 2. Create a new file named .env. 3. Copy the following configuration template and fill in your specific details:

```
1 # Server Configuration  
2 PORT=3000  
3 NODE_ENV=development  
4  
5 # Database Connection (Local Development)  
6 DB_HOST=localhost  
7 DB_PORT=5432  
8 DB_NAME=gymbro_db  
9 DB_USER=gym_admin  
10 DB_PASS=secure_password_123  
11 DB_DIALECT=postgres  
12  
13 # Database Connection String (Alternative for Cloud Deployment)  
14 # DATABASE_URL=postgres://user:pass@host:port/dbname  
15  
16 # Security Keys  
17 JWT_SECRET=your_super_secret_jwt_key_32_chars_long  
18 SESSION_SECRET=another_long_random_string_for_sessions  
19  
20 # External Services (If applicable)  
21 # SMTP_HOST=smtp.gmail.com  
22 # SMTP_USER=your_email@gmail.com  
23 # SMTP_PASS=your_app_password
```

Variable Explanations:

- PORT: The port number on which the Express server will listen (default is 3000).
- DB_*: Connection details for the PostgreSQL database. Ensure these match the database you created.
- JWT_SECRET: A private key used to sign and verify JSON Web Tokens for user authentication. Keep this secure!

A.3 Installation Process

Follow these precise steps to get the application code on your machine and install all necessary dependencies.

A.3.1 Cloning the Repository

Open your terminal and navigate to your desired workspace directory. Run the following command:

```
1 git clone https://github.com/username/gymbro-management-system.git  
2 cd gymbro-management-system
```

A.3.2 Installing Dependencies

The project relies on several Node.js packages listed in `package.json`. Install them using npm (Node Package Manager):

```
1 npm install
```

Key Dependencies Explained:

- `express`: The web framework for handling HTTP requests.
- `sequelize`: The Object-Relational Mapper (ORM) for database interactions.
- `pg` & `pg-hstore`: PostgreSQL drivers for Sequelize.
- `ejs`: Embedded JavaScript templating engine for server-side rendering.
- `bcryptjs`: For hashing passwords securely.
- `jsonwebtoken`: For generating access tokens.
- `dotenv`: For loading environment variables.

A.4 Database Initialization

Before running the application, the database schema must be synchronized, and initial seed data needs to be populated.

A.4.1 Running Migrations

Sequelize migrations allow us to keep the database schema in sync with our application models. Run the following command to create the necessary tables (Users, Trainers, Bookings, etc.):

```
1 npx sequelize-cli db:migrate
```

Output should indicate that migration files were executed successfully.

A.4.2 Seeding Data

To test the application, it is helpful to have some initial data (e.g., default admin account, sample trainers, equipment list). Run:

```
1 npx sequelize-cli db:seed:all
```

This will populate the tables with the data defined in the `seeders` folder.

A.5 Execution and Deployment

A.5.1 Local Development

For development purposes, use `nodemon` to automatically restart the server whenever code changes are detected.

```
1 npm run dev
```

Access the application at `http://localhost:3000`.

A.5.2 Production Deployment

For a production environment, use a process manager like PM2 to ensure the application stays online and reloads automatically after crashes. 1. Install PM2 globally:

```
1 npm install pm2 -g
```

2. Start the application:

```
1 pm2 start server.js --name "gymbro-api"
```

3. Monitor the status:

```
1 pm2 status
2 pm2 logs gymbro-api
```

Appendix B

Core System Source Code Reference

This appendix serves as a technical reference for the core logic and architecture of the GymBro Management System. It presents critical sections of the source code, accompanied by detailed inline comments explaining the functionality, control flow, and design patterns used. These excerpts demonstrate the implementation of key features such as server configuration, data modeling, business logic, and dynamic frontend rendering.

B.1 Main Application Entry Point (server.js)

The `server.js` file is the heart of the backend application. It initializes the Express app, configures middleware (CORS, JSON parsing), establishes the database connection, and mounts the API routes.

```
1 // Import necessary modules
2 require("dotenv").config(); // Load environment variables from .env file
3 const express = require("express"); // Web framework
4 const cors = require("cors"); // Cross-Origin Resource Sharing middleware
5 const sequelize = require("./db"); // Database connection instance
6
7 // Import Models to ensure they are registered with Sequelize
8 const { Customers, Trainers, GymEquipments, TrainingSessions } = sequelize;
9
10 // Initialize the Express application
11 const app = express();
12
13 // --- Middleware Configuration ---
14
15 // 1. CORS Setup: Allow requests from the frontend application
16 app.use(
17   cors({
18     origin: "http://localhost:5500", // Allow only this origin
19     methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"], // Allowed HTTP methods
20     allowedHeaders: ["Content-Type", "Authorization"], // Allowed headers
21   })
22 );
23
24 // 2. Body Parsing: Parse incoming JSON payloads
25 app.use(express.json());
26
27 // --- Database Connection & Synchronization ---
28
29 /**
30  * Establishes connection to the PostgreSQL database and syncs models.
31  * Using 'alter: true' allows Sequelize to update tables to match models
32  * without dropping data (useful for development).
33  */
34 sequelize.authenticate()
35   .then(() => {
36     console.log("Database connection established successfully.");
37     return sequelize.sync({ alter: true });
38   })
39   .then(() => {
40     console.log("Database models synchronized.");
41   })
42   .catch((err) => {
43     console.error("Unable to connect to the database:", err);
44   });
45
46 // --- Route Definitions ---
47
48 // Login Endpoint
49 app.post("/api/login", async (req, res) => {
50   const { email, phone } = req.body;
```

```

51
52 try {
53   // Admin Hardcoded Check (for simplicity in this academic project)
54   if (email === "admin@gymbro.com" && phone === "123456") {
55     return res.json({
56       role: "admin",
57       user: { id: 0, fullName: "Administrator", email: email },
58       token: "mock-admin-token-123" // In production, generate a real JWT here
59     });
60   }
61
62   // Member Authentication
63   const user = await Customers.findOne({ where: { email } });
64
65   if (!user) {
66     return res.status(404).json({ error: "User not found" });
67   }
68
69   // Validate Phone Number (acting as password)
70   if (user.phone !== phone) {
71     return res.status(401).json({ error: "Invalid credentials" });
72   }
73
74   // Successful Login
75   res.json({
76     role: "user",
77     user: user,
78     token: `mock-user-token-${user.id}`
79   });
80
81 } catch (error) {
82   console.error("Login error:", error);
83   res.status(500).json({ error: "Internal server error" });
84 }
85 });
86
87 // Start the Server
88 const PORT = process.env.PORT || 3000;
89 app.listen(PORT, () => {
90   console.log(`Server is running on port ${PORT}`);
91 });

```

Listing B.1: Server Configuration and Setup

B.2 Sequelize Model Definition (models/TrainingSession.js)

This section illustrates how data models are defined using Sequelize. The `TrainingSession` model represents a booking event and includes foreign key associations to `Customers`, `Trainers`, and `Equipment`.

```

1 const { DataTypes } = require("sequelize");
2 const sequelize = require("../db"); // Import the connection instance
3
4 /**
5  * TrainingSession Model
6  * Represents a scheduled training event in the gym.
7  * Maps to the 'TrainingSessions' table in PostgreSQL.
8  */
9 const TrainingSession = sequelize.define(
10   "TrainingSessions",
11   {
12     // Primary Key
13     id: {
14       type: DataTypes.INTEGER,
15       autoIncrement: true,
16       primaryKey: true
17     },
18
19     // The starting date and time of the session
20     sessionDate: {
21       type: DataTypes.DATE,
22       allowNull: false,
23       validate: {
24         isDate: true,
25         isAfter: new Date().toISOString() // Validation: Cannot book in the past

```

```

26   }
27   },
28
29   // The ending date and time (calculated based on duration)
30   endDate: {
31     type: DataTypes.DATE,
32     allowNull: true
33   },
34
35   // Foreign Keys (Explicitly defined for clarity)
36   customerId: {
37     type: DataTypes.INTEGER,
38     allowNull: false,
39     references: { model: 'Customers', key: 'id' }
40   },
41   trainerId: {
42     type: DataTypes.INTEGER,
43     allowNull: true, // Can be null if it's a solo equipment usage
44     references: { model: 'Trainers', key: 'id' }
45   },
46   equipmentId: {
47     type: DataTypes.INTEGER,
48     allowNull: false,
49     references: { model: 'GymEquipments', key: 'id' }
50   },
51 },
52 {
53   tableName: "TrainingSessions",
54   timestamps: true, // Automatically manage createdAt and updatedAt
55   freezeTableName: true // Prevent pluralization of table name
56 }
57 );
58
59 // --- Model Associations ---
60 // These define the relationships for Eager Loading (JOINS)
61
62 // A Session belongs to one Customer
63 TrainingSession.belongsTo(sequelize.models.Customers, {
64   as: "customer",
65   foreignKey: "customerId"
66 });
67
68 // A Session belongs to one Trainer
69 TrainingSession.belongsTo(sequelize.models.Trainers, {
70   as: "trainer",
71   foreignKey: "trainerId"
72 });
73
74 // A Session uses one Equipment
75 TrainingSession.belongsTo(sequelize.models.GymEquipments, {
76   as: "equipment",
77   foreignKey: "equipmentId"
78 });
79
80 module.exports = TrainingSession;

```

Listing B.2: TrainingSession Model Definition

B.3 Core Logic Controller (Booking Conflict Detection)

This excerpt from the booking controller demonstrates the critical algorithm used to prevent double bookings. It checks for time overlaps for both trainers and equipment before confirming a transaction.

```

1  const { Op } = require("sequelize");
2  const { TrainingSessions } = require("../models");
3
4  /**
5   * createBooking
6   * Handles the creation of a new training session.
7   * Performs rigorous checks to ensure resource availability.
8   */
9  async function createBooking(req, res) {
10     const { customerId, trainerId, equipmentId, sessionDate, duration } = req.body;
11

```

```

12 // Start a database transaction to ensure atomicity
13 const t = await sequelize.transaction();
14
15 try {
16   // 1. Calculate Start and End times
17   const startTime = new Date(sessionDate);
18   const endTime = new Date(startTime.getTime() + duration * 60000); // duration in minutes
19
20   // 2. Conflict Detection Logic
21   // We need to find any existing session that overlaps with the requested time.
22   // Overlap condition: (StartA < EndB) AND (EndA > StartB)
23   const conflictWhereClause = {
24     [Op.or]: [
25       // Check Trainer Availability
26       {
27         trainerId: trainerId,
28         sessionDate: { [Op.lt]: endTime },
29         endDate: { [Op.gt]: startTime }
30       },
31       // Check Equipment Availability
32       {
33         equipmentId: equipmentId,
34         sessionDate: { [Op.lt]: endTime },
35         endDate: { [Op.gt]: startTime }
36       }
37     ]
38   };
39
40   // Query the database for conflicts
41   const existingConflict = await TrainingSessions.findOne({
42     where: conflictWhereClause,
43     transaction: t // Include in transaction
44   });
45
46   // 3. Handle Conflict
47   if (existingConflict) {
48     await t.rollback(); // Rollback transaction
49     return res.status(409).json({
50       error: "Conflict detected",
51       message: "The selected trainer or equipment is already booked for this time slot."
52     });
53   }
54
55   // 4. Create the Session
56   const newSession = await TrainingSessions.create({
57     customerId,
58     trainerId,
59     equipmentId,
60     sessionDate: startTime,
61     endDate: endTime
62   }, { transaction: t });
63
64   // 5. Commit Transaction
65   await t.commit();
66
67   // Return success response
68   res.status(201).json({
69     message: "Booking confirmed successfully",
70     data: newSession
71   });
72
73 } catch (error) {
74   // Handle unexpected errors
75   await t.rollback();
76   console.error("Booking Error:", error);
77   res.status(500).json({ error: "Failed to create booking" });
78 }
79 }

```

Listing B.3: Booking Controller with Conflict Detection

B.4 Frontend View Template (views/index.ejs)

This section displays the EJS template for the Admin Dashboard. It showcases how server-side data is injected into the HTML, how dynamic lists are rendered, and how the layout adapts to different user roles.

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <title>GymBro • Admin Dashboard</title>
6   <!-- Tailwind CSS for styling -->
7   <script src="https://cdn.tailwindcss.com"></script>
8   <!-- Lucide Icons -->
9   <script src="https://unpkg.com/lucide@latest"></script>
10 </head>
11 <body class="bg-gray-100 text-gray-900 min-h-screen flex">
12
13 <!-- Sidebar Navigation -->
14 <aside class="w-64 bg-blue-900 text-white flex-shrink-0 hidden md:flex flex-col">
15   <div class="h-20 flex items-center justify-center border-b border-blue-800">
16     <h1 class="text-2xl font-bold italic">Gym<span class="text-blue-400">Bro</span></h1>
17   </div>
18
19   <nav class="flex-1 py-6">
20     <ul>
21       <!-- Navigation Links -->
22       <li>
23         <a href="/admin/dashboard" class="flex items-center gap-4 px-6 py-3 bg-blue-800
24         ↪ border-l-4 border-blue-400">
25           <i data-lucide="layout-dashboard" class="w-5 h-5"></i>
26           <span>Dashboard</span>
27         </a>
28       </li>
29       <li>
30         <a href="/admin/customers" class="flex items-center gap-4 px-6 py-3 hover:bg-blue-800
31         ↪ transition">
32           <i data-lucide="users" class="w-5 h-5"></i>
33           <span>Customers</span>
34         </a>
35       </li>
36       <!-- Logout Button -->
37       <li class="mt-8 pt-4 border-t border-blue-800">
38         <button onclick="logout()" class="w-full text-left px-6 py-3 text-red-300
39         ↪ hover:text-white">
40           <i data-lucide="log-out" class="w-5 h-5 inline mr-3"></i> Logout
41         </button>
42       </li>
43     </ul>
44   </nav>
45 </aside>
46
47 <!-- Main Content Area -->
48 <main class="flex-1 p-8 overflow-y-auto">
49
50   <!-- Header -->
51   <header class="flex justify-between items-end mb-8 border-b-2 border-blue-900 pb-4">
52     <div>
53       <h2 class="text-3xl font-extrabold text-blue-900 uppercase">Overview</h2>
54       <p class="text-gray-600">Daily Statistics and Activity</p>
55     </div>
56     <button class="bg-blue-600 text-white px-4 py-2 rounded shadow hover:bg-blue-700">
57       + Create Report
58     </button>
59   </header>
60
61   <!-- Statistics Cards Grid -->
62   <div class="grid grid-cols-1 md:grid-cols-4 gap-6 mb-8">
63     <!-- Dynamic Data Injection from Server -->
64     <!-- Card 1: Total Customers -->
65     <div class="bg-white p-6 rounded shadow-md border-l-4 border-blue-500">
66       <h3 class="text-gray-500 text-sm font-bold uppercase">Total Customers</h3>
67       <p class="text-3xl font-bold text-blue-900 mt-2"><%= stats.totalCustomers %></p>
68     </div>
69
70     <!-- Card 2: Active Trainers -->
```



```

68     <div class="bg-white p-6 rounded shadow-md border-l-4 border-green-500">
69         <h3 class="text-gray-500 text-sm font-bold uppercase">Active Trainers</h3>
70         <p class="text-3xl font-bold text-green-900 mt-2"><%= stats.totalTrainers %></p>
71     </div>
72
73     <!-- ... other cards ... -->
74 </div>
75
76 <!-- Recent Activity Table -->
77 <div class="bg-white rounded shadow-lg overflow-hidden">
78     <div class="px-6 py-4 bg-blue-900 text-white font-bold">
79         Today's Sessions
80     </div>
81     <table class="w-full text-left border-collapse">
82         <thead>
83             <tr class="bg-gray-200 text-gray-700 text-sm uppercase">
84                 <th class="px-6 py-3">Time</th>
85                 <th class="px-6 py-3">Customer</th>
86                 <th class="px-6 py-3">Trainer</th>
87                 <th class="px-6 py-3">Activity</th>
88                 <th class="px-6 py-3">Status</th>
89             </tr>
90         </thead>
91         <tbody class="divide-y divide-gray-200">
92             <!-- Loop through sessions array passed from controller -->
93             <% if (sessions && sessions.length > 0) { %>
94                 <% sessions.forEach(function(session) { %>
95                     <tr class="hover:bg-gray-50 transition">
96                         <td class="px-6 py-4 font-mono text-blue-600">
97                             <%= new Date(session.sessionDate).toLocaleTimeString('th-TH', {hour:
98     ↪ '2-digit', minute:'2-digit'}) %>
99                         </td>
100                        <td class="px-6 py-4 font-bold"><%= session.customer.fullName %></td>
101                        <td class="px-6 py-4"><%= session.trainer ? session.trainer.trainerName : '-'
102     ↪ %></td>
103                        <td class="px-6 py-4 text-gray-500"><%= session.equipment.equipmentName %></td>
104                        <td class="px-6 py-4">
105                            <span class="px-2 py-1 text-xs rounded bg-green-100 text-green-800 font-bold">
106                                CONFIRMED
107                            </span>
108                        </td>
109                    </tr>
110                <% }); %>
111            <% } else { %>
112                <!-- Empty State -->
113                <tr>
114                    <td colspan="5" class="px-6 py-8 text-center text-gray-500 italic">
115                        No sessions scheduled for today.
116                    </td>
117                </tr>
118            <% } %>
119        </tbody>
120    </table>
121 </div>
122
123 </main>
124
125 <script>
126     // Initialize Lucide Icons
127     lucide.createIcons();
128
129     // Logout Logic
130     function logout() {
131         if(confirm('Are you sure you want to logout?')) {
132             localStorage.removeItem('gymbro_token');
133             window.location.href = '/login';
134         }
135     }
136 </script>
137 </body>
138 </html>

```

Listing B.4: Admin Dashboard EJS Template